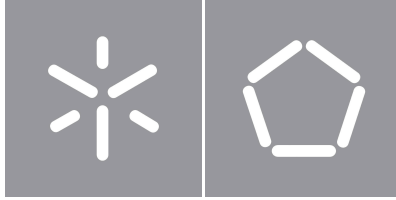


University of Minho
School of Engineering

Eduardo André Silva Cunha

**Dynamic Attack Graph Generation
Using Threat Intelligence and
Threat Modeling**



University of Minho
School of Engineering

Eduardo André Silva Cunha

**Dynamic Attack Graph Generation
Using Threat Intelligence and
Threat Modeling**

Master's Dissertation in Informatics Engineering

Dissertation supervised by
João Marco Cardoso da Silva

Copyright and Terms of Use for Third Party Work

This dissertation reports on academic work that can be used by third parties as long as the internationally accepted standards and good practices are respected concerning copyright and related rights.

This work can thereafter be used under the terms established in the license below.

Readers needing authorization conditions not provided for in the indicated licensing should contact the author through the RepositóriUM of the University of Minho.

License granted to users of this work:



CC BY

<https://creativecommons.org/licenses/by/4.0/>

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I confirm that I have not used plagiarism or any form of undue use of information or falsification of results along the process leading to its elaboration.

I further declare that I have fully acknowledged the Code of Ethical Conduct of the University of Minho.

University of Minho, Braga, September 2026

Eduardo André Silva Cunha

Abstract

Modern organizations face a volatile cybersecurity landscape in which vulnerabilities are disclosed daily, infrastructures evolve and adversaries adapt their tactics. Attack graphs model adversarial behavior across multiple stages and feasible attack paths in networked systems. Each graph derives from the configuration, vulnerability and adversarial facts available for a given environment and represents its state at that time. Refreshing inputs alone does not establish whether changes are relevant to the modeled environment or alter the graph structure. Threat modeling facts are also not systematically integrated into attack graph generation. Consequently, the integration of evolving threat intelligence and threat modeling into attack graph generation remains insufficiently addressed.

This dissertation proposes a dynamic framework that integrates evolving threat intelligence and threat modeling into attack graph generation and maintenance. It correlates threat intelligence with CPE based asset inventories to analyze assets that cannot or should not be scanned and translates structured threat modeling outputs into attack graph facts. An automated open source intelligence (OSINT) pipeline collects and normalizes quantitative metrics and qualitative intelligence, combining them with system configurations, vulnerabilities and adversarial assumptions to derive attack paths with MulVAL. Selective regeneration and contextual reprioritization determine whether changed facts require a new graph or only updated path priorities. A post-processing stage combines CVSS, EPSS and KEV data with asset criticality to calculate and rank path risk scores. Annotated graphs and host level summaries support exposure identification and prioritization. Case studies evaluate functional behavior, dynamic updates and usefulness for proactive cyber defense. Overall, the framework extends point in time attack graph analyses with an intelligence informed maintenance workflow.

Keywords attack graphs, threat intelligence, MulVAL, OSINT, risk assessment, cybersecurity

Resumo

As organizações enfrentam um cenário de cibersegurança volátil, no qual são divulgadas vulnerabilidades diariamente, as infraestruturas evoluem e os adversários adaptam as suas táticas. Os grafos de ataque modelam o comportamento adversarial em múltiplas etapas e os caminhos de ataque viáveis em sistemas em rede. Cada grafo deriva dos factos de configuração, vulnerabilidades e pressupostos adversariais disponíveis para um ambiente e representa o seu estado nesse momento. A atualização dos dados de entrada não determina se as alterações são relevantes para o ambiente modelado ou alteram a estrutura do grafo. Os factos de threat modeling também não são integrados sistematicamente na geração de grafos de ataque. Consequentemente, a integração de threat intelligence em evolução e de threat modeling na geração de grafos de ataque permanece insuficientemente tratada.

Esta dissertação propõe uma plataforma dinâmica que integra threat intelligence em evolução e threat modeling na geração e manutenção de grafos de ataque. Correlaciona a threat intelligence com inventários de ativos baseados em CPEs para analisar ativos que não podem ou não devem ser submetidos a scans e traduz resultados estruturados de threat modeling em factos para a geração de grafos de ataque. Uma cadeia automatizada de inteligência de fontes abertas (OSINT) recolhe e normaliza métricas quantitativas e inteligência qualitativa, combinando-as com configurações do sistema, vulnerabilidades e pressupostos adversariais para derivar caminhos de ataque com o MulVAL. A regeneração seletiva e a repriorização contextual determinam se factos alterados exigem um novo grafo ou apenas a atualização das prioridades dos caminhos. Uma etapa de pós-processamento combina dados de CVSS, EPSS e KEV com a criticidade dos ativos para calcular e ordenar as pontuações de risco dos caminhos de ataque. Grafos anotados e resumos ao nível dos hosts apoiam a identificação e priorização de exposições. Casos de estudo avaliam o comportamento funcional, as atualizações dinâmicas e a utilidade para a ciberdefesa proativa. Em síntese, a plataforma complementa análises pontuais de grafos de ataque com um fluxo de manutenção orientado por inteligência.

Palavras-chave grafos de ataque, threat intelligence, MulVAL, OSINT, avaliação de risco, cibersegurança

Contents

1	Introduction	1
1.1	Objectives	2
1.2	Contributions	3
1.3	Structure	4
2	Background and Related Work	5
2.1	Security Threats and Vulnerabilities	5
2.2	Threat Modeling Frameworks	6
2.2.1	STRIDE Framework	7
2.2.2	PASTA Framework	8
2.2.3	OCTAVE Framework	9
2.2.4	VAST Framework	10
2.2.5	Comparison of Threat Modeling Frameworks	12
2.3	Fundamentals of Attack Trees and Attack Graphs	13
2.3.1	Attack Tree	13
2.3.2	Attack Graph	15
2.3.3	Attack Trees vs Attack Graphs	16
2.4	Vulnerability Databases and Classification Systems	17
2.4.1	CVE (Common Vulnerabilities and Exposures)	17
2.4.2	CWE (Common Weakness Enumeration)	19
2.4.3	CPE (Common Platform Enumeration)	19
2.4.4	Limitations of Severity-Based Vulnerability Scoring	20
2.4.4.1	EPSS (Exploit Prediction Scoring System)	20
2.4.4.2	KEV (Known Exploited Vulnerabilities)	21
2.5	Open-Source Intelligence (OSINT)	22
2.6	Cyber Threat Intelligence (CTI)	22
2.6.1	STIX	23
2.7	Asset Criticality and Risk Assessment	23
2.7.1	Asset Criticality Assessment	24
2.7.2	Risk Assessment Methodologies	24

2.7.2.1	Model-Driven Approaches	25
2.7.2.2	Analysis-Driven Approaches	26
2.7.2.3	Vulnerability-Driven Approaches	26
2.7.3	DREAD Framework	26
2.8	Chapter Summary	28
3	Foundations and Selected Approaches to Attack Graph Generation	29
3.1	Attack Graphs	29
3.1.1	Attack Tree (AT)	30
3.1.2	State Graph (SG)	30
3.1.3	Exploit Dependency Graph (EDG)	30
3.1.4	Logical Attack Graph (LAG)	31
3.1.5	Bayesian Attack Graph (BAG)	32
3.1.6	Multiple Prerequisite Attack Graph (MPAG)	32
3.1.7	Complementary Attack Graph Frameworks	33
3.2	MulVAL	34
3.2.1	Architecture and Data Integration	35
3.2.2	Reasoning Mechanism	36
3.2.3	Limitations	38
3.3	SeaMonster	38
3.3.1	Vulnerability Causes Viewpoint	39
3.3.2	Threats and Attacks Viewpoint	40
3.3.3	Countermeasures Viewpoint	40
3.3.4	Limitations	41
3.4	SAGE	41
3.4.1	Limitations	43
3.5	Comparative Analysis of Attack Graph Generation Approaches	44
3.6	Chapter Summary	46
4	Design and Implementation	47
4.1	Problem Statement	47
4.2	Conceptual Solution	48
4.2.1	Selection of the Logical Reasoning Engine	49
4.2.2	Selection of the Threat Modeling Framework	50
4.2.3	Design Goals	51
4.2.4	Conceptual Architecture of the Framework	52
4.2.5	Threat Intelligence Ingestion Layer	53
4.2.6	Environment Correlation and Threat Modeling Integration Layer	53

4.2.7	Change Detection and Model Orchestration	53
4.2.8	Attack Graph Generation	54
4.2.9	Post-Processing and Dynamic Risk Analysis	54
4.3	Implemented Framework	54
4.3.1	System Overview and Deployment	55
4.3.2	Threat Intelligence Database	57
4.3.3	Threat Intelligence Ingestion	58
4.3.4	Environment Correlation	59
4.3.5	Orchestration	61
4.3.5.1	Attack Graph Generation	62
4.3.6	Post-Processing and Dynamic Risk Scoring	62
4.3.6.1	Delta Analysis and Decision Support	65
4.3.6.2	Host-Level Summary Graph	65
4.4	Chapter Summary	67
5	Case Study and Evaluation	68
5.1	Experimental Setup	68
5.2	Scenario 1: Client-Side Exposure	70
5.2.1	Delta Analysis: Stable Baseline	73
5.3	Scenario 2: Multi-Tier E-commerce Deployment	75
5.4	Scenario 3: Enterprise Healthcare Information System	78
5.5	Evaluation of Dynamic Behavior	81
5.6	Comparative Discussion	82
5.7	Chapter Summary	84
6	Conclusion and Future Work	85
6.1	Summary of Contributions	86
6.2	Future Work	87
6.2.1	Richer cyber threat intelligence ingestion	87
6.2.2	Adoption of CVSS 4.0	87
6.2.3	Richer exploit consequences	87
6.2.4	Automated translation of threat models	87
6.2.5	Learned asset criticality and CIA prioritization	88
6.2.6	Risk-threshold filtering and risk acceptance	88
6.2.7	Graceful handling of the no-attack-path outcome	88
6.2.8	Scalability evaluation and optimization	88
6.2.9	Operational integration and visualization	89
	Bibliography	90

A Risk Scoring Configuration Reference	97
B STRIDE-to-MulVAL Mapping	99
C Threat Intelligence Database Schema	101
D Representative Scenario Input Artifacts	103
E STIX Campaign Object Example	107
F Detailed Annotated Attack Graph Output	108
F.1 Scenario 3: Enterprise Healthcare Information System	109

List of Figures

Figure 1	Comparison between a Data Flow Diagram (DFD) and a Process Flow Diagram (PFD) on a login flow	11
Figure 2	Example of an Attack Tree	14
Figure 3	Example of an Attack Graph	15
Figure 4	Evolution of AG Representations	30
Figure 5	Example of EDG	31
Figure 6	Example of BAG	32
Figure 7	Example of MPAG	33
Figure 8	Example of a MulVAL-generated attack graph [1]	38
Figure 9	An alert-driven attack graph of data exfiltration over <i>remoteware-cl</i> (IDs are state identifiers, capturing context) [2]	43
Figure 10	Conceptual architecture of the proposed framework.	48
Figure 11	End-to-end architecture of the proposed pipeline, from environment and threat intelligence inputs to the annotated attack graph and risk report.	56
Figure 12	Logical model of the Threat Intelligence Database, organized around the CVE identifier.	57
Figure 13	Threat intelligence ingestion: CVE/CPE and KEV data from the VulnCheck API and EPSS scores from FIRST.org.	58
Figure 14	Environment correlation: asset CPEs are matched against the database to inject vulnerability facts, optionally enriched with STRIDE derived threats.	59
Figure 15	Change detection logic of the orchestrator: MulVAL is executed when a previous graph or scenario is absent, or when the scenario fact set differs after comments and formatting are ignored.	61
Figure 16	Top-level view of the post-processing service: its file and database inputs and the risk report and annotated graphs it produces.	63
Figure 17	Illustrative detailed attack graph.	65
Figure 18	Illustrative host-level summary graph generated by the post-processing service.	66

Figure 19	Scenario 1 topology: three independent victim hosts browsing outbound to an attacker-controlled internet.	71
Figure 20	Scenario 1 detailed annotated attack graph produced by the post-processing layer, with nodes coloured by computed risk level.	72
Figure 21	Scenario 1 host-level summary graph: each victim is reached directly from the internet, annotated with its worst-case path risk and exploited CVEs.	72
Figure 22	Scenario 2 topology: a multi-tier e-commerce deployment with a DMZ proxy, an application server, a data tier and a back-office workstation.	76
Figure 23	Scenario 2 detailed annotated attack graph produced by the post-processing layer, with nodes coloured by computed risk level.	77
Figure 24	Scenario 2 host-level summary graph: lateral movement from the DMZ proxy through the application server into the data tier, with a separate workstation threat path.	78
Figure 25	Scenario 3 topology: an eight-host healthcare information system with a DMZ load balancer, an application tier, an identity provider, a data tier and a clinician workstation.	79
Figure 26	Scenario 3 host-level summary graph: deep lateral movement from the DMZ to the API server, which fans out into the identity and data tiers.	80
Figure 27	Scenario 3 annotated attack graph produced by the post-processing layer, with threat intelligence enrichment and per-node risk scores.	109

List of Tables

Table 1	Comparison of Threat Modeling Frameworks	12
Table 2	Comparison of Attack Graph Generation Tools	34
Table 3	Comparative analysis of MulVAL, SeaMonster and SAGE	45
Table 4	Scenario 2: per asset criticality and confidentiality (C), integrity (I) and availability (A) priorities.	77
Table 5	Scenario 3: representative highest numerically scored attack paths and their threat intelligence drivers. Paths without CVSS are excluded from this numerical ranking.	81
Table 6	Effect of asset reclassification on the highest-ranked path of Scenario 1, computed from its measured base risk. Reclassification re-ranks the path without graph regeneration.	82
Table 7	Asset criticality factor α per named level.	97
Table 8	Global scoring and enumeration parameters.	98
Table 9	Default implementation mapping of STRIDE categories to MulVAL exploitation primitives. Unrecognized categories default to remoteExploit.	100
Table 10	Default implementation mapping of declared threat impacts to MulVAL consequences. Unrecognized impacts default to privEscalation.	100

List of Listings

Listing 1	Simplified MulVAL attack graph specification for a multi-hop attack scenario [1]	37
Listing 2	Illustrative fragment of the enriched scenario.P produced by the environment correlation layer.	60
Listing 3	Current Scenario 1 risk report: the structural summary and the single highest risk path.	74
Listing 4	Scenario 1 delta report for an unchanged repeat execution.	75
Listing 5	Threat Intelligence Database schema (PostgreSQL).	102
Listing 6	Scenario 1 asset_cpe_mapping.json.	104
Listing 7	Scenario 1 stride_definition.json (one of three analogous threats shown). .	105
Listing 8	Enriched MulVAL scenario (scenario.P) for Scenario 1.	106
Listing 9	Example of a STIX 2.1 Campaign object [3].	107

Acronyms

AG Attack Graph.

AT Attack Tree.

BAG Bayesian Attack Graph.

CIA Confidentiality, Integrity and Availability.

CISA Cybersecurity and Infrastructure Security Agency.

CNA CVE Numbering Authority.

CPE Common Platform Enumeration.

CTI Cyber Threat Intelligence.

CVE Common Vulnerabilities and Exposures.

CVSS Common Vulnerability Scoring System.

CWE Common Weakness Enumeration.

DFD Data Flow Diagram.

DMZ Demilitarized Zone.

DoS Denial of Service.

DREAD Damage, Reproducibility, Exploitability, Affected Users and Discoverability.

EDG Exploit Dependency Graph.

EPSS Exploit Prediction Scoring System.

FTA Fault Tree Analysis.

FTP File Transfer Protocol.

GPL GNU General Public License.

IDS Intrusion Detection System.

IOC Indicator of Compromise.

KEV Known Exploited Vulnerabilities.

LAG Logical Attack Graph.

MPAG Multiple Prerequisite Attack Graph.

NVD National Vulnerability Database.

OASIS Organization for the Advancement of Structured Information Standards.

OCTAVE Operationally Critical Threat, Asset and Vulnerability Evaluation.

OSINT Open-Source Intelligence.

OVAL Open Vulnerability and Assessment Language.

PASTA Process for Attack Simulation and Threat Analysis.

PFD Process Flow Diagram.

RCA Root Cause Analysis.

RSA Rivest-Shamir-Adleman.

RSH Remote Shell.

SAGE IntruSion alert-driven Attack Graph Extractor.

SAND Sequential AND.

SG State Graph.

SIEM Security Information and Event Management.

S-PDFA Suffix-based Probabilistic Deterministic Finite Automaton.

SSH Secure Shell.

STIX Structured Threat Information Expression.

STRIDE Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service and Elevation of Privilege.

TVA Topological Vulnerability Analysis.

UML Unified Modeling Language.

URI Uniform Resource Identifier.

VAST Visual, Agile and Simple Threat.

VCG Vulnerability Cause Graph.

XSS Cross-Site Scripting.

Glossary

Attack A deliberate attempt to compromise a system by exploiting one or more vulnerabilities.

Datalog A declarative logic programming language and subset of Prolog that supports efficient, rule-based reasoning.

Exploit The concrete mechanism, such as code, a sequence of actions or the abuse of legitimate functionality, through which a vulnerability is leveraged against a system.

Hyper-alert A group of intrusion alerts from the same attacker and victim pair and attack stage within a fixed time window, used by SAGE to derive episodes.

Monotonicity assumption The assumption that the successful execution of an attack step never invalidates the preconditions of other steps, allowing previously satisfied conditions to remain valid.

MulVAL An open-source, logic-based framework that automatically generates logical attack graphs by reasoning over network configuration and vulnerability data expressed as Datalog rules.

Nessus A widely used vulnerability scanner that identifies misconfigurations, missing patches and known weaknesses in networked systems.

Prolog A declarative logic programming language used to express facts and rules for automated reasoning.

SeaMonster A graphical security modeling tool that relates the causes of vulnerabilities, potential attack paths and corresponding countermeasures through multiple complementary viewpoints.

State space explosion The exponential growth in the number of global system states as hosts and vulnerabilities increase, which makes the explicit enumeration of attack graphs computationally infeasible.

Threat Any circumstance, event or entity with the potential to cause harm to a system or its assets.

Vulnerability A flaw or weakness in a system's design, implementation, configuration or operation that can be exploited by a threat.

XSB A logic programming engine whose tabled resolution mechanism reuses intermediate results and prevents infinite recursion, used by MulVAL to construct attack traces in polynomial time.

Chapter 1

Introduction

Modern organizations face an increasingly complex and volatile cybersecurity landscape. Cyberweapons are becoming more accessible, new vulnerabilities are disclosed daily, and digital infrastructures are growing increasingly complex and interconnected [4]. To effectively defend against adaptive adversaries, security professionals must be capable of continuously acquiring, integrating and analyzing threat data in order to detect emerging trends and adjust defensive measures in real time.

Threat data can be categorized into quantitative metrics, including attack frequencies and exploit costs and qualitative intelligence, such as Indicator of Compromises (IOCs) and information about emerging vulnerabilities or adversarial behaviors. Organizations acquire such data from diverse external sources, such as threat intelligence feeds, Open-Source Intelligence (OSINT), public security reports and media monitoring as well as from internal systems, including Security Information and Event Management (SIEM) platforms [4]. Nevertheless, despite the dynamic nature of this information, most threat modeling and analysis methods used in practice remain static, with limited ability to integrate and interpret continuously evolving threat intelligence [4].

Among existing security modeling approaches, attack trees and attack graphs are well-established graphical formalisms for analyzing potential attack scenarios. They are widely employed to visualize attack paths, support security evaluations and perform quantitative risk assessments. Although the facts supplied to these models can be refreshed with updated vulnerability information or scan results, each inference execution produces a representation of the input available at that time [4]. MulVAL can refresh vulnerability repository data and process scanner output before a new execution [5], but this does not determine whether newly available information is relevant to the modeled environment, changes the attack graph structure or only affects the priority of existing paths. Separately, threat modeling outputs are not systematically translated into facts used in automated attack graph generation.

This dissertation proposes a framework that addresses these limitations. It correlates quantitative and qualitative threat intelligence with CPE based asset inventories, enabling analysis of assets for which vulnerability scans cannot be performed or are inappropriate. It translates structured threat modeling outputs into MulVAL facts, allowing adversarial assumptions identified during threat modeling to affect attack graph generation. A post-processing stage combines CVSS, EPSS, KEV status and asset criticality to score and rank the feasible paths produced by MulVAL. The resulting workflow supports repeated analysis of the same environment as relevant threat intelligence and threat modeling facts change, regenerating the graph only when the modeled attack surface changes and otherwise recalculating the risk scores and priorities of existing paths.

1.1 Objectives

The primary objective of this dissertation is to design and implement a dynamic framework that integrates continuously evolving threat intelligence and threat modeling with attack graphs to enhance cybersecurity modeling, situational awareness and decision making.

To achieve this goal, the dissertation pursues the following specific objectives:

1. Develop a conceptual and architectural model defining how dynamic threat intelligence (both quantitative and qualitative) and threat modeling outputs can be systematically linked to attack graphs, allowing these models to evolve in parallel with the changing threat landscape;
2. Design and implement an automated OSINT acquisition and integration pipeline for collecting and normalizing threat intelligence from external sources, supporting periodic updates of the modeling environment;
3. Develop mechanisms for selective model regeneration and contextual reprioritization, so that new intelligence or threat modeling facts that change the modeled attack surface trigger a new attack graph, while updates that affect only EPSS, KEV status or asset criticality recalculate the risk scores and priorities of existing paths without structural recomputation.
4. Create post-processing, visualization and analytical components that enrich attack paths with CVSS, EPSS, KEV status and asset criticality, calculate risk scores and summarize changes in risk across successive analyses.

5. Evaluate the proposed framework through simulated case studies that examine its functional behavior, dynamic update mechanisms and analytical usefulness in representative scenarios.

1.2 Contributions

This dissertation advances cybersecurity modeling by extending conventional attack graph snapshot analysis with a workflow that integrates evolving threat intelligence, structured threat modeling outputs and contextual risk scoring. Its contributions are summarized as follows:

1. **Dynamic Integration of Threat Intelligence into Attack Models:** This dissertation proposes a conceptual and architectural framework for the continuous integration of both quantitative and qualitative threat intelligence into attack graphs. The framework correlates vulnerability information with asset CPEs, providing an alternative source of vulnerability facts when scan output is unavailable, and determines whether changes in the resulting facts affect the modeled attack surface. The objective is to maintain attack representations that reflect the threat information relevant to the environment.
2. **Selective Model Regeneration and Contextual Reprioritization:** This work introduces a mechanism that distinguishes updates that change environment-specific vulnerability or threat modeling facts from updates that affect only their risk context. The former causes MulVAL to regenerate the attack graph, whereas the latter are handled during post-processing, which enriches the existing graph with CVSS, EPSS, KEV status and asset criticality to calculate and rank path risk scores. This distinction complements the data refresh capabilities of MulVAL adapters and avoids unnecessary graph generation when EPSS, KEV status or asset criticality changes.
3. **Integration of Threat Modeling into Attack Graph Generation:** This work translates structured STRIDE threat modeling outputs into logical facts compatible with MulVAL. It validates modeled assets and threat categories, maps each threat category to a MulVAL exploitation primitive and each declared impact to a consequence, allowing adversarial assumptions identified during threat modeling to participate in attack graph generation alongside vulnerability facts. This operationalizes the connection between qualitative threat modeling and automated reasoning within a reproducible pipeline.

In summary, this dissertation investigates a dynamic modeling paradigm for attack graphs within continuously changing threat environments. It aims to extend existing theoretical

foundations by incorporating temporal and adaptive dimensions, while laying the basis for future research on intelligence-driven, self-evolving cybersecurity modeling.

1.3 Structure

This dissertation is organized into six chapters that progressively build towards the proposed framework. Chapter 1 introduces the problem, motivation and objectives that guide the work. Chapter 2 presents the background concepts required to understand the rest of the dissertation, while Chapter 3 reviews the foundations and selected approaches to attack graph generation that are most relevant to this work. Chapter 4 describes the designed solution, detailing its architecture and implementation, and Chapter 5 evaluates it through representative case studies. Finally, Chapter 6 summarizes the main conclusions and outlines directions for future work.

Chapter 2

Background and Related Work

This chapter introduces foundational concepts of security modeling and risk analysis. It establishes the background needed to understand the work developed in Chapter 3 through Chapter 6.

2.1 Security Threats and Vulnerabilities

In cybersecurity, a threat refers to any circumstance, event or entity with the potential to cause harm to a system or its assets [6]. Threats may arise from diverse sources, including natural events (e.g., environmental hazards affecting data centers), human actions (whether accidental or malicious) and technical failures or system malfunctions. Threats are typically characterized by their potential impact and the conditions under which they may occur [6], particularly when they endanger core security properties such as confidentiality, integrity and availability.

For a threat to materialize, it must be able to leverage weaknesses within the target system. These weaknesses are known as vulnerabilities. A vulnerability is a flaw or weakness in a system's design, implementation, configuration or operation that can be exploited by a threat [6]. Vulnerabilities may result from software defects, insecure configurations, architectural limitations or inadequate operational practices.

An exploit represents the concrete mechanism through which a vulnerability is leveraged [6]. In cybersecurity terms, exploits may take the form of software code, sequences of actions or the abuse of legitimate system functionality. They provide the practical means by which a threat is transformed into an operational action against a system. When an exploit is successfully applied, it enables an attack, defined as a deliberate attempt to compromise a system by exploiting one or more vulnerabilities [6].

2.2 Threat Modeling Frameworks

Threat modeling frameworks are systematic and structured methodologies designed to identify, analyze and mitigate potential security threats within a system, application or process. Their primary objective is to anticipate and address security risks early in the design and development lifecycle, thereby enabling the proactive implementation of countermeasures rather than relying on reactive incident response [7]. By adopting such structured approaches, organizations can gain a clearer understanding of how potential adversaries might exploit vulnerabilities to compromise critical assets, assess the likelihood and potential impact of these threats and prioritize mitigations accordingly.

The threat modeling process typically consists of four high-level stages:

1. Defining scope and assets: Establishing the system boundaries, identifying critical assets and determining security objectives;
2. Identifying threats: Using a threat categorization methodology to systematically enumerate possible threats relevant to the system's architecture and data flows;
3. Determining potential vulnerabilities: Analyzing how identified threats could exploit weaknesses within the system's components or processes;
4. Proposing mitigations or controls: Defining countermeasures to eliminate or reduce identified risks. Such countermeasures can be guided by established threat countermeasure mapping lists [8].

Once potential mitigations are identified, prioritization becomes a key consideration. Organizations may adopt different prioritization schemes based on their operational context and risk tolerance [7], balancing factors such as the likelihood of attack, potential damage and the complexity or cost of implementation.

Over the years, a variety of threat modeling frameworks have been developed to support these processes across diverse domains and levels of system complexity. Among the most widely recognized are Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service and Elevation of Privilege (STRIDE), Process for Attack Simulation and Threat Analysis (PASTA), Operationally Critical Threat, Asset and Vulnerability Evaluation (OCTAVE) and Visual, Agile and Simple Threat (VAST). Each framework offers a distinct analytical perspective, ranging from technically focused, developer-oriented approaches to more strategic, business-centric methodologies. This diversity enables organizations to select, adapt or combine frameworks according to their specific architecture, security maturity and operational objectives.

2.2.1 STRIDE Framework

The STRIDE model, introduced by Microsoft [9], is a foundational framework for identifying security threats during software design and analysis. It defines six categories of potential threats, each targeting specific attack vectors that can compromise an application's confidentiality, integrity or availability. The categories are as follows:

1. Spoofing - Spoofing is the act of forging an identity to gain unauthorized access to systems or data. Attackers impersonate trusted entities (such as users, devices or services) to deceive systems into accepting fraudulent requests. Common examples include phishing emails, fake login portals and forged network identities that trick victims or systems into revealing sensitive information or granting access [10]
2. Tampering - Tampering refers to the unauthorized modification of data, configuration files or system logs. Attackers may alter data to hide their tracks, introduce vulnerabilities or manipulate system behavior for malicious purposes [10]
3. Repudiation - Repudiation occurs when an actor performs an action, whether legitimate or malicious, and is later able to deny responsibility for it. It arises when a system lacks sufficient logging, tamper resistant audit trails or reliable mechanisms to bind actions to the entities that performed them, which undermines accountability and complicates forensic investigation [10]
4. Information Disclosure - Information Disclosure refers to the exposure of sensitive or confidential information. Such leakage may result from insecure system configurations, overly verbose error messages, improperly protected storage or unsecured backups. The disclosure of this information can lead to privacy violations, regulatory non-compliance and loss of competitive advantage [10]
5. Denial of Service (DoS) - Denial of Service attacks aim to disrupt the normal functioning of a service by overwhelming it with traffic or requests, thereby denying legitimate users access. These attacks can target network resources, application endpoints or backend systems [10]
6. Elevation of Privilege - Elevation of privilege occurs when a user or process gains access to functionalities or data beyond their authorization. This can happen due to missing authorization checks, configuration errors or vulnerabilities in the application's logic [10].

Applying STRIDE involves decomposing a system into its key components, analyzing each for potential threats and implementing appropriate mitigations. This process is repeated iteratively until the remaining risks are deemed acceptable [11].

STRIDE's structured taxonomy promotes consistency in threat identification and clear communication across teams. It is nonetheless subject to several criticisms: it focuses primarily on technical, software level threats and offers little support for capturing broader business or organizational risk, and its fixed set of six threat categories does not adapt to the particular context of each system. Despite these limitations, it remains a foundational tool in practical threat modeling [12].

2.2.2 PASTA Framework

The PASTA is a risk-centric threat modeling framework proposed by *Tony Uceda Vélez* in 2012 [13]. PASTA incorporates multiple layers of abstraction, ranging from high-level business objectives to low-level technical attack vectors.

PASTA suits systems already in operation as well as new systems still in the design phase. The framework consists of seven sequential stages, each containing multiple activities that collectively support comprehensive threat and risk analysis [13].

The seven stages of the PASTA framework are as follows:

1. **Define Objectives** - This stage focuses on understanding the business context of the system. It includes identifying business objectives, defining security and compliance requirements and performing a business impact analysis to determine the potential consequences of security failures [13]
2. **Define Technical Scope** - In this stage, the technical boundaries of the system are established. This includes capturing the scope of the technical environment and identifying infrastructure, application, software components and their dependencies [13]
3. **Application Decomposition** - The system is broken down into its functional components to better understand how it operates. Key activities include identifying use cases, application entry points, trust levels, actors, assets, services, roles and data sources. Data Flow Diagrams (DFDs) and trust boundaries are also defined during this stage [13]
4. **Threat Analysis** - This stage identifies potential threats by analyzing probabilistic attack scenarios. It includes the analysis of security events and the correlation of threat intelligence data to better understand likely and emerging threats [13]
5. **Vulnerability and Weakness Analysis** - Existing vulnerabilities and weaknesses are examined by querying vulnerability reports and issue tracking systems. Threats are mapped to known vulnerabilities and design flaws are analyzed through use and abuse cases. Standard scoring and classification systems such as Common Vulnerability Scoring System (CVSS),

Common Weakness Enumeration (CWE) and Common Vulnerabilities and Exposures (CVE) are used to assess severity and exposure [13]

6. Attack Modeling - This stage focuses on modeling how attacks could be executed. Activities include attack surface analysis, development of attack trees (defined in detail in Chapter 2.3), management of attack libraries (repositories of known attack patterns and techniques) and analysis of the relationship between attacks, vulnerabilities and potential exploits [13]
7. Risk and Impact Analysis - The final stage evaluates and prioritizes risks by qualifying and quantifying business impact. It also involves identifying countermeasures, analyzing residual risk and defining risk mitigation strategies to reduce overall exposure [13].

By linking business risk analysis with technical cybersecurity analysis, PASTA provides a structured method to identify, analyze and prioritize threats and risks within an application environment. This depth comes at a price, however: its seven-stage process is complex and resource-intensive, demanding considerable time, expertise and cross team collaboration, which can limit its adoption in smaller or fast-moving projects [13].

2.2.3 OCTAVE Framework

OCTAVE is a risk-focused threat modeling framework that emphasizes a deep understanding of organizational assets, associated threats and underlying vulnerabilities. Rather than concentrating solely on technical attack vectors, OCTAVE adopts an asset-centric perspective, enabling organizations to assess and manage risks in a structured and business-aligned manner [14]. By explicitly linking assets to threats and vulnerabilities, the framework supports a comprehensive understanding of organizational risk exposure.

Developed in 1999 at Carnegie Mellon University's Software Engineering Institute [15], OCTAVE is designed as a comprehensive risk assessment and mitigation framework suitable for organizations of different sizes and maturity levels. The framework is particularly effective in strengthening digital business environments by aligning security practices with organizational objectives and operational realities [14].

The OCTAVE framework is structured around a sequence of core activities that guide analysts through systematic risk identification, evaluation, and prioritization:

1. Asset Identification: The process begins with identifying and prioritizing critical assets that require protection. This step establishes the scope of the assessment and ensures that security efforts focus on the most valuable and sensitive components of the organization [16].

2. **Architectural Overview:** An architectural analysis is performed to understand how information flows through the system. This includes identifying subsystems, data paths and trust boundaries using DFDs, providing a clear security blueprint of the application or organizational environment [14].
3. **Application Decomposition:** The system is examined at a granular level to uncover vulnerabilities arising from design decisions, implementation flaws and deployment configurations. This decomposition highlights potential weak points that may otherwise remain hidden [17].
4. **Threat Identification:** Based on the identified assets and architecture, potential threat actors and threat scenarios are analyzed. This step focuses on understanding how assets could be compromised and the nature of the threats they face [18].
5. **Threat Documentation:** All identified threats are systematically documented to ensure traceability, consistency and comprehensive visibility throughout the risk management process [19].
6. **Threat Rating and Prioritization:** Threats are evaluated and prioritized using established risk classification methodologies such as CVSS and the OWASP Risk Rating Methodology [20], a structured approach that estimates the severity of a risk from its likelihood and impact factors. This enables organizations to focus mitigation efforts on the most critical and high-impact risks.

OCTAVE's framework uses structured artifacts, such as asset profiles, risk flow analyses and threat mappings, to support both analytical rigor and stakeholder communication [14]. Despite its comprehensive capabilities, OCTAVE presents certain challenges. The framework requires extensive documentation, which can make its adoption resource-intensive [21]. Additionally, its highly structured and methodical nature may introduce bureaucratic overhead, potentially slowing down the risk management process [21]. To address these limitations, lighter-weight variants such as *OCTAVE-S* and *OCTAVE Allegro* [15] have been developed, offering streamlined alternatives better suited for smaller organizations or teams with limited risk assessment resources.

2.2.4 VAST Framework

The VAST modeling framework is designed to enable practical, scalable threat modeling that can be continuously applied throughout the software development lifecycle. Rather than aiming for exhaustive or highly formal security models, VAST focuses on identifying

actionable threats and mitigations that align with organizational risk management objectives and development workflows [15].

VAST employs two complementary and independent methodologies: application threat modeling and operational threat modeling [15]. Application threat modeling concentrates on identifying threats within individual applications by analyzing execution flows, trust boundaries and interaction points. Operational threat modeling addresses infrastructure-level and organizational threats, including deployment architectures, supporting services and operational processes. Together, these methodologies enable VAST to address both software-centric risks and broader operational exposures within a unified and consistent framework.

A defining characteristic of VAST is its use of Process Flow Diagrams (PFDs) rather than traditional DFDs, particularly for application threat modeling [15]. PFDs emphasize process logic, execution paths and system interactions as they occur in practice. By prioritizing clarity and accessibility over formal completeness, VAST facilitates collaboration across development, security and operations teams while maintaining a consistent approach to threat identification and risk prioritization.

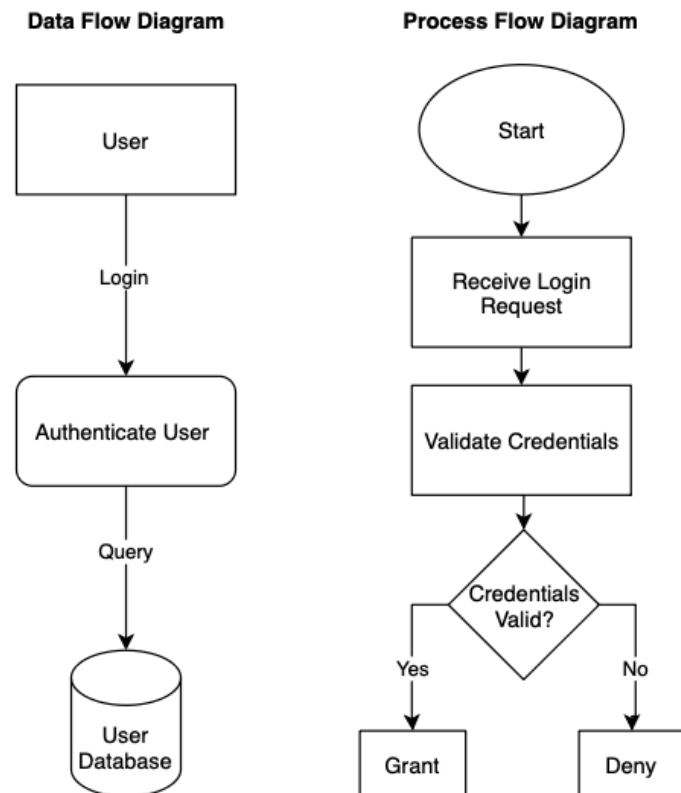


Figure 1: Comparison between a Data Flow Diagram (DFD) and a Process Flow Diagram (PFD) on a login flow

Figure 1 illustrates how a DFD highlights data movement and storage, whereas a process flow diagram emphasizes execution logic and decision paths within the same application scenario. The two representations therefore complement each other when analysts need to understand both information exchanges and operational behavior.

Within application threat modeling, VAST guides analysts through a structured process that includes identifying assets and entry points, analyzing potential threats, assessing their impact and defining appropriate mitigation strategies. The process-flow-based representation enhances visibility into how threats emerge across application workflows, enabling faster identification and resolution of security issues compared to more static modeling approaches [15].

Despite its strengths, VAST has limitations. It is not a publicly available standard and is protected by patents covering both its methodology and associated tooling [15]. This proprietary nature restricts its distribution, customization and adoption, particularly in open or highly regulated environments. Additionally, while VAST is effective for application threat modeling, its operational threat modeling component may be less comprehensive when compared to frameworks that place stronger emphasis on organizational or infrastructure-level risk analysis [15].

2.2.5 Comparison of Threat Modeling Frameworks

STRIDE, PASTA, OCTAVE and VAST each offer distinct perspectives on how threats should be identified, analyzed and mitigated. While some frameworks emphasize business risk and asset value, others focus on technical attack vectors or scalability. Understanding the strengths and limitations of each framework is essential for selecting the most appropriate approach for a given system.

Table 1 provides a comparative overview of the four frameworks discussed in this dissertation, highlighting their primary focus, scope, methodology, strengths and limitations. It makes the trade-offs that inform the framework selection in the following chapter explicit.

Table 1: Comparison of Threat Modeling Frameworks

Framework	Focus	Scope	Method	Benefits	Limitations
STRIDE	Software threats	Application	Six-category taxonomy	Simple and widely adopted	Limited business context
PASTA	Risk analysis	Business and technical	Seven-stage process	Strong business alignment	Resource-intensive
OCTAVE	Asset-based risk	Organization and systems	Asset, threat and risk analysis	Asset and risk-focused	Documentation heavy
VAST	Scalable threat modeling	Application and operations	Process flow modeling	Scalable and visual	Proprietary

In summary, STRIDE serves as a foundational and accessible entry point for technical threat identification, particularly during software design. PASTA provides a comprehensive, risk driven framework that integrates business objectives with technical threat analysis, making it well suited for complex systems. OCTAVE emphasizes organizational assets and risk governance, offering a strong foundation for enterprise wide risk management. VAST, in contrast, is designed for large development environments in which scalability and rapid threat identification are important.

Selecting an appropriate threat modeling framework depends on multiple factors, including system complexity, organizational maturity, development methodology and the need to align security with business objectives. In practice, no single framework is universally sufficient, organizations often achieve more effective outcomes by selectively integrating elements from multiple approaches to address their specific technical and organizational contexts.

2.3 Fundamentals of Attack Trees and Attack Graphs

2.3.1 Attack Tree

Originally introduced by *Schneier* in 1999 [22], an attack tree is a hierarchical graphical model used to represent how an adversary may exploit system vulnerabilities to achieve a specific malicious objective. The attacker's goal is depicted as the root node, while branches represent alternative or complementary strategies that can lead to that goal. By structuring attack paths, dependencies and potential weaknesses within a single diagram, attack trees provide a systematic and visual approach to security threat modeling [23].

Attack trees support the analysis of complex systems by decomposing high-level attack objectives into progressively more detailed subgoals and concrete actions. This decomposition enables security analysts to examine potential attack strategies in a structured manner, highlighting relationships and dependencies among different attack vectors [23]. As a result, attack trees offer insight into how adversaries may realistically proceed in real-world scenarios.

Attack trees consist of three primary components: a root node representing the attacker's main objective, intermediate nodes corresponding to subordinate goals or conditional steps, and leaf nodes that identify specific techniques, vulnerabilities or system weaknesses that can be exploited directly [23]. Logical relationships between nodes are expressed using operators such as *AND* and *OR*, where an *AND* relationship indicates that all connected conditions must be satisfied together and an *OR* relationship denotes alternative paths that independently achieve the parent goal. Formally, an attack tree is a 3-tuple $(N, \rightarrow, n_0)$, where N is a finite

set of nodes, $\rightarrow$ is a finite acyclic relation of type $\rightarrow \subseteq N \times M(N)$, where $M(N)$ is the multi-set of N , and n_0 is the root node, such that every node in N is reachable from n_0 [23].

As an analytical tool, attack trees facilitate proactive risk assessment by enabling security analysts to visualize possible attack pathways, identify exploitable weaknesses and evaluate the feasibility of different attack combinations. By supporting the assessment of attack likelihood, impact and dependencies, attack trees help prioritize defensive measures and contribute to informed, evidence-based decision making in cybersecurity risk management [24].

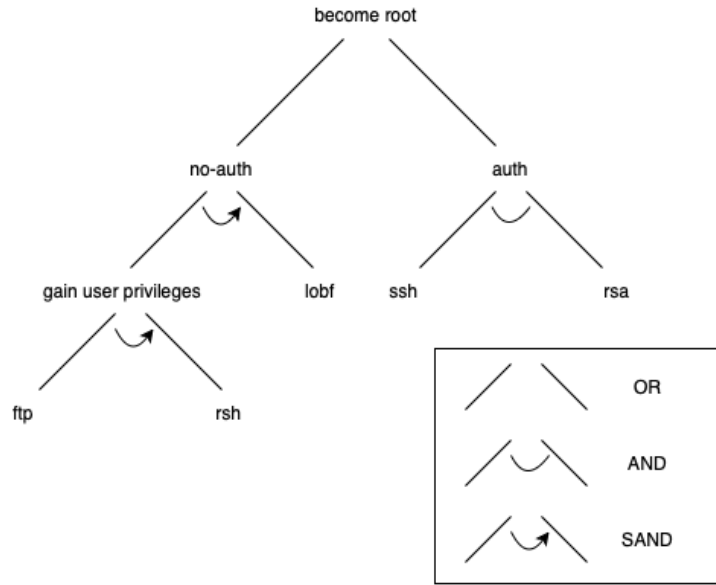


Figure 2: Example of an Attack Tree

Figure 2 shows an example of an attack tree. In this example, there are two distinct paths by which an attacker can achieve the ultimate goal of obtaining root access: with authentication or without authentication. In the authentication based path, the two refined options are Secure Shell (SSH) and Rivest-Shamir-Adleman (RSA). Both conditions must be satisfied, as they are connected by an *AND* operator. In contrast, the unauthenticated path requires the attacker to first obtain user-level privileges without authentication by performing the File Transfer Protocol (FTP) and Remote Shell (RSH) actions. After these privileges are acquired, the attacker can exploit the local buffer overflow labeled lobf in the figure. This sequence is represented using a Sequential AND (SAND) operator, which is similar to an *AND* operator but enforces a specific order in which the child actions must be carried out.

Overall, attack tree notation is both intuitive and effective for threat analysis, as it enables the systematic representation of multiple attack scenarios arising from physical and technical vulnerabilities [25].

2.3.2 Attack Graph

Attack graphs provide a structured representation of how cyberattacks may progress within an organization's infrastructure. By modeling systems, vulnerabilities and the dependencies between them, they expose the potential paths an adversary could exploit to compromise assets and achieve malicious objectives [26].

An attack graph is a visual model that describes the set of feasible attack paths within a networked system. Nodes represent system entities or vulnerabilities, while edges capture the actions or dependencies that allow an attacker to move between states [26]. This structure illustrates how attackers can chain exploits, move laterally and escalate privileges to reach critical resources. Formally, an attack graph is a tuple $G = (S, \tau, S_0, S_s)$, where S is a set of states, $\tau \subseteq S \times S$ is a transition relation, $S_0 \subseteq S$ is a set of initial states and $S_s \subseteq S$ is a set of success states. Intuitively, S_s denotes the set of states in which the intruder has achieved their goals. Unless stated otherwise, the transition relation τ is assumed to be total. An execution fragment is a finite sequence of states $s_0, s_1, \dots, s_n$ such that $(s_i, s_{i+1}) \in \tau$ for all $0 \leq i < n$. An execution fragment with $s_0 \in S_0$ constitutes an execution, and an execution whose final state is in S_s constitutes an attack. In other words, the execution corresponds to a sequence of atomic attacks leading to one of the intruder's goals [27].

As an analytical tool, attack graphs enable security teams to reason about the sequence and feasibility of attacks rather than isolated vulnerabilities. They support proactive threat analysis by modeling the relationships between system components, known vulnerabilities and potential attack paths, enabling the identification of viable attack vectors [28]. Through the evaluation and simulation of attack scenarios, organizations can prioritize mitigations, focus defensive controls and allocate resources more effectively.

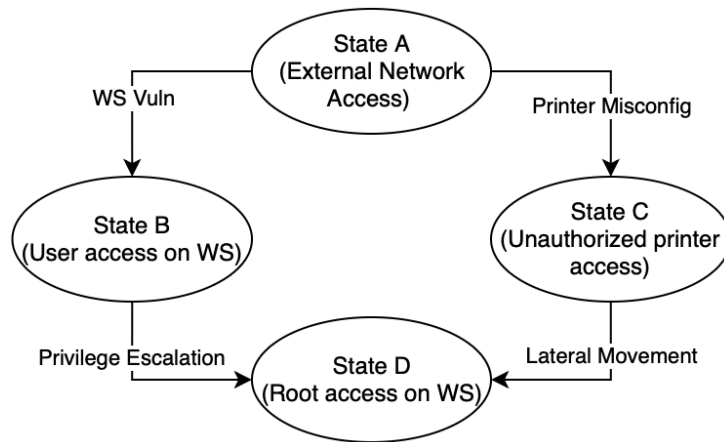


Figure 3: Example of an Attack Graph

Figure 3 presents an attack graph for a simple network containing a workstation, labeled WS in the figure, and a printer. State A represents an attacker with external network access to services exposed by these assets. A vulnerability on the workstation leads to State B, representing user-level access on that asset. Alternatively, a printer misconfiguration leads to State C, representing unauthorized access to the printer. From State B, a privilege escalation can lead to root access on the workstation, represented by State D. Access to the printer can also enable lateral movement to the workstation and lead to the same final state. The graph illustrates alternative attack paths and the convergence of distinct exploit sequences on a common objective.

2.3.3 Attack Trees vs Attack Graphs

A graph is a mathematical structure composed of nodes connected by edges and may be either directed or undirected [26]. In directed graphs, edges indicate transitions from a source node to a destination node, whereas undirected graphs represent bidirectional relationships without an inherent direction.

A tree is a specialized type of Directed Acyclic Graph (DAG) with a strict hierarchical structure [23]. It consists of a single root node, internal nodes with one parent and one or more children and leaf nodes with no children. This structure enforces a clear decomposition of objectives and explicitly prohibits cycles.

These structural differences directly influence how attack trees and attack graphs represent adversarial behavior. Attack trees are acyclic by definition and are centered around a single attacker goal, represented by the root node. They decompose this goal into subgoals and atomic attack steps arranged hierarchically. Attack trees typically express attack logic in a bottom-up manner, combining individual exploits to reach the final objective. In most formulations, nodes represent exploits, while the satisfaction of preconditions is assumed implicitly during transitions. Attack graphs, in contrast, offer a more expressive modeling framework. They may include multiple goal states and allow cycles, enabling the representation of repeated or looping actions and evolving attack strategies. Attack graphs commonly represent attack progression in a state based manner and explicitly model both exploits and the preconditions that enable them. This allows for a detailed representation of intermediate system states, partial attacks and alternative execution paths [26].

While both approaches use graph-based structures to model attack behavior, they differ in expressive power, treatment of preconditions and representation of attack progression. Because attack trees organize a single objective into a clear hierarchy of subgoals while leaving preconditions implicit, they are well suited for high-level, goal-oriented analysis and

for communicating attack logic to stakeholders. Attack graphs, by explicitly modeling system states, exploit preconditions and the dependencies between steps, and by allowing multiple goals and cycles, capture how the many steps of complex, multi-stage attacks interact, which makes them more appropriate for detailed technical modeling.

2.4 Vulnerability Databases and Classification Systems

Vulnerability databases and classification systems provide structured repositories of information about known security weaknesses, supporting vulnerability management, risk assessment and security analysis. By standardizing the identification and description of vulnerabilities across software, hardware and networked systems, these resources facilitate information sharing and serve as key inputs to security assessment methodologies and analytical models.

2.4.1 CVE (Common Vulnerabilities and Exposures)

CVE¹ is a publicly accessible catalog of disclosed cybersecurity vulnerabilities. The CVE system was created in 1999 by the MITRE Corporation [29], a U.S. government funded research and development organization, to establish a standardized method for reporting and tracking software vulnerabilities. Each CVE entry provides a brief description of the issue, but does not include in depth technical details, risk assessments or remediation steps. Those elements are documented in complementary databases such as the U.S. National Vulnerability Database (NVD)² and various vendor maintained advisory lists.

Despite these different sources, the CVE ID serves as a consistent reference point, enabling security teams, vendors and researchers to coordinate vulnerability analysis, compare data and develop tools and mitigation measures [29]. While MITRE maintains the CVE list, entries are often submitted by vendors, security researchers and members of the open-source community.

According to CVE Numbering Authority (CNA) operational rules, a vulnerability must meet specific conditions to receive a CVE ID:

1. Independently fixable: The vulnerability must be remediable without requiring changes to other unrelated issues [29]
2. Acknowledged or demonstrably impactful: The affected vendor must acknowledge the flaw and confirm its security impact or the reporter must provide a detailed vulnerability report demonstrating both the security impact and its violation of the system's security policy [29]

¹<https://www.cve.org>

²<https://nvd.nist.gov/vuln>

3. Affects a single codebase: Each affected codebase is assigned its own CVE. When several products share the same flawed library, protocol or standard, a single CVE is used only if every implementation is inherently affected by that flaw. Otherwise, each affected product or codebase receives a separate CVE entry [29].

Vulnerabilities are evaluated based on their severity, and several frameworks exist to support this assessment. The most widely adopted is the CVSS, which assigns a numerical score ranging from 0.0 to 10.0 according to a vulnerability's exploitability, impact and environmental characteristics [30]. Higher scores correspond to greater potential risk. CVSS is used by the NVD and many security vendors [30], although some organizations complement it with proprietary or context-specific scoring methodologies.

Beyond providing a numerical score, CVSS is central to modern vulnerability management, helping organizations prioritize which security issues to address first. Over time, several CVSS versions have been released. The latest, CVSS 4.0, refines how severity is assessed to better support real-world risk decisions [30].

CVSS 4.0 organizes vulnerability characteristics into four metric groups [30]. Together, these groups provide a more comprehensive view of vulnerability severity by incorporating both technical and contextual factors:

- Base - Intrinsic qualities of a vulnerability (e.g., ease of exploitation and potential impact on confidentiality, integrity and availability);
- Threat - Factors that change over time, such as the availability of exploit code or evidence of active attacks;
- Environmental - Adjustments that reflect how critical the affected asset is in a specific organization's environment;
- Supplemental - Additional context beyond technical severity, such as ease of automation or potential safety implications.

Despite the release of CVSS 4.0, CVSS v3.x (most commonly v3.0 or v3.1) remains the predominant version used in practice. Many tools, databases and workflows continue to rely on CVSS v3 due to its widespread adoption and ecosystem support [31].

CVSS v3.x centers on the Base Score, which captures the inherent severity of a vulnerability using the following metrics: Attack Vector (AV), Attack Complexity (AC), Privileges Required (PR), User Interaction (UI), Scope (S), Confidentiality Impact (C), Integrity Impact (I) and Availability Impact (A) [31]. These metrics collectively describe how a vulnerability can be

exploited and the extent of its potential impact, forming the foundation for severity-based vulnerability prioritization.

2.4.2 CWE (Common Weakness Enumeration)

The CWE³ is a community maintained catalog of common software and hardware weaknesses [32]. A weakness refers to a design or implementation flaw that may lead to a security vulnerability when exploited under specific conditions. CWE provides a standardized taxonomy for describing and classifying such weaknesses, enabling consistent communication about the root causes of security flaws across tools, organizations and development teams [32].

Unlike vulnerability databases that document specific, observed flaws, CWE focuses on generalized patterns of error that recur across systems and technologies. For example, CWE-79 (Improper Neutralization of Input During Web Page Generation) describes the underlying weakness that can result in Cross-Site Scripting (XSS) vulnerabilities [32]. Multiple CVEs may reference CWE-79, indicating that distinct vulnerabilities share the same fundamental cause.

By abstracting individual vulnerabilities to their underlying weaknesses, CWE enables a more proactive approach to security improvement. Developers can use CWE to guide secure coding practices, architects can identify design-level risks and security teams can use CWE classifications to identify recurring weakness patterns across applications. This allows organizations to address the root causes of vulnerabilities through improved design decisions, coding standards and development processes, rather than repeatedly patching individual flaws [33].

2.4.3 CPE (Common Platform Enumeration)

Common Platform Enumeration (CPE)⁴ is a standardized naming scheme used to uniquely identify information technology products, including operating systems, applications and hardware devices [34]. A CPE name provides a structured and unambiguous way to describe a specific product and its version, enabling consistent reference to the same platform across different security tools, databases and organizations.

CPE names follow a formal specification derived from the generic syntax of Uniform Resource Identifiers (URIs) [34], which allows them to be reliably interpreted and used in automated security analysis and vulnerability management workflows. For example:

```
cpe:2.3:a:apache:http_server:2.4.49:*:*:*:*:*:*
```

This CPE uniquely identifies version 2.4.49 of the Apache HTTP Server. In this string, 2.3 indicates the CPE specification version, the part field a denotes an application (the other possible

³<https://cwe.mitre.org>

⁴<https://nvd.nist.gov/products/cpe>

values being o for operating systems and h for hardware), and the wildcard fields (*) specify that the identifier applies regardless of update, edition, language or platform. Vulnerabilities referencing this CPE can be automatically matched to systems running the affected software, independent of variations in local inventories or vendor documentation.

The Official CPE Dictionary is a publicly available repository containing an authoritative list of standardized CPE names. Each entry represents a specific product release and serves as a common reference for vulnerability databases, scanners, and security assessment tools [34]. The dictionary is maintained as a living resource, regularly updated with new releases and corrections, ensuring that CPE remains a reliable foundation for automated vulnerability identification and correlation.

2.4.4 Limitations of Severity-Based Vulnerability Scoring

The CVSS is widely used to assess the potential severity of software vulnerabilities. However, CVSS scores do not capture how likely a vulnerability is to be exploited in real-world attacks. This limitation has significant implications for vulnerability management, where effective prioritization depends not only on potential impact but also on the probability of exploitation [35].

Empirical studies consistently demonstrate the shortcomings of severity-based prioritization: only a small fraction of disclosed vulnerabilities are ever exploited, with some studies reporting exploitation rates as low as 1.4% [35]. Moreover, remediation strategies based solely on CVSS severity have been shown to perform poorly when evaluated against actual attacker behavior, offering little improvement over random selection when the objective is to reduce exposure to exploited vulnerabilities [35]. These findings highlight the need for complementary approaches that explicitly incorporate exploitation likelihood and observed attacker activity.

2.4.4.1 EPSS (Exploit Prediction Scoring System)

The Exploit Prediction Scoring System (EPSS)⁵ was developed to address this need by estimating the probability that a vulnerability will be exploited within a defined period following public disclosure. EPSS is a predictive, data-driven model that produces a continuous probability score between 0 and 1, where lower values indicate a minimal likelihood of exploitation and higher values indicate increased risk [35].

This score is derived from a binary prediction task: whether a vulnerability is exploited in the wild within 12 months of disclosure. EPSS models this outcome using standard logistic regression, selected for its balance between predictive performance, interpretability and com-

⁵<https://www.first.org/epss/>

putational efficiency [35]. The resulting coefficients can additionally be interpreted as changes in odds ratios, enabling analysts to understand the influence of different predictors [35].

The result is an interpretable scoring system that focuses on exploitation probability rather than severity alone. A key advantage of EPSS is its ability to generate probability estimates immediately upon vulnerability disclosure, enabling early prioritization without reliance on confirmed exploit activity [35]. As new exploit and threat intelligence becomes available, the model can be updated to reflect evolving attacker behavior.

In practice, EPSS complements CVSS by introducing a predictive threat dimension to vulnerability assessment. While CVSS characterizes potential impact, EPSS supports more effective risk reduction by helping security teams prioritize vulnerabilities that are most likely to be exploited, thereby improving the allocation of limited remediation resources.

2.4.4.2 KEV (Known Exploited Vulnerabilities)

While EPSS provides a predictive estimate of future exploitation likelihood, the Known Exploited Vulnerabilities (KEV) catalog, maintained by the Cybersecurity and Infrastructure Security Agency (CISA)⁶, supports vulnerability prioritization based on observed rather than predicted threat activity [36]. The catalog serves as an authoritative reference of vulnerabilities that have been confirmed to be exploited in the wild, offering directly actionable intelligence for security teams [36]. Consequently, organizations are encouraged to incorporate KEV as a primary input to remediation planning, ensuring that vulnerabilities associated with demonstrated attacker behavior receive priority attention.

Unlike scoring systems such as CVSS, inclusion in the KEV catalog is not based on theoretical severity or exploit complexity. Instead, the primary criterion for inclusion is reliable evidence that a vulnerability has been, or is being, exploited in the wild [36]. In the context of the KEV catalog, the terms “exploited” and “actively exploited” are used interchangeably to describe confirmed cases in which malicious actors have successfully executed code against a vulnerable system without authorization from the system owner [36].

By emphasizing observed exploitation rather than hypothetical risk, KEV provides an important corrective to purely score based prioritization. For example, a vulnerability with a moderate CVSS score may warrant immediate remediation if it appears in the KEV catalog, while a higher scoring vulnerability without known exploitation may be addressed with lower urgency.

⁶<https://www.cisa.gov/known-exploited-vulnerabilities-catalog>

2.5 Open-Source Intelligence (OSINT)

OSINT refers to the process of collecting, evaluating, and analyzing publicly available information in order to produce actionable intelligence that answers a specific question or supports a defined objective [37]. To understand OSINT, it is essential to distinguish between information and intelligence. Information alone consists of raw and unprocessed data, it becomes intelligence only after it has been critically assessed, contextualized and analyzed to generate insight or understanding. In other words, intelligence is the result of transforming unstructured or fragmented data into meaningful conclusions that can inform and guide decision making [37].

There are two primary categories of OSINT collection: passive and active.

- Passive OSINT refers to the process of gathering information without directly interacting with a target, for example through publicly available sources such as vulnerability databases, social media content or archived web pages. Passive collection minimizes the risk of exposure and maintains a non intrusive approach;
- Active OSINT, in contrast, involves some degree of interaction with the target, such as querying publicly exposed services or enumerating domain-related information. This form of collection can resemble investigative or undercover activities and depending on organizational policies, may require prior authorization or ethical review before engagement.

In summary, OSINT serves as a powerful and ethically grounded means of collecting and interpreting publicly available data. When applied responsibly and in accordance with legal and professional standards, OSINT enables organizations and individuals to make informed, data-driven decisions across a wide range of operational and strategic contexts.

2.6 Cyber Threat Intelligence (CTI)

The term Cyber Threat Intelligence (CTI) is often intuitively misunderstood, because it does not lend itself to a single, universally accepted definition [38]. To define CTI clearly, it is necessary to examine the two constituent concepts: intelligence and cyber threat. As discussed in the previous sections, intelligence refers to evidence-based knowledge that reduces uncertainty and supports informed decision making. A cyber threat, in turn, denotes any potential or actual malicious activity enabled through digital systems that may compromise the confidentiality, integrity or availability of information and services. CTI can therefore be understood as the provision of structured and contextualized insight that enables organiza-

tions to comprehend adversaries, anticipate their actions and implement effective defensive measures [39].

CTI is closely related to OSINT, but the two terms are not equivalent. OSINT transforms publicly available information into intelligence for a defined purpose, whereas CTI applies intelligence processes specifically to cyber threats and cyber defense [37], [39]. CTI may use OSINT alongside internal telemetry, commercial reporting and other sources. Thus, OSINT can contribute to CTI, but not all OSINT concerns cybersecurity and CTI is not limited to publicly available information.

2.6.1 STIX

Structured Threat Information Expression (STIX) is an open, standardized language for representing and exchanging cyber threat intelligence in a machine-readable and interoperable format. It is developed by the Organization for the Advancement of Structured Information Standards (OASIS), an international standards body responsible for widely adopted specifications for information exchange. STIX defines a common data model for describing cyber threat entities, such as threat actors, malware, attack techniques, vulnerabilities, indicators and observed incidents, and the relationships between them [3].

A key advantage of STIX over unstructured threat reports is its ability to preserve analytical context. Rather than sharing isolated indicators, STIX represents intelligence as interconnected objects that reflect real-world relationships between adversaries, infrastructure and behaviors. This structured representation enables automated processing, correlation across data sources and integration with security platforms, thereby aligning technical exchange with the analytical objectives of CTI [3]. In this way, STIX acts as a foundational enabler of large-scale intelligence sharing and collaborative defense.

STIX 2.x, the current generation of the standard, represents these objects in JSON, so that each models a distinct concept in the threat landscape and can be linked to others to form a coherent intelligence picture. A representative STIX 2.1 Campaign object is reproduced in Appendix E.

2.7 Asset Criticality and Risk Assessment

Effective risk assessment requires more than identifying vulnerabilities and estimating their likelihood of exploitation, it must also account for the value and operational importance of the assets affected. Asset criticality provides this business context, allowing the consequences of a successful attack to be incorporated into the prioritization of security measures.

2.7.1 Asset Criticality Assessment

Asset criticality assessment is the process of identifying and evaluating the importance of system components with respect to an organization's mission, operations and overall security posture [40]. The assessment establishes which assets are most valuable, most sensitive or most essential to operational continuity, providing a foundation for subsequent risk assessment and prioritization of security measures.

The first step in asset criticality assessment is to systematically identify all relevant assets within the system or environment under consideration [40]. This includes hardware components such as servers, network devices and storage systems, software assets including applications, services and operating systems, and information and data assets such as databases, configuration files, sensitive documents and proprietary information. The assessment must also consider users and digital identities, such as privileged accounts and roles with access to critical resources, as well as business processes and services that rely on IT systems to ensure continuity and operational performance.

Once assets are identified, their criticality is evaluated based on criteria that reflect the potential consequences of compromise. Common criteria include [40]:

- Confidentiality Impact: The severity of consequences if unauthorized disclosure occurs;
- Integrity Impact: The potential harm from unauthorized modification or corruption of the asset;
- Availability Impact: The operational and business impact if the asset becomes unavailable;
- Business Impact: Financial loss, legal exposure, regulatory compliance issues and reputational damage;
- Dependency and Inter-connectivity: The degree to which other assets, services or processes depend on the asset in question.

By quantifying the importance of assets, criticality assessment provides a basis for estimating the impact component of risk. High criticality assets amplify the consequences of successful attacks, guiding analysts to prioritize mitigation strategies for these assets. When combined with likelihood estimations derived from vulnerabilities and attack modeling, asset criticality forms a core component of a structured risk assessment framework.

2.7.2 Risk Assessment Methodologies

Risk assessment provides a systematic means of evaluating the security posture of a system by estimating the potential consequences of successful attacks. It combines information about

threats, vulnerabilities and assets to support prioritization and decision making. Within this process, asset criticality determines the impact dimension of risk, while the likelihood of exploitation must be estimated through appropriate analytical methods. Approaches for this estimation differ in their level of formality, required input data, and degree of automation.

Qualitative methods estimate likelihood and impact through expert judgment and descriptive scales. Techniques such as likelihood impact matrices and scenario based analysis allow organizations to reason about risk even when detailed technical data are unavailable. These methods are easy to apply and to communicate, but because they depend on subjective judgment, different analysts or organizations may reach different conclusions about the same risk.

Quantitative and semi-quantitative methods seek to express likelihood and impact numerically in order to compare risks more precisely. Scoring schemes such as CVSS, as well as probabilistic models, represent attempts to approximate the likelihood component of risk in a repeatable manner. However, these approaches often rely on simplifying assumptions and may not fully capture complex attack dependencies or adversary behavior.

Model-based methods address this limitation by deriving likelihood estimates from structured representations of the system, such as attack graphs or attack trees. In these models, the probability of compromise can be inferred from properties including attack path length, reachability of critical assets and required attacker capabilities. By weighting model elements according to asset criticality, these approaches directly connect the technical structure of attacks with the business impact dimension of risk.

However, the quality of these estimates ultimately depends on how the attack model itself is generated. The literature identifies three model construction approaches [41]:

2.7.2.1 Model-Driven Approaches

Model-driven approaches begin with a structured description of the system and apply predefined transformation rules to generate an attack model [41]. The input may take the form of architectural diagrams, configuration specifications or dedicated security modeling languages. Because these descriptions are typically formulated to capture threats or attacker capabilities, the transformation process does not require detailed vulnerability analysis. In some cases, the initial representation already resembles an attack model or a closely related threat model.

The primary strength of model-driven approaches lies in their systematic and largely automated construction of the attack model. However, the realism of the resulting attack models depends heavily on the accuracy and completeness of the input description. When this

abstraction diverges from the actual security posture of the system, important attack paths may be missed [41].

2.7.2.2 Analysis-Driven Approaches

Analysis-driven approaches construct attack models through explicit reasoning over available information about the system [41]. The starting point is a system description that may be enriched with formal security properties, design constraints or data from vulnerability repositories. Analytical procedures then derive intermediate results that are translated into feasible attack scenarios.

Methods in this category vary with respect to the abstraction level of the system model, the formalisms used to express threats and the analysis techniques employed. Techniques range from basic reachability analysis in graphs to more advanced methods based on model checking, constraint satisfaction or automated planning. Threats are typically specified using formal or semi-formal languages, and the analysis extracts attack paths that are summarized in the resulting model [41].

2.7.2.3 Vulnerability-Driven Approaches

Vulnerability-driven approaches focus on known weaknesses and attempt to instantiate attack models by matching vulnerability patterns against available system information [41]. The required level of knowledge varies: some methods assume detailed inventories of network topology and installed software, while others rely on logs or expert input.

Vulnerability patterns are commonly represented as fragments of attack trees or graphs that refine or extend an existing model. Implementations range from semi-automatic techniques requiring analyst guidance to fully automated processes. These methods are particularly effective when comprehensive and up-to-date vulnerability data are available, yet their coverage is inherently limited by the completeness and accuracy of those sources.

While the approaches discussed above derive risk estimates from explicit system and attack models, not all security assessments can rely on such detailed representations. In many practical settings, particularly during early design phases or exploratory threat modeling, analysts must reason about risk before architectural models or comprehensive vulnerability data are available. In these contexts, structured scoring frameworks are commonly employed to support comparative prioritization rather than precise estimation.

2.7.3 DREAD Framework

Unlike the model-driven, analysis-driven and vulnerability-driven approaches described above, DREAD does not attempt to represent attack paths, dependencies or system structure explicitly. Instead, it evaluates individual threats through an ordinal scoring scheme that

approximates both impact and likelihood-based on expert judgment. DREAD is therefore best understood as a threat prioritization mechanism rather than a comprehensive risk modeling technique.

The DREAD framework is a quantitative risk rating model proposed by Microsoft to support the prioritization of identified security threats [42]. Unlike threat modeling approaches that focus primarily on threat discovery, DREAD is applied after threats have been identified to assess their relative risk and determine which threats require immediate mitigation. This prioritization is essential in practical security engineering, where time, budget and personnel constraints make it infeasible to address all identified threats simultaneously.

DREAD evaluates risk by scoring threats across five dimensions: Damage, Reproducibility, Exploitability, Affected Users, and Discoverability. Each dimension captures a different aspect of threat severity and likelihood, providing a structured and repeatable method for comparing heterogeneous threats.

- Damage refers to the potential impact of a successful attack, including financial loss, service disruption, data compromise or reputational harm;
- Reproducibility measures how consistently an attack can be reproduced once discovered, reflecting the effort required to repeat the exploit;
- Exploitability assesses how easy it is to launch the attack, considering factors such as required skill level, tool availability and access prerequisites;
- Affected Users represents the proportion of users, systems or stakeholders impacted if the threat is realized;
- Discoverability indicates how easily an attacker can identify the vulnerability or attack vector, either through system observation, documentation or automated scanning.

Each category is assigned a numerical score reflecting its severity. The scoring scheme employs a three point ordinal scale, where values of 1, 2 and 3 correspond to low, medium and high risk respectively, with 0 indicating the absence of risk [43]. The overall risk score is then calculated by summing the individual category scores [42]:

$$\text{Risk Score} = D + R + E + A + D$$

The resulting score ranges from 0 to 15, where higher values indicate greater risk exposure. Based on the total score, threats are commonly classified into qualitative risk levels, such as

low (0-6), medium (7-11) and high (12-15), enabling security teams to prioritize mitigation efforts accordingly [42].

DREAD's primary strength lies in its simplicity and ease of integration with other threat modeling techniques. When used in conjunction with frameworks such as STRIDE or attack tree analysis, DREAD provides a pragmatic mechanism for translating identified threats into actionable risk priorities [42]. However, the framework has notable limitations. The scoring process remains partially subjective and may vary between assessors, particularly in the absence of clear scoring guidelines. Additionally, DREAD does not explicitly account for business context, threat actor intent or evolving risk over time, which can limit its effectiveness in complex or highly dynamic environments [44].

Despite these limitations, DREAD remains a widely referenced and practical risk assessment technique, particularly suited for early stage design analysis and comparative threat prioritization within application-centric security assessments [43]. It is most useful when its subjective scores are supported by complementary technical and contextual evidence.

2.8 Chapter Summary

This chapter established the foundations required to reason about dynamic attack graph generation. It introduced security threats and vulnerabilities, compared the principal threat modeling frameworks, distinguished attack trees from attack graphs, and examined the vulnerability and threat intelligence standards that describe software exposure. It also discussed asset criticality, risk assessment methodologies and the limits of severity-only prioritization, motivating the use of exploitation likelihood and contextual risk signals.

These concepts provide the vocabulary and analytical basis for the remainder of the dissertation. The next chapter examines the foundations and selected approaches to attack graph generation, comparing the representations and tools most relevant to the proposed framework.

Chapter 3

Foundations and Selected Approaches to Attack Graph Generation

Building on the concepts introduced in Chapter 2, this chapter reviews foundational attack graph representations and selected tools relevant to this dissertation. It first surveys historically influential representations and the tools developed to construct them automatically, and then examines in greater depth the three systems most relevant to this dissertation, namely MulVAL⁷, SeaMonster⁸ and IntruSion alert-driven Attack Graph Extractor (SAGE)⁹, before comparing their respective strengths, limitations and suitability as a foundation for the proposed framework.

3.1 Attack Graphs

As previously established, an Attack Graph (AG) is a modeling formalism used to represent the sequences of actions that may lead to a successful cyberattack. Over the past two decades, numerous AG representations and automated generation techniques have been proposed to support security analysis and decision making [45]. This section reviews the historically influential AG frameworks relevant to this dissertation: Attack Trees (ATs), State Graphs (SGs), Exploit Dependency Graphs (EDGs), Logical Attack Graphs (LAGs), Bayesian Attack Graphs (BAGs) and Multiple Prerequisite Attack Graphs (MPAGs). It then outlines the principal tools developed for their automated construction. Figure 4 illustrates the chronological evolution of these representations and their associated toolchains, highlighting their historical development and influence within cybersecurity research [45].

⁷<https://github.com/risksense/mulval?tab=readme-ov-file>

⁸<https://sourceforge.net/projects/seamonster/>

⁹<https://github.com/tudelft-cda-lab/SAGE>

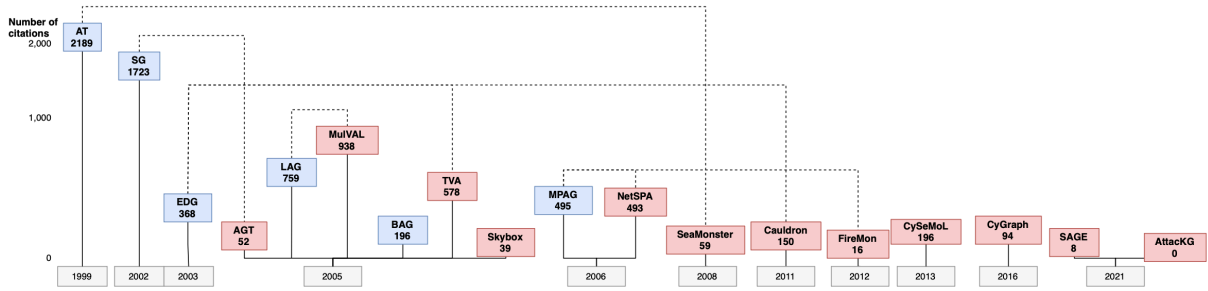


Figure 4: Evolution of AG Representations

The diversity of attack graph representations reflects the wide range of domains in which they have been applied. The data summarized in Figure 4 is based on studies available up to 2021 and is therefore used here as a historical overview rather than as a description of the current state of the field. More recent representations and tools may not be reflected in the distribution. Within its period of coverage, the figure indicates the prominent role of ATs and SGs and the established position of MuVAL among AG generation tools.

3.1.1 Attack Tree (AT)

Attack trees are an early, widely adopted approach to security threat modeling [22]. Their conceptual foundations, strengths and limitations were presented in Chapter 2.3. SeaMonster is an open-source security modeling tool that uses attack trees to relate vulnerabilities, attack paths and countermeasures throughout the software development lifecycle [46].

3.1.2 State Graph (SG)

The SG representation, introduced with the Attack Graph Toolkit¹⁰ in 2002, is an early formal approach to attack graph modeling [27]. Nodes represent global system states, while edges represent attacker actions that transition between them. This captures multi-step attack dynamics. However, explicit enumeration of states and transitions leads to the *state space explosion* problem [27]. As hosts and vulnerabilities increase, the number of global states grows exponentially, making enumeration infeasible for realistic environments.

The monotonicity assumption mitigates this limitation [47]. It assumes that an attack step does not invalidate other preconditions, so satisfied conditions remain valid. Under this constraint, attack graph generation complexity can be reduced from exponential to polynomial, enabling analysis of larger systems.

3.1.3 Exploit Dependency Graph (EDG)

The EDG, introduced in 2003 [48], enumerates exploit sequences under the monotonicity assumption. Each exploit and dependency appears once, and only exploits contributing to the attacker’s objective are included. This reduces redundancy, yielding a graph whose size grows

¹⁰<https://www.cs.cmu.edu/~scenario/graph/>

quadratically with the number of exploits and scales better than full state enumeration. EDG construction can nevertheless remain demanding in large environments [48].

The Topological Vulnerability Analysis (TVA) approach generates the graph backward from the attacker’s goal, using exploit and condition nodes to identify paths relevant to compromise [49]. Cauldron, an enterprise TVA implementation, adds visualization, vulnerability data integration and automated mitigation recommendations [50]. Figure 5 illustrates a representative EDG, where exploit and condition nodes express the prerequisites and dependencies leading to the attacker’s objective.

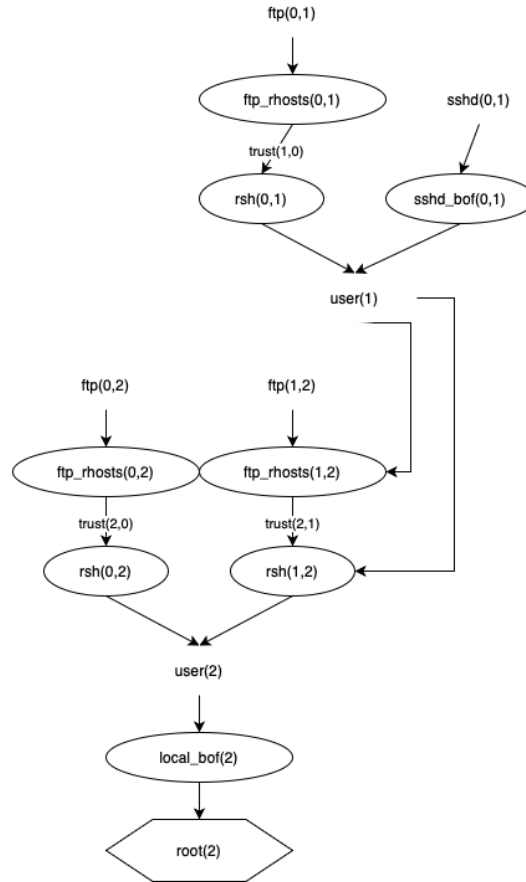


Figure 5: Example of EDG

3.1.4 Logical Attack Graph (LAG)

The LAG, introduced in 2005 [51], represents attack scenarios as a directed graph that can be structured as a tree. With the monotonicity assumption, its size grows polynomially with the number of exploits, enabling more scalable analysis than state enumeration [51].

Logical inference generates LAGs from formal rules expressing attack preconditions and effects. Tools such as MulVAL combine system configuration and vulnerability data to derive feasible attack paths [51].

3.1.5 Bayesian Attack Graph (BAG)

BAGs model attack progression as a directed acyclic graph in which nodes represent security states and edges encode probabilistic dependencies between them. By associating nodes with conditional probability tables, BAGs enable quantitative reasoning about the likelihood of compromise and support risk assessment through Bayesian inference [52], whereby the probability of system compromise is derived by propagating uncertainties along the attack graph and revised as new evidence is observed. This approach allows analysts to update the estimated probability of attack success whenever new evidence (such as alerts, vulnerability disclosures or observed exploits) becomes available.

Despite their strong analytical capabilities, BAGs remain less common in practice due to the difficulty of obtaining reliable probability estimates and the absence of widely adopted automated generation tools [45]. Figure 6 illustrates a typical BAG, where the directed structure expresses conditional relationships between attack steps and the associated probabilities used for inference.

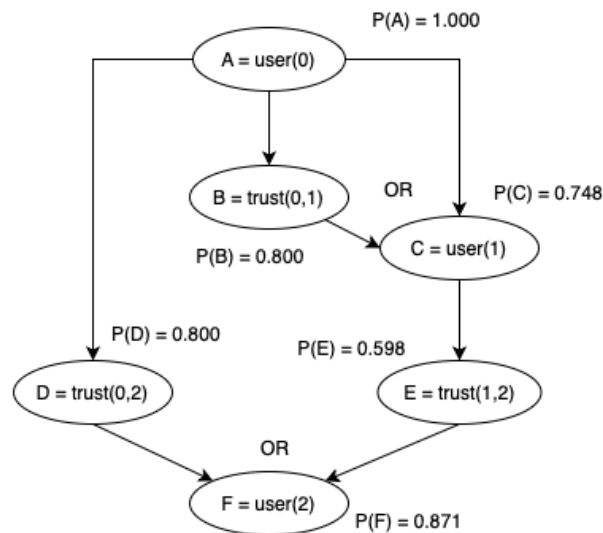


Figure 6: Example of BAG

3.1.6 Multiple Prerequisite Attack Graph (MPAG)

The MPAG, introduced in 2006 [53], extends exploit dependency models by representing prerequisites shared by multiple exploits. It employs state nodes for attacker access on a host, prerequisite nodes for conditions such as network reachability or credentials, and vulnerability nodes for exploited vulnerabilities. Aggregating prerequisites across attack paths reduces the number of edges and improves scalability.

¹¹<https://www.firemon.com/>

NetSPA and its commercial derivative FireMon¹¹ implement MPAGs for vulnerability prioritization and mitigation planning [54]. Figure 7 illustrates a representative MPAG, where distinct node types show how conditions are reused across attack steps. Numerous cyclic dependencies can nevertheless make large MPAGs difficult to interpret [53].

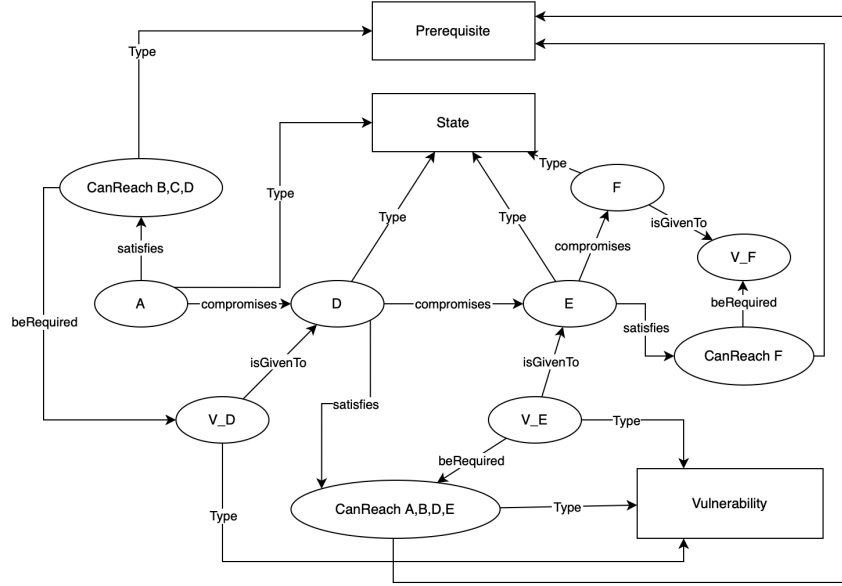


Figure 7: Example of MPAG

3.1.7 Complementary Attack Graph Frameworks

In addition to the frameworks discussed above, other attack graph generation tools are used in practice. Skybox View, introduced in 2005, is a commercial platform for vulnerability and threat management whose proprietary attack graph representation supports vulnerability assessment and remediation workflows [45]. CySeMoL¹² provides a modeling language and toolset for estimating enterprise cybersecurity through quantitative attack and defense relationships while requiring limited security expertise [55]. MITRE’s CyGraph¹³, introduced in 2016, combines TVA-based network modeling with layers for cyber threats and mission dependencies [56]. More recently, CTI-driven approaches, such as SAGE [57] and AttackKG¹⁴ [58], construct attack graphs from cyber threat intelligence rather than solely from system vulnerabilities. DeepAG further combines attack graph techniques with deep learning for prediction and prioritization [59].

Despite these advances, attack graph generation continues to face scalability and visualization challenges in large environments [26]. Table 2 compares the tools by accessibility, representation, generation complexity and analyst usability. In this comparison, generation complexity refers to the expected asymptotic growth in the computational cost of constructing an attack

¹²<https://www.kth.se/nse/research/software-systems-architecture-and-security/projects/old-projects/cysemol/downloads-1.432383>

¹³<https://www.mitre.org/our-impact/intellectual-property/cygraph>

¹⁴<https://github.com/li-zhenyuan/AttackKG>

graph as the size of the input model increases. Depending on the representation, input size may be measured by hosts, vulnerabilities, exploits, prerequisites or reachable states. The asymptotic expressions therefore identify each approach’s principal scalability limitation rather than apply a common definition of N .

Table 2: Comparison of Attack Graph Generation Tools

Tool	Accessibility	AG Representation	Generation Complexity	Usability
Attack Graph Toolkit	Open-source	State Graph (SG)	$O(2^N)$	Moderate
MulVAL	Open-source	Logical AG (LAG)	$O(N^2) - O(N^3)$	High
TVA	Restricted / not publicly available	EDG	$O(N^3)$	High
Skybox View	Commercial	Proprietary	$O(N^3)$	High
NetSPA	Restricted / not publicly available	MPAG	$O(N \log N)$	Moderate
SeaMonster	Open-source	Attack Tree (AT)	$O(N^k)$	Moderate
Cauldron	Commercial	EDG	$O(N^3)$	High
FireMon	Commercial	MPAG	$O(N \log N)$	High
CySeMoL	Restricted / not publicly available	Model-based	$O(N^k)$	Not reported
CyGraph	Restricted / not publicly available	Hybrid / TVA-based	$O(N^k)$	Very High
SAGE	Open-source	Alert-driven	N/A	High
AttacKG	Open-source	CTI-based	N/A	Moderate

Three of these tools, spanning the field’s main paradigms, are examined in greater detail: MulVAL (logic-based), SeaMonster (expert-driven) and SAGE (alert-driven). Together, they illustrate the contrasting roles of formal reasoning, expert modeling and alert-driven analysis in attack graph generation.

3.2 MulVAL

MulVAL is a well-established foundation for security modeling and attack graph analysis, with strong and sustained adoption within the research community. Since its introduction in 2005, it has been referenced and used in hundreds of publications [41], demonstrating both its maturity and its recognition as a standard approach for logic-based attack graph generation.

Beyond its widespread use, MulVAL has proven to be highly extensible. A substantial body of work builds directly on the framework by introducing new interaction rules or by proposing methodologies for defining such rules. Rather than developing entirely new models, researchers have consistently chosen to extend MulVAL, underscoring its flexible design and long-term viability as a research platform [41].

3.2.1 Architecture and Data Integration

MulVAL is an open-source framework released under the GNU General Public License (GPL), a license that grants users the right to study, modify and redistribute the software [5]. It is designed to help security practitioners and administrators analyze, manage and monitor the security configurations of enterprise networks [60]. MulVAL leverages data from vulnerability databases and scanning tools to reason about network configurations and to generate potential attack traces, which describe how an attacker could exploit existing weaknesses to reach a specified objective. The resulting analysis can further support the selection of countermeasures, such as patching vulnerable software, modifying network topology or adjusting access permissions.

The MulVAL distribution includes an adapter for the NVD that automatically downloads vulnerability specifications and synchronizes them with a MySQL database [5]. This synchronization is designed to run periodically so that the local repository reflects the most recent vulnerability information. The stored data are then used by MulVAL adapters to translate the output of vulnerability scanning tools into a format suitable for analysis by the MulVAL engine.

These adapters allow MulVAL to consume current vulnerability and scan data when the corresponding synchronization and scanning processes are performed. They do not determine whether an update is relevant to a particular modeled environment before inference is executed, nor do they distinguish changes that alter the logical attack surface from those that only affect the priority of existing paths.

MulVAL currently provides two primary adapters. The first processes Open Vulnerability and Assessment Language (OVAL) reports, a standardized language for assessing and reporting the configuration and security state of computer systems [61]. The second adapter consumes Nessus¹⁵ scan results, where Nessus is a widely used vulnerability scanner that identifies misconfigurations, missing patches and known weaknesses in networked systems. Both adapters produce a structured description of detected vulnerabilities on scanned hosts, expressed in the input format required by the MulVAL analyzer [5].

¹⁵<https://www.tenable.com/products/nessus>

3.2.2 Reasoning Mechanism

MulVAL generates attack traces by combining a rule file, which can be extended with custom interaction rules, and an input file describing the target environment [60]. Both files are written in Datalog, a declarative logic language and subset of Prolog that supports efficient, rule-based reasoning. The analysis is executed using the XSB logic programming engine, whose tabled resolution mechanism reuses intermediate results and avoids infinite recursion [1]. As a result, attack traces are constructed in polynomial time [51]. This gives MulVAL a clear and formal specification of attack reasoning while keeping performance practical for realistic inputs.

The MulVAL rule base describes common attack scenarios, including the exploitation of software vulnerabilities and privilege escalation patterns. To perform an analysis, MulVAL requires several categories of input [60]:

- Vulnerability declarations, including classifications, consequences and the presence of vulnerabilities on specific hosts, typically obtained through MulVAL adapters;
- Host configuration information, such as installed software, user and process privileges, and active network services with their associated protocols and ports;
- Network configuration details, specifying connectivity between hosts, ports and protocols;
- Principals and access control information, describing users, files and permission relationships;
- Component interaction definitions, modeling how system elements interact;
- Attacker specifications, including initial access, network location and attack goals.

MulVAL input files consist of a human-readable set of primitive predicates that express system configurations and preconditions. Derived predicates, defined by interaction rules, represent the post conditions or outcomes of exploits, from which the final attack graph is generated [60].

Listing 1 presents a simplified example. A host named *webServer* runs an HTTP service accessible from the internet, while a second host, *fileServer*, runs a Samba service that is only reachable from *webServer*. Both services contain remotely exploitable vulnerabilities:

```

1. attackerLocated(internet).
2. attackGoal(execCode(fileServer, _)).

3. hacl(internet, webServer, tcp, 80).
4. hacl(webServer, fileServer, tcp, _).

5. vulExists(webServer, 'CVE1', httpd).
6. vulProperty('CVE1', remoteExploit, privEscalation).
7. vulExists(fileServer, 'CVE2', _).
8. vulProperty('CVE2', remoteExploit, privEscalation).

9. networkServiceInfo(webServer, httpd, tcp, 80, u1).
10. networkServiceInfo(fileServer, samba, tcp, 445, u2).

```

Listing 1: Simplified MulVAL attack graph specification for a multi-hop attack scenario [1]

Line 1 uses `attackerLocated` to define the attacker’s initial position as the internet. Line 2 uses `attackGoal` to specify the desired outcome, namely code execution on *fileServer*. The `hacl` predicates in lines 3 and 4 describe the permitted network communication. The first grants access from the internet to the HTTP service on *webServer* through TCP port 80, while the second allows *webServer* to contact *fileServer* through any TCP port. Lines 5 and 7 use `vulExists` to associate CVE1 and CVE2 with the affected hosts and, for CVE1, with the HTTP service. Lines 6 and 8 use `vulProperty` to classify both vulnerabilities as remotely exploitable and capable of privilege escalation. Finally, `networkServiceInfo` in lines 9 and 10 identifies the service, transport protocol, listening port and executing user for each host. Together, these facts allow MulVAL to infer that an attacker can compromise *webServer* from the internet and subsequently reach *fileServer*. The graph generator can subsequently convert the resulting attack trace into several output formats, including plain text, CSV files containing graph vertices and edges, DOT files or visual representations exported as EPS or PDF [5].

Figure 8 illustrates the attack graph produced for this scenario, where nodes correspond to derived security conditions and exploit steps, and edges represent the logical dependencies inferred by MulVAL. The highlighted path from the internet to *webServer* and subsequently to *fileServer* reflects the multi-hop attack sequence described above.

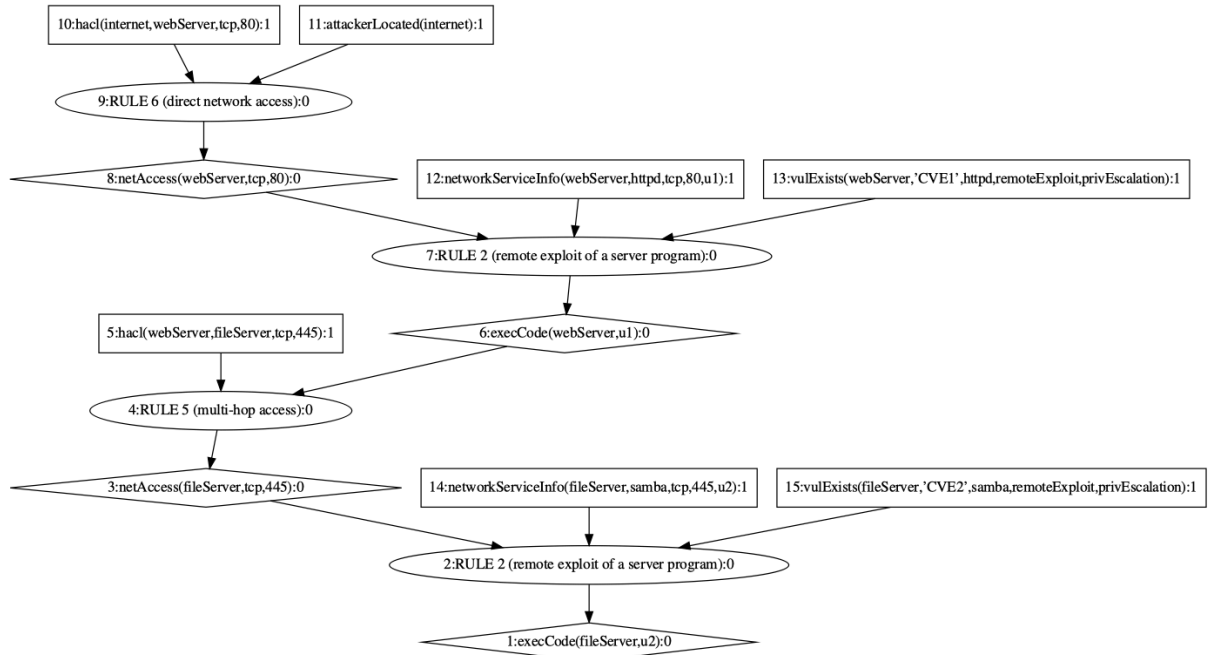


Figure 8: Example of a MulVAL-generated attack graph [1]

3.2.3 Limitations

MulVAL ships with only a basic set of interaction rules that primarily model exploit behavior at the network level. This standard rule set recognizes just three kinds of exploitability [1]: local vulnerabilities, which require the attacker to already run code on the target host, remote vulnerabilities, which require only network access, and client-side vulnerabilities, which are triggered when a vulnerable client interacts with a malicious remote party. In addition, the default rules assume that every successful exploit results in privilege escalation, so other kinds of impact cannot be represented [1]. Consequently, attack paths whose primary outcomes are data disclosure, data modification or service disruption cannot be represented as distinct logical consequences, which restricts the range of security goals that the graph can express.

Further limitations restrict MulVAL’s applicability to modern infrastructures. It has no native notion of virtual hosts and does not represent the physical machines on which they run, offering only limited support for virtualized or clustered environments [1]. It also cannot express combinations of prerequisites that must hold together for an exploit to succeed [1]. Together, these constraints limit the expressiveness of the generated attack graphs for contemporary, highly dynamic systems.

3.3 SeaMonster

SeaMonster is a graphical security modeling tool designed to support the structured analysis of software vulnerabilities from multiple complementary perspectives. Rather than introduc-

ing new modeling formalisms, SeaMonster integrates established techniques within a unified information model, enabling analysts to relate the causes of vulnerabilities, potential attack paths and corresponding mitigation strategies [46].

The tool organizes security analysis around three interconnected viewpoints:

- Vulnerability causes, focusing on the underlying design or implementation weaknesses that give rise to vulnerabilities;
- Threats and attacks, describing how those vulnerabilities can be exploited by an adversary;
- Countermeasures, addressing how vulnerabilities and their consequences can be prevented or mitigated.

Each viewpoint may contain multiple views, allowing the same system to be examined at different levels of abstraction or from alternative analytical angles. SeaMonster is extensible through a plugin based architecture and is implemented using the Eclipse Modeling Framework and the Graphical Modeling Framework, which provide support for model definition and visual editing [46].

3.3.1 Vulnerability Causes Viewpoint

The Vulnerability Causes viewpoint is grounded in security modeling approaches that seek to identify the root causes of vulnerabilities [46]. Its primary objective is to understand why vulnerabilities arise, enabling preventive action rather than merely addressing their observable effects. Such analyses are typically conducted by security experts after recurring or representative vulnerability instances have been identified [46].

Root Cause Analysis (RCA) comprises a family of techniques aimed at tracing observed problems back to their underlying origins. In contrast to traditional troubleshooting (which focuses on correcting individual failures), RCA seeks to eliminate the systemic conditions that enable vulnerabilities to emerge [62].

One widely adopted RCA technique is Fault Tree Analysis (FTA), which models how combinations of events can lead to system failure. By using logical AND and OR operators, FTA supports a structured exploration of causal dependencies and enables reasoning about multiple interacting factors [63].

SeaMonster adopts Vulnerability Cause Graphs (VCGs) as a representation tailored to security analysis. A VCG is a directed acyclic graph composed of four node types: simple, compound, conjunction and exit nodes [64]. Simple nodes denote basic causes, while compound nodes encapsulate complex substructures that can be further decomposed. Conjunction nodes

express dependencies in which multiple conditions must hold simultaneously, and exit nodes connect the model to other viewpoints [64].

VCGs support both generalization and specialization, enabling analysts to model vulnerabilities at different levels of abstraction [64]. This capability is particularly valuable because vulnerabilities often result from combinations of factors rather than isolated causes. Through the use of compound nodes, SeaMonster facilitates incremental decomposition, allowing large models to remain visually and cognitively manageable [46].

3.3.2 Threats and Attacks Viewpoint

The Threats and Attacks viewpoint addresses how identified vulnerabilities can be exploited to compromise a system. SeaMonster represents this perspective using attack trees [46]. In practice, threat-oriented and attack-oriented analyses are tightly interrelated, as analysts typically consider both the characteristics of potential adversaries and the concrete exploitation steps in parallel. SeaMonster therefore integrates these aspects into a single viewpoint, reflecting how attack trees are constructed and reasoned about in real-world security assessments.

Attack trees are often developed early in the system lifecycle to assess the attack surface and to reason about potential risks and consequences before implementation decisions are finalized. The relevance of particular attack paths depends strongly on the assumed attacker model. SeaMonster therefore encourages explicit consideration of attacker characteristics such as skill level, available resources, access privileges, risk tolerance and motivation [46].

3.3.3 Countermeasures Viewpoint

The Countermeasures viewpoint addresses how vulnerabilities can be prevented and how the threats and attacks they enable can be mitigated [46]. It complements the previous viewpoints by shifting the focus from analysis toward risk reduction and control selection.

One modeling approach supported by SeaMonster is the extension of Unified Modeling Language (UML) use case diagrams with security-related concepts, commonly known as misuse case diagrams. While standard use case diagrams effectively capture functional requirements, they provide limited support for expressing security concerns [46]. Misuse case diagrams extend this notation to explicitly represent threats, vulnerabilities and their interactions with intended system functionality [65].

In this extension, two types of mis-actors are introduced: attackers, representing external adversaries [65], and insiders, representing authorized users who may abuse legitimate access [66]. Additional relationships, namely exploit, detect and prevent, link threat and vulnerability use cases to regular use cases, making mitigation strategies explicit [46]. When applied

judiciously, misuse case diagrams provide a high-level overview of security requirements and countermeasures and serve as an effective communication tool among stakeholders [46].

SeaMonster also supports security activity graphs, which offer a more detailed and process-oriented representation of countermeasures. These graphs describe concrete activities that can mitigate a vulnerability, often derived directly from a corresponding VCG [67]. The root node represents the vulnerability, while leaf nodes denote preventive or mitigating activities. Logical gates capture dependencies and alternative mitigation strategies [67].

Compared with misuse case diagrams, security activity graphs provide finer grained guidance for design, implementation and testing. They are particularly useful for reasoning systematically about which combinations of activities are sufficient to mitigate a vulnerability and the associated level of effort or cost [67]. As such, they complement higher-level views and are most effectively developed by security experts as part of a broader security analysis framework.

3.3.4 Limitations

Despite its integrative design, SeaMonster presents some limitations. The tool focuses primarily on qualitative modeling and does not provide native support for probabilistic reasoning or quantitative risk estimation. Furthermore, attack trees and VCGs must be manually constructed by experts, a process that can be time-consuming and subject to analyst bias [46]. The approach is largely oriented toward design-time analysis and offers limited automation for incorporating real-world vulnerability data or scan results [46]. Finally, scalability can become problematic in large environments, as graphical models may grow complex and difficult to maintain.

3.4 SAGE

SAGE is an unsupervised visual analytics system designed to automatically derive attack graphs from intrusion detection alerts [2]. Unlike expert-driven modeling approaches, SAGE operates directly on raw alert data and does not require prior knowledge of system architecture, vulnerability databases or manual model construction [2]. It therefore operates after the fact, reconstructing attacks from alerts generated by a live, monitored environment, and cannot be applied to a system that is still being designed or built.

SAGE groups intrusion alerts from the same attacker and victim pair and attack stage within fixed time windows, producing hyper-alerts. It then derives temporally ordered episodes from these groups. Temporal and probabilistic dependencies between these episodes are learned

using a Suffix-based Probabilistic Deterministic Finite Automaton (S-PDFA) [2]. This model captures both short and long-term relationships between alerts while maintaining a deterministic and interpretable structure [2].

A central design objective of the S-PDFA is to emphasize infrequent but high severity alert sequences that are more likely to correspond to meaningful attack behavior [2]. To avoid overgeneralization, the model distinguishes between episodes that share identical signatures but occur in different contexts [2]. When the statistical properties of the preceding or subsequent alerts differ, such episodes are represented as distinct states in the automaton, preserving contextual differences between attack paths [2].

From the learned S-PDFA, SAGE extracts objective-oriented attack graphs on a per victim and per objective basis [2]. Each graph provides an aggregated view of relevant alerts, where attack paths originate from starting vertices and converge toward a designated objective vertex. Individual attack attempts are decomposed into separate paths, enabling comparative analysis across attackers and objectives [2]. Attacker identities are encoded through edge coloring, with source IP addresses associated with corresponding starting vertices [2].

To reduce visual clutter, SAGE applies a post-processing step that filters low severity episodes, which tend to occur frequently and can dominate the visualization [2]. This filtering highlights rare, high-impact behaviors and their contextual variations, improving interpretability for analysts.

SAGE is intended to complement existing Intrusion Detection Systems (IDS), which monitor network or host activity to identify potentially malicious events, and SIEM platforms, which aggregate and correlate security logs from multiple sources for centralized monitoring. By learning patterns directly from IDS alerts through the S-PDFA model, SAGE provides an AI-enabled capability, where attack graphs are derived automatically using unsupervised probabilistic learning rather than manually defined rules [2]. The extracted graphs support alert triage, forensic analysis and visual exploration, enabling analysts to reconstruct past attacks, compare alternative strategies and identify recurring behavioral patterns [2]. The approach further supports attacker fingerprinting and ranking based on the severity and uniqueness of observed behaviors, thereby enhancing cyber threat intelligence while reducing analyst workload.

Figure 9 illustrates an alert-driven attack graph generated by SAGE for a data exfiltration scenario over the *remoteware-cl* service. The nodes correspond to contextual states learned by the S-PDFA, while the edges represent probabilistic dependencies between temporally related

alert episodes. State identifiers encode the surrounding context of each alert, enabling the model to distinguish behaviorally different attack paths even when they involve identical alert signatures [2].

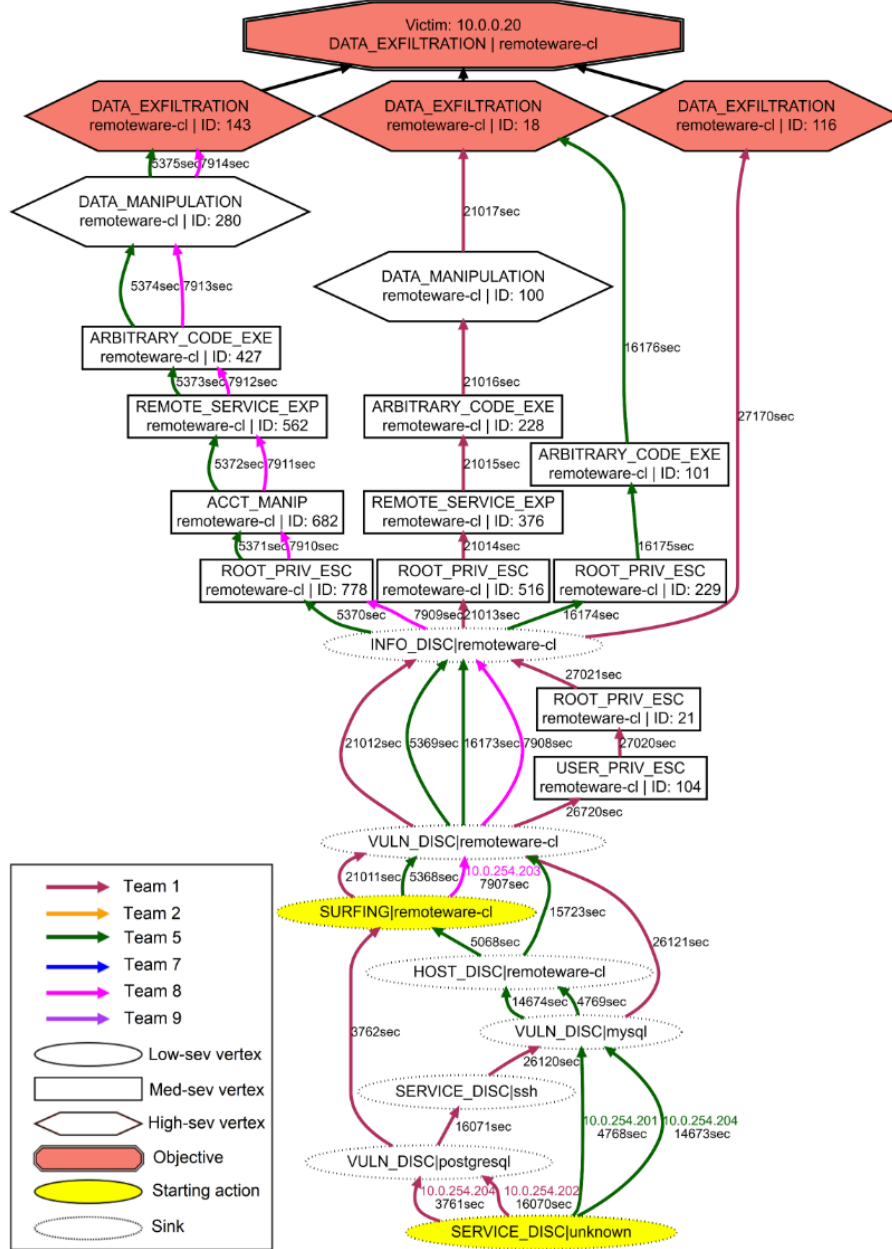


Figure 9: An alert-driven attack graph of data exfiltration over *remoteware-cl* (IDs are state identifiers, capturing context) [2]

3.4.1 Limitations

Although SAGE enables automated extraction of attack graphs from IDS alerts, several limitations affect its reliability and general applicability. Learning from infrequent sequences remains inherently difficult. When high severity sink states (terminal states representing attack objectives) are included in the training data, the resulting attack graph may present

distinct objective types for behaviorally similar sequences. While this occurs rarely, the current approach does not provide a mechanism to reconcile such inconsistencies [68].

A second limitation arises from the fact that only state sequences that reach a predefined objective are incorporated into the corresponding attack graph. In practice, adversaries may distribute their actions across multiple partial sequences, such that the complete attack path becomes visible only when these fragments are combined. The current construction process cannot natively merge such distributed behaviors [68].

Third, the S-PDFA model is sensitive to small perturbations in alert sequences at test time. Although robustness could be improved by augmenting the training set with perturbed traces, this would distort the true data distribution and was therefore not adopted [68]. In addition, the framework lacks a reliable metric for evaluating model interpretability, since standard model comparison criteria are not applicable when each objective yields an S-PDFA with a different structure, number of states and training data, making the resulting scores incomparable and largely reflective of model size rather than explanatory quality [68].

Beyond these methodological issues, SAGE also inherits practical limitations from its alert-driven nature. The approach depends heavily on the quality of IDS alerts. False positives, missing events or inconsistent signatures can directly distort the extracted graphs. Because SAGE reasons solely from observed alerts rather than system configuration or vulnerability knowledge, it may omit feasible but unobserved attack paths, and the resulting graphs represent behavioral correlation rather than verified exploit causality.

3.5 Comparative Analysis of Attack Graph Generation Approaches

Table 3 compares MulVAL, SeaMonster and SAGE across the dimensions most relevant to their use in security analysis. It highlights their distinct sources of knowledge, analytical paradigms and practical constraints.

Table 3: Comparative analysis of MulVAL, SeaMonster and SAGE

Dimension	MulVAL	SeaMonster	SAGE
Primary Objective	Predict feasible attack paths from system configuration and vulnerabilities.	Explain vulnerability causes, attack logic and countermeasures.	Reconstruct attacks from operational IDS alerts.
Knowledge Source	Formalized topology, vulnerabilities, privileges.	Expert knowledge encoded in graphical models.	Empirical alert data from IDS/SIEM.
Core Assumptions	Input data are accurate and sufficiently complete.	Analyst abstraction correctly reflects the system.	Alerts capture meaningful attacker behavior despite noise.
Automation Level	High: automatic graph generation after input acquisition.	Low: manual model creation and maintenance.	High: graphs learned automatically from sequences.
Analytical Paradigm	Logic reasoning (Datalog/XSB).	Multi-view conceptual modeling.	Probabilistic sequence learning (S-PDFA).
Nature of Graph	Logic-derived, exhaustive within the model.	Conceptual/explanatory structures.	Alert-driven empirical structures.
Attack Coverage	All attacks feasible under assumptions.	Only modeled scenarios.	Only attacks that generated alerts.
Lifecycle Phase	Pre-deployment configuration analysis and what-if reasoning.	Design-time threat and mitigation analysis.	Run-time monitoring and forensics.
Scalability Driver	Rule complexity and model size.	Human effort and visual complexity.	Alert volume and IDS quality.
Maturity	Long-term research adoption and tooling.	Academic prototype environment.	Research system validated on datasets.

The three approaches address complementary yet fundamentally different aspects of security analysis. SeaMonster emphasizes explanatory modeling, enabling analysts to reason about the causes of vulnerabilities, potential attack strategies and corresponding countermeasures within a unified conceptual framework. Its primary strength lies in supporting expert-driven understanding and communication, particularly during early design and risk assessment. This flexibility, however, depends heavily on manual modeling effort and analyst expertise, which limits scalability and repeatability.

SAGE adopts an empirical, behavior-centric perspective, deriving attack graphs directly from IDS alerts through probabilistic learning. This enables effective forensic reconstruction, alert triage and threat intelligence without requiring prior knowledge of the system architecture.

The approach is inherently constrained to attacks that generate alerts. It cannot reason about unobserved or hypothetical attack paths, nor assess the security impact of configuration changes.

MulVAL provides a predictive and formal approach, reasoning over explicit representations of configuration, vulnerabilities and attacker assumptions to enumerate all attack paths that are possible within the modeled environment. Although its expressiveness depends on the available rule set and input abstractions, MulVAL offers strong guarantees of soundness and coverage under those assumptions, making it particularly suitable for proactive analysis.

3.6 Chapter Summary

This chapter reviewed foundational representations used to generate attack graphs, including state graphs, Exploit Dependency Graphs, Logical Attack Graphs, Bayesian Attack Graphs and Multiple Prerequisite Attack Graphs. It then compared selected tools and examined MulVAL, SeaMonster and SAGE in detail. The comparison showed that MulVAL offers the most appropriate basis for this work because it combines formal reasoning, extensibility and automated generation from environmental and vulnerability facts.

The analysis also highlighted the limitations of static model generation and the lack of systematic integration between threat intelligence and attack graph reasoning. The next chapter presents the design and implementation of a framework that addresses these limitations by combining MulVAL with threat intelligence ingestion, environment correlation, orchestration and dynamic risk analysis.

Chapter 4

Design and Implementation

4.1 Problem Statement

Modern cybersecurity environments are characterized by continuous structural change. New vulnerabilities are disclosed on a daily basis, infrastructures evolve rapidly and adversaries constantly adapt their tactics, techniques and procedures [4]. MulVAL performs static logical inference over the facts supplied to a given execution, so that each generated attack graph reflects the system and intelligence available at that time [4]. This characterizes an individual inference run, rather than an inability to refresh its inputs. Existing MulVAL adapters can periodically update vulnerability repository data and translate scanner output before a later execution [5]. The proposed framework complements these capabilities by correlating refreshed threat intelligence with an asset inventory expressed through CPEs before invoking MulVAL. It therefore supports analysis where vulnerability scans cannot be performed or are inappropriate. It determines whether refreshed information changes the vulnerability facts for the modeled environment and therefore requires graph regeneration, or whether the information affects only the contextual risk of existing paths.

A separate gap remains between threat modeling practices and automated attack graph generation [69]. While threat modeling frameworks aim to identify and characterize potential adversarial behavior, their outputs are rarely integrated in a systematic manner into automated attack graph construction processes. The proposed framework addresses this gap by translating threat modeling outputs into MulVAL facts alongside correlated vulnerability facts.

As discussed in Chapter 2 and Chapter 3, attack graphs provide a structured mechanism for modeling multi-stage adversarial behavior and reasoning about feasible attack paths within a networked system [26]. They show which assets an attacker can reach and through which paths, but structural reachability alone does not establish which feasible paths should be mitigated first. The framework therefore combines CVSS, EPSS, KEV status and asset criticality

to calculate and rank path risk scores. It repeats this calculation during every execution, even when the graph structure is unchanged, so that revised threat intelligence or organizational context can update mitigation priorities without invoking MulVAL again. Accordingly, this dissertation examines how selective structural regeneration, threat modeling integration and contextual path risk scoring can be combined within a single workflow.

4.2 Conceptual Solution

This section presents the conceptual design of the proposed framework. It describes the architectural decisions, design goals and overall structure through which dynamic threat intelligence and threat modeling are integrated into attack graph generation. Its concrete realization, including the technologies and algorithms employed, is described in Chapter 4.3.

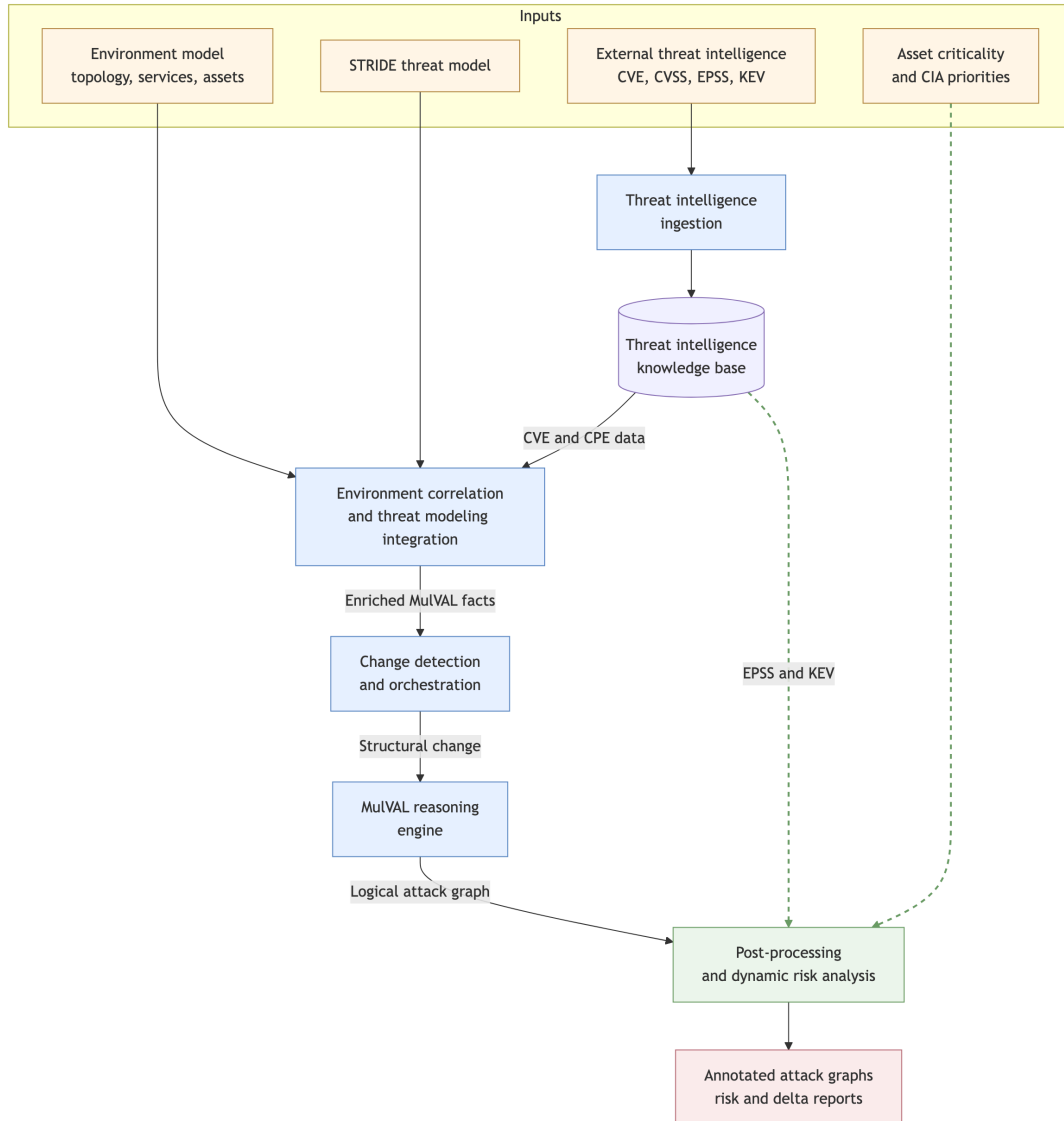


Figure 10: Conceptual architecture of the proposed framework.

Figure 10 shows the inputs, processing stages and outputs of the framework. The environment model and the STRIDE threat model are combined during environment correlation and threat modeling integration, while external threat intelligence is ingested into a shared knowledge base and matched to asset CPEs. The resulting enriched MulVAL facts pass through change detection and orchestration, which invokes the MulVAL reasoning engine when a structural change is detected. The logical attack graph is then combined in post-processing with current risk information from the knowledge base and with the configured asset criticality and CIA priorities, producing annotated attack graphs, risk reports and delta reports. The following sections develop the design decisions represented in this flow.

4.2.1 Selection of the Logical Reasoning Engine

The selection of the logical reasoning engine constitutes a critical architectural decision within the proposed framework. The reasoning engine determines how system configurations, vulnerabilities and adversarial capabilities are represented and how potential attack paths are derived from these inputs.

Several attack graph generation approaches were analyzed in Chapter 2 and Chapter 3. Among the evaluated options, MulVAL emerges as the most suitable foundation for the proposed framework. MulVAL represents one of the most established and widely adopted academic systems for logical attack graph generation [45]. Its extensive use in prior research demonstrates both its robustness and its extensibility across multiple security analysis applications.

MulVAL provides automated attack graph generation based on a formal logical reasoning model. Unlike purely topological or alert-driven approaches, MulVAL relies on a Datalog-based inference engine, enabling explicit modeling of exploit preconditions, vulnerability relationships, privilege escalation mechanisms, and dependency chains between attack steps. Through declarative inference rules, attack paths can be derived logically rather than through hard-coded procedural logic [60]. This approach significantly improves modularity and extensibility.

Another important advantage of MulVAL lies in its ability to integrate heterogeneous system information. The framework supports inputs describing host configurations, network connectivity and vulnerability information such as CVE-based data [60]. This flexible fact-based representation makes MulVAL particularly suitable for integration with external data sources, including CTI and OSINT pipelines, which form a central component of the dynamic framework proposed in this dissertation.

MulVAL also cleanly separates the facts describing an environment from the interaction rules that reason over them. The model can therefore be updated simply by changing its facts,

without touching the reasoning logic, which provides a natural foundation for incremental updates and dynamic re-evaluation as the environment evolves.

Due to its strong academic foundation, open-source availability, and demonstrated extensibility, MulVAL has been widely adopted as a research platform for attack graph generation and security analysis. Its extensive use in prior work further highlights its suitability as a foundational reasoning engine for advanced security modeling frameworks [45]. Consequently, MulVAL is selected in this work not merely as an attack graph generation tool, but as the core logical inference engine underpinning the proposed dynamic attack modeling framework.

4.2.2 Selection of the Threat Modeling Framework

Threat modeling frameworks provide systematic methodologies to identify, analyze and mitigate potential threats within a system by examining its architecture, assets and trust boundaries [7]. Integrating such methodologies into automated security analysis enables the incorporation of adversarial perspectives during early stages of system analysis and allows threat intelligence to be contextualized within the system’s architecture.

As discussed in the problem statement, a significant gap exists between traditional threat modeling practices and automated attack graph generation. While threat modeling frameworks identify potential adversarial actions and system weaknesses, their outputs are typically produced as qualitative analyses or documentation artifacts that are rarely integrated into automated reasoning systems [69]. Consequently, attack graph generation often relies primarily on vulnerability data and system configuration information, without incorporating structured representations of adversarial intent or threat scenarios.

In addition to the logical reasoning engine, the framework requires a structured methodology capable of translating threat modeling outputs into machine-processable representations. Of the frameworks analyzed in Chapter 2, PASTA and OCTAVE are too heavyweight and documentation intensive for an automated pipeline, while VAST’s proprietary nature limits its use in open, research-oriented settings [15]. STRIDE is therefore the natural candidate.

STRIDE decomposes a system into components and data flows and evaluates each against a fixed set of threat categories, so that individual threats can be tied to specific components and trust boundaries and then translated into structured, machine-processable facts [11]. This structured output makes STRIDE suitable for the automated pipeline developed in this work.

Another important advantage is its compatibility with logical reasoning systems. Because STRIDE threats correspond to clearly defined adversarial actions, they can be translated into logical predicates that enrich the fact base used by the MulVAL inference engine. For instance,

threats such as identity spoofing or privilege escalation can be mapped to assumptions about attacker capabilities, authentication weaknesses or trust violations. This mapping enables attack graph generation to incorporate adversarial assumptions derived from threat modeling, helping bridge the gap between qualitative threat analysis and automated attack path generation.

Although STRIDE is often criticized for its software-centric focus and limited business context, its simplicity, structured taxonomy and wide adoption make it the most practical choice for this technical, automated setting. STRIDE is therefore selected as the threat modeling methodology integrated into the proposed architecture [12].

4.2.3 Design Goals

The design of the framework is guided by four goals: automated threat intelligence acquisition, a modular architecture, dynamic model evolution and efficiency during repeated analysis. Together, these goals define the functional and operational requirements of the proposed solution.

First, the framework emphasizes automation in the acquisition and integration of threat intelligence. It must support the automated ingestion, normalization and processing of multiple intelligence sources, including vulnerability disclosures with their associated CVSS severity metrics, exploit prediction scores (EPSS) and known exploited vulnerability catalogues (KEV), while remaining extensible to additional structured intelligence formats such as STIX. By automating these processes, the framework reduces the latency between intelligence publication and model adaptation, ensuring that attack graphs remain analytically relevant in rapidly evolving security environments.

Second, the architecture is designed to be modular in order to accommodate the heterogeneity of intelligence sources and analytical components. Intelligence ingestion, normalization, scoring and reasoning modules are intentionally decoupled, allowing individual components to evolve independently. This modular structure enables the integration of alternative scoring mechanisms, the replacement or extension of attack graph engines and the incremental enhancement of reasoning capabilities. Such flexibility aligns with the rule-based architecture used by MulVAL [51].

A third design goal concerns dynamic model evolution and efficiency during repeated analysis. The framework must correlate newly disclosed vulnerabilities with asset CPEs and regenerate the graph only when the resulting facts alter the attack surface. Updates to EPSS scores, KEV status or asset criticality should instead update the prioritization of existing paths. It should also enable temporal comparisons between successive model states. By avoiding MulVAL

execution when the structural fact set remains unchanged, the framework limits unnecessary recomputation. This is relevant because attack graph generation can become costly as system and vulnerability models grow [26]. The performance benefit requires dedicated measurement in larger environments and is outside the scope of this work.

4.2.4 Conceptual Architecture of the Framework

As shown in Figure 10, the proposed framework adopts a layered, modular architecture built around MulVAL. It separates structural attack reasoning from intelligence acquisition, correlation and contextual risk computation, allowing each layer to evolve independently while retaining MulVAL's established reasoning model.

MulVAL derives the possible attack paths by applying a fixed set of interaction rules to a collection of facts that describe the environment and its vulnerabilities. The framework leaves these rules unchanged and lets only the facts evolve over time. This preserves compatibility with existing MulVAL rule sets and MulVAL's polynomial time performance, while all dynamic behavior comes from the surrounding layers that generate and update the facts MulVAL consumes.

Conceptually, the framework is organized into a sequence of cooperating layers, each responsible for a well-defined stage of the analytical process:

1. Threat intelligence ingestion: acquires and normalizes external intelligence into a database.
2. Environment correlation and threat modeling integration: binds relevant vulnerabilities and STRIDE-derived threats to the modeled assets, translating both into logical facts.
3. Change detection and orchestration: governs when the attack graph must be recomputed.
4. MulVAL reasoning engine: generates the LAG.
5. Post-processing and risk analysis: enriches, scores and visualizes the resulting model.

The overall system operates according to an event-driven update model, in which changes in the threat landscape or in the modeled environment trigger the selective recomputation of the attack graph. Such events may include newly disclosed vulnerabilities, significant updates to EPSS scores, modifications to the asset inventory or the appearance of vulnerabilities in the KEV catalogue. When an event affects the logical structure of the model, for example by introducing a new vulnerability on a reachable host, the attack graph is regenerated. When it affects only the contextual weighting of existing elements, such as a revised EPSS score, the model is re-scored without structural regeneration. This separation preserves the logical model while avoiding a new MulVAL execution for contextual updates.

The conceptual roles of the layers are described in Chapter 4.2.5 through Chapter 4.2.9. The implementation of these layers is described in Chapter 4.3, including the services, data flows and algorithms that realize the framework.

4.2.5 Threat Intelligence Ingestion Layer

The first architectural component is responsible for the automated acquisition and normalization of external threat intelligence. This layer retrieves vulnerability disclosures and their CVSS severity metrics, EPSS exploitation probabilities and KEV catalogue updates from authoritative sources. Retrieved data is parsed, normalized and deduplicated before being stored in a centralized Threat Intelligence Database. The layer is designed to be extensible, so that additional structured intelligence formats, such as STIX, can be incorporated without affecting the downstream reasoning stages.

This database functions as a persistent knowledge repository that enables temporal comparison of intelligence states and supports incremental updates. By decoupling intelligence acquisition from attack reasoning, the framework allows intelligence connectors to evolve independently without affecting the logical inference mechanisms.

4.2.6 Environment Correlation and Threat Modeling Integration Layer

This layer links external intelligence with the local environment and, in the same step, incorporates information from threat modeling. Vulnerabilities are matched to the infrastructure through a mapping pipeline in which CVE identifiers are associated with CPE entries and then matched against the software and services installed on each asset. In parallel, STRIDE-based threat models are translated into logical facts compatible with MulVAL, so that adversarial assumptions identified during threat modeling participate in attack graph generation alongside concrete vulnerabilities.

The result is a set of vulnerability and threat facts bound to specific hosts and services. This stage therefore determines whether newly acquired intelligence introduces structural changes to the attack surface represented in the attack graph.

4.2.7 Change Detection and Model Orchestration

The orchestration layer governs the dynamic evolution of the model. It monitors both environmental changes and intelligence updates to determine whether logical recomputation of the attack graph is required. Structural changes such as new vulnerabilities affecting assets, topology modifications, service exposure changes, or the addition and removal of assets trigger regeneration of MulVAL input facts and execution of the reasoning engine.

When no structural changes are detected, logical recomputation is skipped. This conditional execution preserves the logical model while avoiding graph regeneration when only contex-

tual information has changed. The evaluation verifies this behavior but does not measure its performance at large scale.

4.2.8 Attack Graph Generation

When structural updates are detected, MulVAL is executed to regenerate the LAG. The resulting model captures feasible attack paths, privilege escalation chains, and the logical dependencies between exploit steps. Because the underlying reasoning engine remains unchanged, the framework maintains compatibility with established MulVAL rule sets while benefiting from automated updates to environmental facts.

4.2.9 Post-Processing and Dynamic Risk Analysis

While MulVAL focuses on structural reachability, the final architectural layer introduces dynamic risk contextualization. After the attack graph is generated, vulnerability nodes are enriched with EPSS exploitation probability and KEV exploitation status. The risk calculation also uses asset criticality levels and CIA priorities specified by the analyst or organization in the configuration supplied to the framework before graph generation.

These attributes enable the computation of higher-level analytical metrics, including path level risk scores, identification of the highest risk attack paths, and comparative analysis between successive model states. To support temporal comparison, the post-processing component compares the risk report generated in the current execution with the report from the preceding execution. It identifies added or removed CVEs, changes in the highest-ranked paths, variation in their average risk score and newly observed KEV entries. The corresponding graph version identifiers are recorded to distinguish structural changes from risk re-prioritization.

Importantly, this post-processing stage is executed even when the logical structure of the graph remains unchanged. As a result, threat intelligence updates, such as changes in EPSS scores, can dynamically influence risk prioritization without requiring full regeneration of the attack graph.

4.3 Implemented Framework

Whereas Chapter 4.2 defined the framework at a conceptual level, this section describes its concrete realization. The implemented solution materializes the layered, event-driven architecture as a set of independent, containerized services orchestrated through a single reproducible workflow. Each conceptual layer corresponds to a dedicated service that com-

municates either through a shared PostgreSQL¹⁶ knowledge base or through file artifacts written to a common test case directory mounted into the containers. This subsection details the deployment model, the individual services, the data structures they exchange and the algorithms that implement dynamic risk analysis.

The framework and its services were designed and implemented as part of this work. These artifacts include the threat intelligence ingestion, environment correlation, STRIDE-to-MulVAL translation, change detection and orchestration, risk scoring, delta analysis and graph visualization components. MulVAL is reused as an external logical attack graph generation engine, and its interaction rules remain unmodified. Docker¹⁷ and XSB¹⁸ are third-party technologies used to deploy or support the framework, while VulnCheck¹⁹, FIRST EPSS and the CISA KEV catalogue provide external data rather than artifacts developed in this work.

4.3.1 System Overview and Deployment

The framework is implemented as a modular pipeline composed of five processing services and a central database, deployed and coordinated with Docker Compose. Each stage of the conceptual architecture is encapsulated in its own container with explicitly declared dependencies, ensuring that services execute in the correct order and that intermediate artifacts are produced before they are consumed. The complete pipeline, from input artifacts to the final risk report, is illustrated in Figure 11.

¹⁶<https://www.postgresql.org/>

¹⁷<https://www.docker.com/>

¹⁸<https://xsb.com/xsb-prolog/>

¹⁹<https://docs.vulncheck.com/>

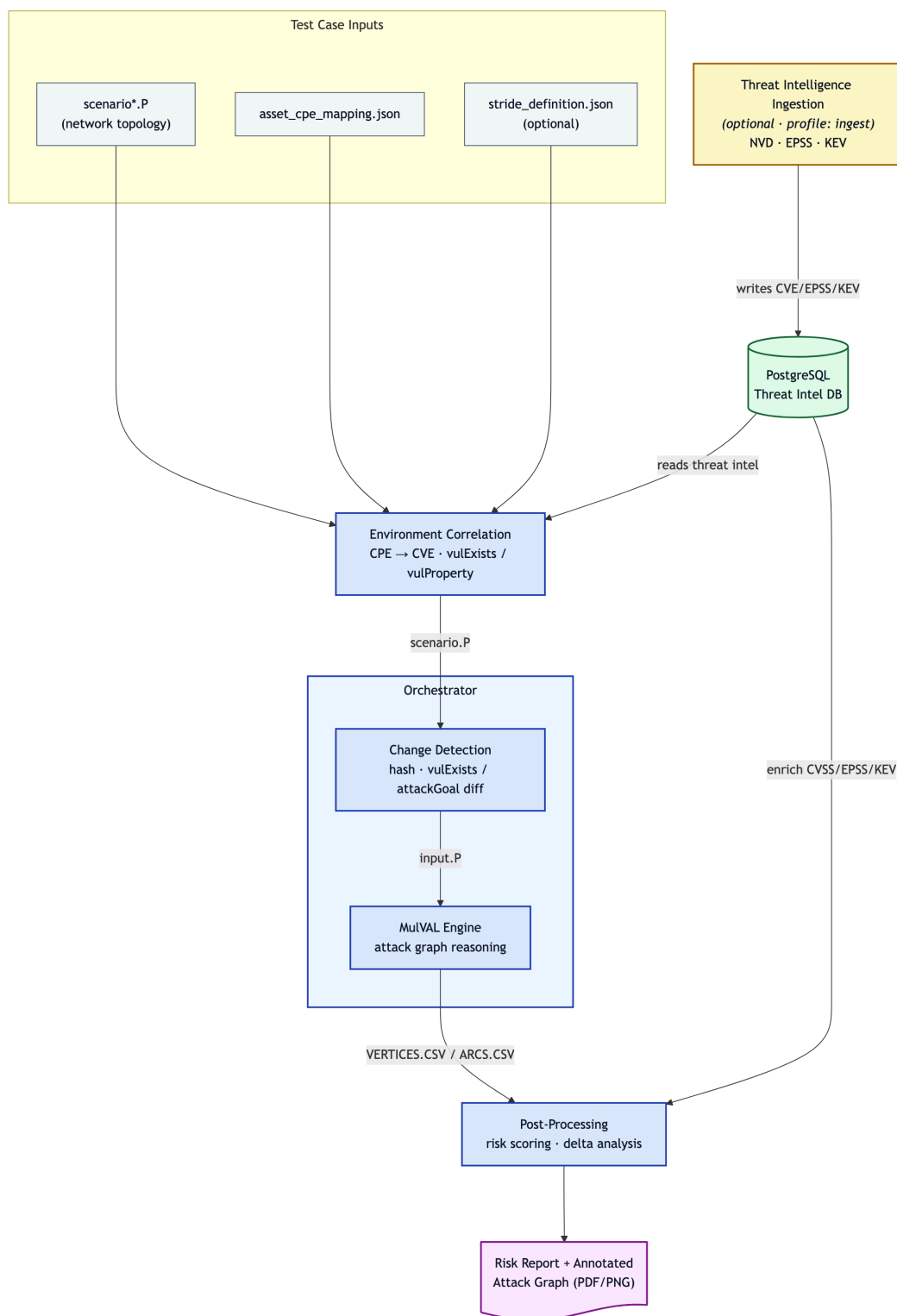


Figure 11: End-to-end architecture of the proposed pipeline, from environment and threat intelligence inputs to the annotated attack graph and risk report.

The execution order is enforced through service dependencies. Environment correlation waits for the threat intelligence database to be available, the orchestrator waits for correlation to

complete and post-processing waits for the orchestrator. Two entry points expose the pipeline to the user through a Makefile: `make run` executes the full workflow reusing the existing contents of the threat intelligence database, whereas `make ingest` additionally activates the optional ingestion service to refresh the database before analysis. The target environment is selected through a `SCENARIO` variable, which mounts the corresponding test-case directory into the relevant containers. This design directly supports the modularity and reproducibility goals defined in the conceptual solution, as each service can be developed or replaced independently while sharing well-defined interfaces.

The services exchange information through two complementary channels. Persistent threat intelligence (CVE, CVSS, EPSS and KEV data) is stored in a shared PostgreSQL database, while environment-specific artifacts, the Prolog scenario, the generated attack graph and the risk reports are exchanged as files within the mounted test-case directory. This dual channel design keeps the two kinds of state apart: the threat intelligence, which evolves slowly and is common to any organization, is managed separately from the environment-specific model, which changes far more frequently as new scenarios are analyzed.

4.3.2 Threat Intelligence Database

At the centre of the framework lies a PostgreSQL database that acts as the persistent knowledge base shared by the ingestion, correlation and post-processing layers. Its schema is intentionally compact and is organized around the vulnerability identifier (CVE) as the principal key, so that severity, exploitability and exploitation-status signals can be retrieved efficiently during correlation and enrichment. The logical model is depicted in Figure 12.

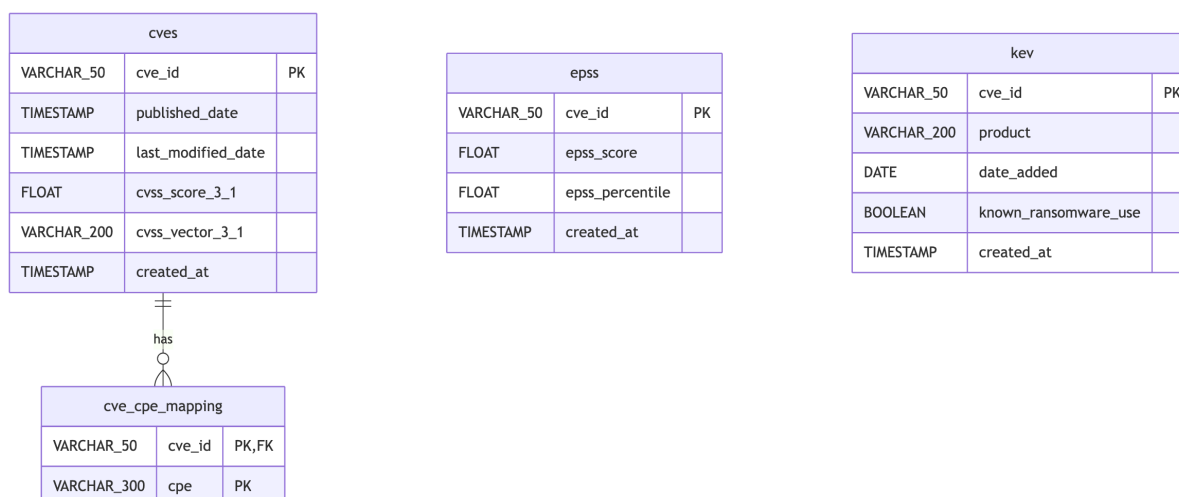


Figure 12: Logical model of the Threat Intelligence Database, organized around the CVE identifier.

The schema comprises four tables. The `cves` table stores each vulnerability together with its publication and modification timestamps and its CVSS v3.1 base score and vector. The `cve_cpe_mapping` table associates each CVE with the products it affects, expressed as CPE identifiers, thereby connecting each vulnerability to the specific software in which it occurs. The `epss` table holds the EPSS exploitation probability and percentile for each CVE, while the `kev` table records whether a vulnerability appears in CISA’s Known Exploited Vulnerabilities catalogue, including the affected product, the date of addition and a flag indicating known ransomware usage. Although the `epss` and `kev` data stand in a one-to-one relationship with `cves` and could be folded into it as additional columns, they are kept as dedicated tables so that each intelligence source maps to its own table, mirroring the modular ingestion design. This separation also leaves room to store richer per-source information that is not yet exploited by the scoring model but could be in the future, such as the EPSS percentile. The schema is provisioned automatically when the database container is initialized, ensuring a consistent structure across deployments. Its complete definition is reproduced in Appendix C.

4.3.3 Threat Intelligence Ingestion

The ingestion service populates the Threat Intelligence Database from authoritative external sources. It aggregates three complementary categories of intelligence, as shown in Figure 13.

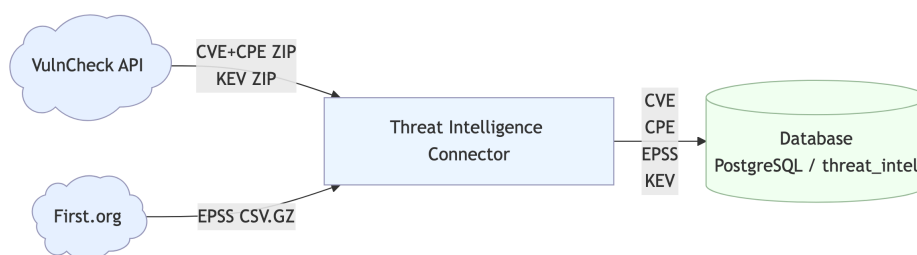


Figure 13: Threat intelligence ingestion: CVE/CPE and KEV data from the VulnCheck API and EPSS scores from FIRST.org.

NVD CVE records, including CVSS metrics and CPE configurations, together with the CISA KEV catalogue, are obtained through the VulnCheck API, whereas EPSS scores are retrieved as a compressed dataset from FIRST.org. The service is structured around a controller that orchestrates the sequential synchronization of the three sources, a set of source-specific handlers that fetch and transform each dataset, and a repository component that performs context-managed, batched writes into PostgreSQL. For each CVE, the handler discards records whose source provided summary marks the identifier as rejected. Such identifiers no longer represent valid vulnerability records, for example because they were withdrawn, duplicated or assigned in error, and are therefore excluded from the risk analysis. The handler then preferentially extracts the CVSS v3.1 base score and vector, falling back to v3.0 when the former is absent.

This choice reflects the predominance of CVSS v3.x in operational vulnerability records at the time of implementation [31]. Supporting CVSS 4.0 requires extensions to the ingestion and scoring logic to accommodate its revised metric model, and is therefore identified as future work. Synchronization proceeds in three stages: CVE records and their CPE mappings are ingested first, followed by EPSS scores and finally the KEV catalogue. Records are written using batched upsert operations, which insert new entries and update existing ones in place, while new CVE-CPE mappings are added only when they do not already exist, so that repeated runs never create duplicates. This keeps the knowledge base consistent across repeated synchronization runs while accommodating the large volume of vulnerability data efficiently. By decoupling intelligence acquisition from the reasoning stages, the ingestion connectors can evolve, or new sources can be added, without impacting the rest of the pipeline.

4.3.4 Environment Correlation

The environment correlation service is the component that binds external intelligence to the specific infrastructure under analysis, transforming a generic network description into an enriched, MulVAL-ready scenario. Its inputs and outputs are summarized in Figure 14.

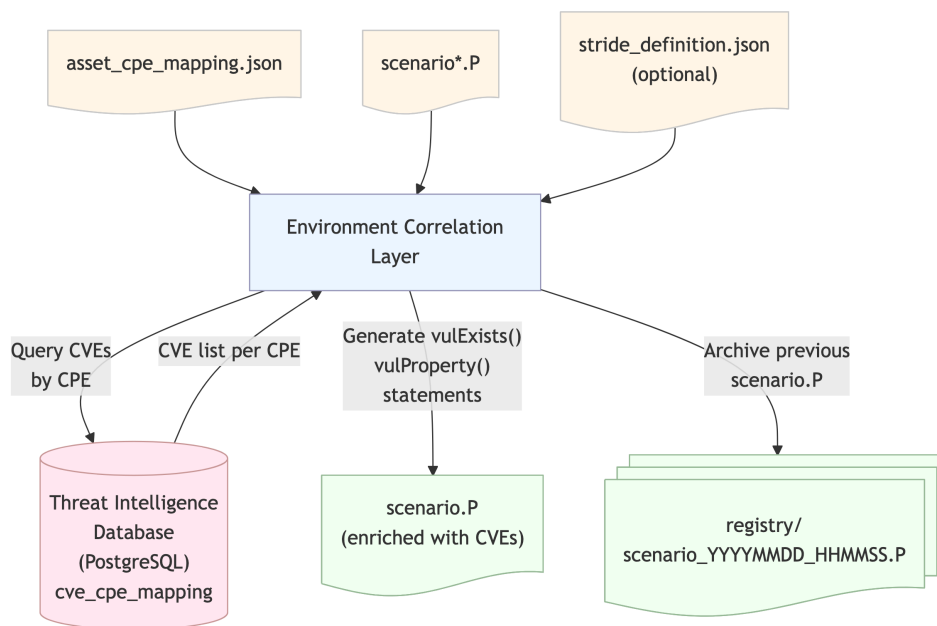


Figure 14: Environment correlation: asset CPEs are matched against the database to inject vulnerability facts, optionally enriched with STRIDE derived threats.

Before any correlation work begins, the service applies a pre-flight quality gate to the Threat Intelligence Database. It queries the row count of each intelligence table and compares it against a configurable minimum. If any table is empty or only partially populated, for instance because an ingestion run failed midway, the pipeline is aborted immediately rather than producing an attack graph from incomplete intelligence. The minimum thresholds are

conservative floors set below the expected data volumes and can be overridden in the service configuration.

The service consumes a base scenario file (scenario*.P) describing the network topology, connectivity and attack goals, together with an `asset_cpe_mapping.json` file that associates each asset with the CPE identifiers of its modeled services and client applications. An asset can therefore declare multiple CPEs, with each mapping identifying the vendor, product and version of a specific component. These entries are configuration inputs supplied by the analyst as part of each scenario definition. In the simulated scenarios, they were selected from the software stack defined for each asset and recorded explicitly in the corresponding `asset_cpe_mapping.json` file. The framework does not infer CPEs from the network model. In an operational deployment, the same mapping could instead be obtained from an asset inventory, software discovery tool or vulnerability scanner. When a current asset inventory is available, the CPE-based approach avoids direct interaction with the target systems and the execution of vulnerability scans during correlation. It is therefore suitable for environments in which such interaction is undesirable or not permitted, although the quality of the resulting facts depends on the accuracy of the inventory. For every CPE, the service queries the database for the corresponding CVEs and generates the appropriate MulVAL facts, namely `vulExists()` statements binding a vulnerability to a host and service, and `vulProperty()` statements describing how the vulnerability can be exploited and its consequence. The choice of exploitation primitive depends on the type of component affected: network-facing services use `remoteExploit`, whereas client-side software such as browsers or document readers uses `remoteClient`. In both cases, the consequence is modeled as privilege escalation because the framework preserves MulVAL's standard interaction rules. Vulnerabilities whose primary effect is data disclosure, data modification or service disruption are therefore not represented as distinct logical consequences in the attack graph. Their confidentiality, integrity and availability impacts still contribute to post-processing risk scores. These statements are injected into the scenario immediately after the configuration comment of the relevant asset, producing an enriched scenario.P. Listing 2 shows an illustrative fragment of the generated output, and the complete set of input artifacts for one scenario is provided in Appendix D.

```
/* configuration information of webServer */  
vulExists(webServer, 'CVE-2021-41773', httpd).  
vulProperty('CVE-2021-41773', remoteExploit, privEscalation).  
networkServiceInfo(webServer, httpd, tcp, 80, apache).
```

Listing 2: Illustrative fragment of the enriched scenario.P produced by the environment correlation layer.

Optionally, the service integrates a STRIDE threat model supplied as `stride_definition.json`. A dedicated handler performs two reference checks. First, every threat must target an asset declared in the STRIDE model’s own asset list. Second, every asset declared in the STRIDE model must also exist in the system configuration given by the asset mapping. It further verifies that each threat belongs to a valid STRIDE category. Validated threats are then translated into additional vulnerability facts: each STRIDE category is mapped to a MulVAL exploitation primitive and each declared impact to a consequence. The rationale and complete mapping tables are given in Appendix B. This allows adversarial assumptions derived from threat modeling to participate in attack graph generation alongside concrete CVEs, realizing in practice the conceptual bridge between qualitative threat modeling and automated reasoning. To support temporal comparison and reproducibility, each previously generated scenario is archived, with a UTC timestamp, into a rotating registry that retains the most recent versions.

4.3.5 Orchestration

The orchestration service governs when the MulVAL reasoning engine is invoked, implementing the conceptual distinction between structural and contextual updates. Its decision logic is summarized in Figure 15.

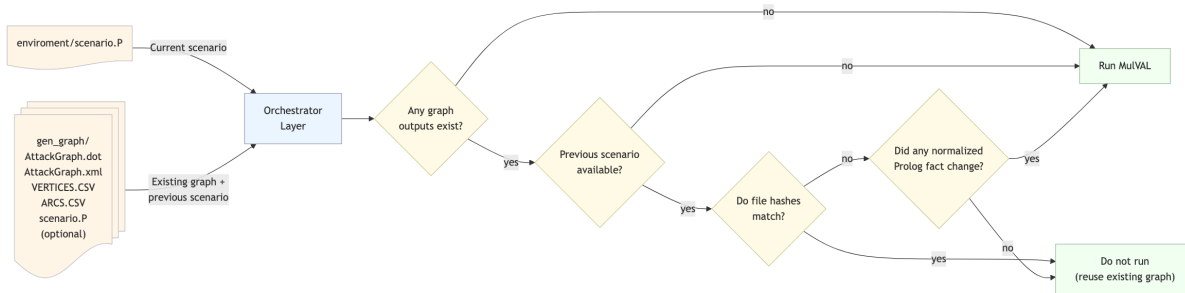


Figure 15: Change detection logic of the orchestrator: MulVAL is executed when a previous graph or scenario is absent, or when the scenario fact set differs after comments and formatting are ignored.

When invoked, the orchestrator first checks whether attack graph outputs already exist. If none are present, MulVAL is executed unconditionally. If a graph exists but no archived scenario is available, the graph is also regenerated. Otherwise, the service compares the current enriched scenario against the previously processed one using a two-phase strategy: a SHA-256 hash of both files acts as an inexpensive equality test, and only when the hashes differ does the comparator extract and compare the scenario facts. The comparison ignores comments and whitespace, then identifies additions, removals and modifications to facts. It consequently detects changes to `vulExists` and `attackGoal`, as well as changes to topology and service configuration facts such as `hacl` and `networkServiceInfo`. Adding or removing an

asset also changes its associated facts and is detected through the same mechanism. If the fact sets are unchanged, regeneration is skipped and the existing graph is reused. If any difference is detected, or if execution is explicitly forced, MulVAL is re-executed. This change detection strategy, implemented by a scenario comparator and a MulVAL executor component, prevents redundant graph regeneration during repeated analyses of an unchanged scenario. Its impact on processing time and resource consumption was not measured in this work.

4.3.5.1 Attack Graph Generation

When regeneration is required, the orchestrator invokes MulVAL on the enriched scenario. MulVAL applies its static interaction rules to the environmental and vulnerability facts and derives the LAG, capturing feasible attack paths, privilege escalation chains and the logical dependencies between exploit steps. The engine emits the graph both as a Graphviz²⁰ description (`AttackGraph.dot`) and as a pair of comma-separated files, `VERTICES.CSV` and `ARCS.CSV`, which enumerate the graph nodes and edges respectively. Nodes are typed as disjunctive (OR) capability nodes, conjunctive (AND) rule application nodes, or LEAF nodes representing base facts such as vulnerabilities and network configuration. Because the rule base is left unmodified, the framework remains fully compatible with established MulVAL rule sets while benefiting from the automatically updated fact base produced by the preceding layers.

4.3.6 Post-Processing and Dynamic Risk Scoring

The post-processing service transforms the structural attack graph produced by MulVAL into actionable risk intelligence. It implements the contextual dimension of the framework, enriching and scoring the graph using threat intelligence even when its logical structure is unchanged. The service operates as a multi-stage pipeline: it parses the MulVAL output into an in-memory AND/OR graph, enriches vulnerability nodes with data from the database, computes path-level risk scores, performs delta analysis against the previous report and finally renders annotated visualizations. Its top-level inputs and outputs are shown in Figure 16.

²⁰<https://graphviz.org/>

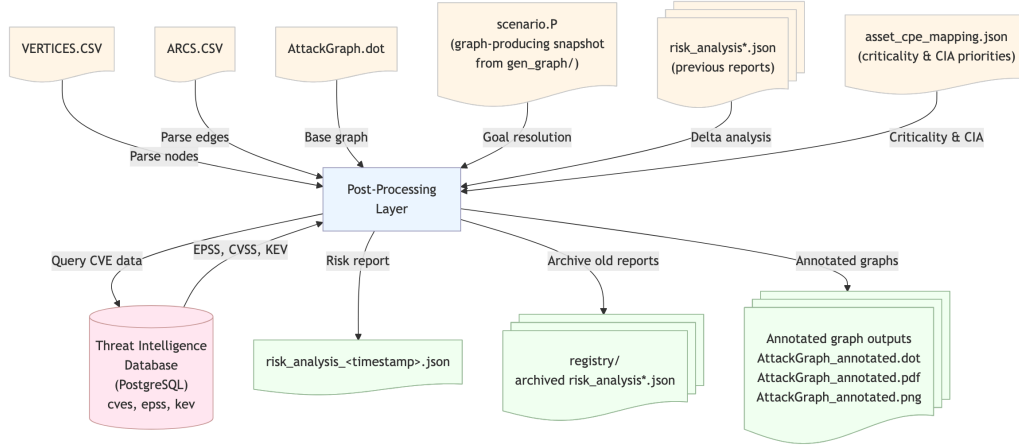


Figure 16: Top-level view of the post-processing service: its file and database inputs and the risk report and annotated graphs it produces.

The risk scoring model proposed in this work provides a transparent and reproducible comparative prioritization of attack paths. It is not a calibrated estimate of the probability that a path will be exploited successfully in a particular environment. The model combines signals with distinct roles. EPSS captures estimated exploitation likelihood. CVSS captures technical severity. KEV identifies confirmed exploitation. The CIA priorities and asset criticality represent the organization’s assessment of impact. The numerical multipliers are design parameters selected to retain a bounded 0 to 10 scale and to make the contribution of each signal explicit. They can be adjusted to reflect the risk appetite and operational priorities of a given organization.

Risk is computed in two stages. First, each vulnerability leaf node receives a **base risk score** on a 0 to 10 scale that combines its exploitation probability, severity, impact profile and exploitation status:

$$\text{base_risk}(v) = \min\left(10, \text{EPSS}(v) \times \frac{\text{CVSS}(v)}{10} \times k_{\text{CIA}}(v) \times k_{\text{KEV}}(v) \times 10\right)$$

where $\text{EPSS}(v)$ is the exploitation probability, $\text{CVSS}(v)$ is the CVSS v3.1 base score (falling back to the v3.0 base score when no v3.1 score is available), and the two multipliers capture asset specific impact prioritization and active exploitation respectively. The Confidentiality, Integrity and Availability (CIA) multiplier weights the confidentiality, integrity and availability impacts of the vulnerability according to per asset priority levels. These priority levels are contextual inputs defined by the organization and assigned by the analyst for each asset in the `asset_cpe_mapping.json` file. They express the relative importance that the organization attributes to confidentiality, integrity and availability for that asset. If an asset has no explicit

configuration, the post-processing service applies global default priorities, which assign MEDIUM to all three dimensions:

$$k_{\text{CIA}}(v) = 0.5 + \frac{C(v) \cdot w_C + I(v) \cdot w_I + A(v) \cdot w_A}{w_C + w_I + w_A}$$

in which $C(v)$, $I(v)$ and $A(v)$ denote the confidentiality, integrity and availability impacts read from the CVSS vector. Each takes the value 0 for no impact, 0.5 for low impact and 1.0 for high impact. The values w_C , w_I and w_A are weights derived from the configured priority of each dimension. The weighting is interpreted relatively: when all three dimensions share the same priority level the multiplier collapses to a neutral value of 1.0, so that prioritization only takes effect when an analyst deliberately favors one dimension over the others. The KEV multiplier amplifies vulnerabilities with confirmed exploitation:

$$k_{\text{KEV}}(v) = \begin{cases} 1.5 & \text{if } v \in \text{CISA KEV} \\ 1.0 & \text{otherwise} \end{cases}$$

In the second stage, the scorer enumerates the root-to-leaf attack paths and assigns each path P a *path risk score* derived from its highest risk vulnerability v^* , adjusted by asset criticality and path directness:

$$\text{path_risk}(P) = \min(10, \text{base_risk}(v^*) \times \alpha \times (1 + d(P)))$$

where α is a configurable asset criticality factor selected from four named levels: LOW (0.4), MEDIUM (0.7), HIGH (1.0) and CRITICAL (1.3). These values are design parameters rather than estimates calibrated from experimental data or observed incidents. They retain 1.0 as the neutral reference, reduce the contribution of assets below the HIGH level and amplify the contribution of CRITICAL assets, while the outer minimum keeps the final path risk score within the 0 to 10 scale. The full set of scoring parameters is listed in Appendix A. The term $d(P) = \frac{1}{\log_2}(|P| + 2)$ is a directness factor. Because its value decreases as the path grows longer, it gives greater weight to shorter paths, which represent more direct routes to the goal. This is a prioritization choice and does not express an independently measured probability. Since the base risk scales linearly with the CVSS score, a vulnerability with a zero CVSS base score contributes no risk. Two boundary cases are handled explicitly. When the vulnerabilities on a path carry no CVSS data at all, the path risk cannot be computed and is left undefined. Such paths are deliberately ranked at the very top of the report so that an analyst can review them manually. Conversely, a path that reaches its goal without traversing any vulnerability has no vulnerability based risk to quantify and is assigned a score of zero, ranking it last.

To enumerate the root-to-leaf paths, the graph is traversed depth first from each goal and bounded by a configurable maximum depth that also guards against cycles. The resulting paths are then deduplicated through content hashing and ranked by descending risk, with the highest-ranked paths retained in the summary report. This scoring model implements the dynamic reprioritization goal of the framework, since a change in any input signal, such as a rising EPSS score or a newly catalogued KEV entry, immediately propagates to the path rankings without requiring structural regeneration of the graph.

4.3.6.1 Delta Analysis and Decision Support

A distinctive feature of the proposed solution is its ability to reason about how risk evolves over time. Before each run, the previous risk report is archived, and after scoring, a delta analyzer compares the current and previous reports to surface meaningful changes. The resulting delta report identifies newly introduced and removed CVEs, changes in the ranking of the most critical attack paths, the variation in average top path risk and any newly catalogued KEV entries. This comparison between successive model states (t_0 versus t_1) provides analysts with direct visibility into emerging threats and shifting exposures, realizing the temporal comparison capability envisioned in the conceptual design.

To translate these analytical results into intuitive insight, this work extends the post-processing service with a component that renders color coded visualizations of the attack graph. The detailed annotated graph overlays the computed risk levels onto the MulVAL output, highlighting the most critical attack paths and the vulnerabilities that drive them. This detailed view is always produced, both as Graphviz sources and as rendered PDF and PNG images, while the complete analysis is persisted as a structured JSON report. An example of a detailed annotated attack graph generated by the post-processing service is shown in Figure 17.

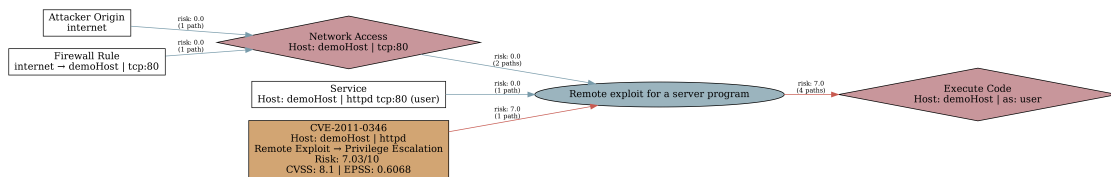


Figure 17: Illustrative detailed attack graph.

4.3.6.2 Host-Level Summary Graph

The attack graph produced by MulVAL is complete but hard to read: in complex environments it easily contains hundreds or thousands of nodes that mix individual exploit steps, rule applications and base facts. This detail is necessary for the formal reasoning, but it offers little help to an analyst who must quickly decide where to act. To address this visualization limitation, this work further extends the post-processing service with an optional condensed *host-level summary graph*. It reduces the detailed model to a simple host-to-host map built

from the highest-ranked attack paths. This representation improves visual legibility but does not reduce the cost of MulVAL inference or path enumeration.

In this graph, each node is a host rather than a single fact, and carries the information an analyst actually needs: the highest privilege the attacker gains on that host (for example, user or root), its worst case path risk, whether any of its vulnerabilities are listed in the CISA KEV catalogue, and the CVEs used to reach it. Host nodes use the same risk colors as the detailed graph, and separate colors mark where the attacker starts and the target host. Arrows show the attacker moving from one host to the next and are labelled with how many of the top paths pass through each step, so the hosts that most attack paths go through, are easy to spot. A configurable limit on how many paths are included lets the analyst balance completeness against readability. A representative host-level summary generated from a multi-tier illustrative topology is shown in Figure 18.

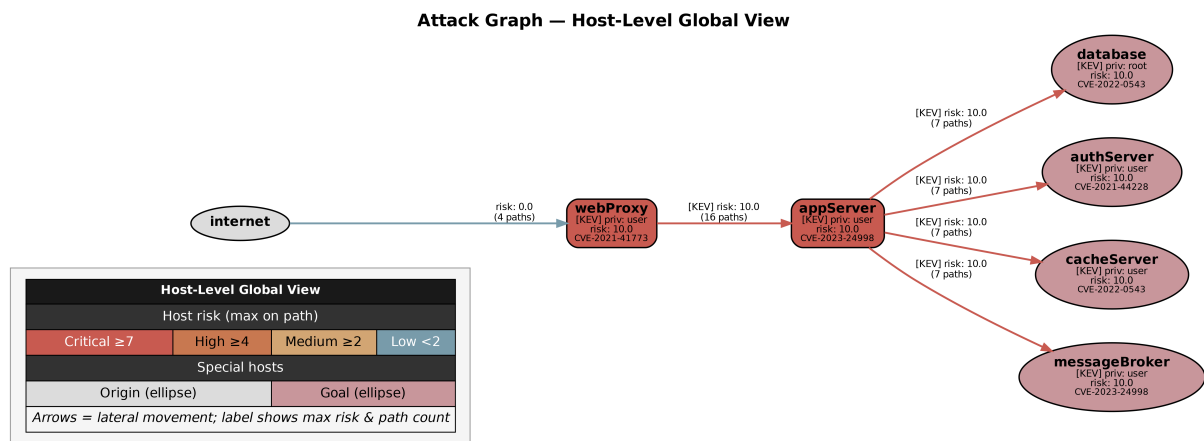


Figure 18: Illustrative host-level summary graph generated by the post-processing service.

The value of this graph is that it deliberately leaves out most of the detail. The detailed graph records every exploit and how each one enables the next, and it is precisely that completeness that makes it unreadable once the environment grows. The summary graph keeps only what an analyst needs in order to respond: which hosts can be reached, in what order, with what privilege, and which hosts most attack paths pass through. An analyst can read it directly instead of working through the full graph node by node. Together with the detailed annotated graph and the JSON report, it is the framework’s main decision support output, giving a compact and readable summary of how the organization’s risk changes over time.

4.4 Chapter Summary

This chapter described the proposed framework as a modular pipeline built around MulVAL. It detailed the design choices behind the logical reasoning engine and STRIDE integration, the flow of intelligence from ingestion to environment correlation, the orchestration of structural updates and the post-processing techniques used to score, compare and visualize attack paths. The implementation separates stable MulVAL interaction rules from the evolving fact base, allowing updated intelligence to change risk prioritization without unnecessary graph regeneration.

The resulting system provides an end-to-end workflow that transforms infrastructure descriptions and current threat intelligence into enriched attack graph outputs. The next chapter evaluates this workflow through case studies of increasing complexity and examines its dynamic behavior under structural changes, intelligence updates and asset reclassification.

Chapter 5

Case Study and Evaluation

This chapter evaluates the framework presented in Chapter 4 through a set of case studies of increasing complexity and realism. The evaluation has two complementary goals. First, it demonstrates that the pipeline operates as a complete workflow across representative network environments, transforming a plain network description into an enriched and prioritized attack graph informed by threat intelligence. Second, it assesses the dynamic behavior that distinguishes the proposed approach from a conventional, static attack graph generator. The evaluation therefore combines a qualitative analysis of each scenario with a quantitative comparison of structural metrics and dynamic re-prioritization.

The chapter is organized as follows: The experimental setup describes the methodology, the deployment environment and the configuration common to all experiments. Three simulated case studies are then presented in order of increasing complexity: a client-side exposure scenario, a multi-tier e-commerce deployment and an enterprise healthcare information system. A dedicated evaluation of the framework’s dynamic behavior then examines change detection, contextual re-scoring and asset reclassification. The chapter closes with a comparative discussion that relates the observed results to the objectives defined in Chapter 1.

5.1 Experimental Setup

The framework was evaluated using three simulated test scenarios, each describing a distinct network environment together with its asset inventory, software stack, network reachability and attacker assumptions. The scenarios were deliberately designed to span a range of sizes and topologies, from a flat set of independent client hosts to a deep, multi-tier server architecture, allowing the framework’s analytical output and dynamic behavior to be examined across environments of increasing complexity. They do not reproduce specific production environments. Instead, they represent recurring architectural and exposure patterns found in organizational infrastructures.

Their representativeness derives from the diversity of security situations they cover. The first scenario represents internet delivered client-side compromise of end-user systems. The second represents a conventional multi-tier web application with an internet-facing reverse proxy, application server, data services and an administrative workstation. The third represents a larger enterprise service architecture with a segmented front end, application tier, identity service, database, cache, message broker and clinician workstation. Together, the scenarios exercise direct exposure, client-side exploitation, lateral movement, tiered network segmentation, protected backend services and assets with different criticality profiles.

Every scenario was processed through the complete pipeline described in Chapter 4, deployed as a set of containerized services orchestrated with Docker Compose on a single host. Each experiment exercises the full sequence of layers: environment correlation binds vulnerabilities to assets, the orchestrator decides whether regeneration is required and invokes MulVAL, and the post-processing layer enriches, scores and visualizes the resulting graph. The persistent threat intelligence database, populated by the ingestion layer from the National Vulnerability Database, EPSS and the CISA KEV catalogue, was shared across all experiments, ensuring that every scenario was scored against the same intelligence snapshot.

A common configuration was applied throughout, so that differences in the results are attributable to the scenarios themselves rather than to parameter changes. The post-processing layer retained the ten highest risk attack paths per run, bounded path enumeration at a maximum depth of thirty (which also serves as a cycle guard), and aggregated up to two hundred paths into the host-level summary graph. Unless explicitly overridden per asset, the global default asset criticality was set to MEDIUM and the confidentiality, integrity and availability priorities were left equal, so that the impact multiplier remained neutral in the absence of a deliberate prioritization decision. Where defined, per asset criticality and CIA priorities were specified in the asset mapping file of each scenario.

To keep the analysis tractable and the attack graphs interpretable, the software versions assigned to each asset use explicit version-specific CPEs. This constrains correlation to named product releases but does not impose a fixed CVE count, since fresh intelligence snapshots can contain additional mappings. The scenarios use real software products identified by CPEs and are enriched with CVE, CVSS, EPSS and KEV data obtained through the pipeline, preserving a grounded mapping between products, identifiers and vulnerabilities. The results therefore demonstrate the operation and comparative prioritization behavior of the framework across representative architectural patterns. They do not measure the security posture of a real organization, reproduce the full vulnerability surface of a production environment, establish statistical generalizability or constitute a performance benchmark. Each scenario additionally

includes a STRIDE threat model constructed for this study from its defined assets, network connections, exposed services and trust boundaries. The model applies the STRIDE categories systematically to the components of each simulated architecture and translates the identified threats into supplementary MulVAL-compatible logical facts. It is representative of the threat classes associated with the selected architectural patterns, rather than a threat model elicited from a specific organization or validated against incident data.

For each scenario, the evaluation reports the structure of the generated attack graph, the asset inventory and attacker goals, the host-level summary view produced by the post-processing layer and the ranked set of highest risk attack paths with their associated threat intelligence attributes. The first scenario additionally includes a delta analysis, comparing successive risk reports to quantify how the prioritization responds to a change in the environment or in the underlying intelligence. These per-scenario results are then consolidated in the cross-scenario analysis.

5.2 Scenario 1: Client-Side Exposure

The first scenario models a common form of initial compromise: client-side exploitation of vulnerable end-user software. It consists of three independent victim hosts, each running a distinct vulnerable client application and each permitted to browse outbound to the internet, where the attacker is located. The hosts do not communicate with one another, so the scenario contains no lateral movement and instead isolates the client-side exploitation primitive. The topology is shown in Figure 19.

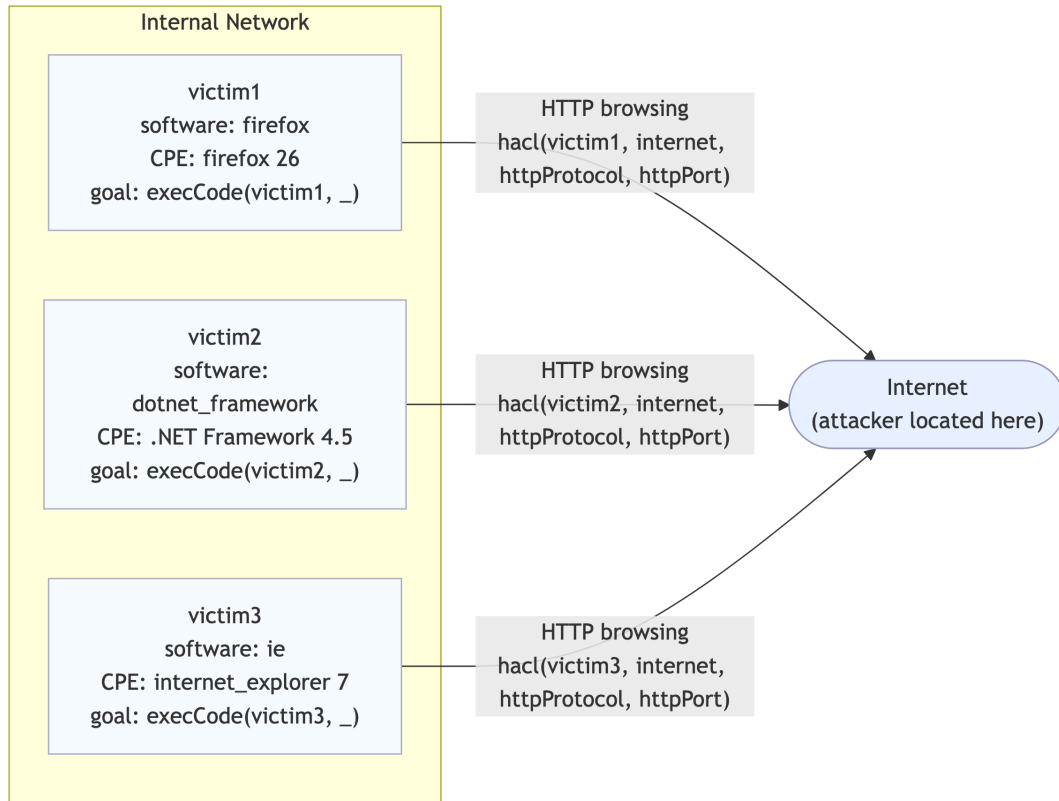


Figure 19: Scenario 1 topology: three independent victim hosts browsing outbound to an attacker-controlled internet.

The asset inventory comprises victim1 running Mozilla Firefox 26, victim2 running ABBYY FineReader 10.0 Pro and victim3 running Mozilla VPN 2.3. The attacker is located on the internet (`attackerLocated(internet)`) and the analysis goal is to obtain code execution on each host, expressed as `attackGoal(execCode(victimN, _))`. The current CPE mappings yield CVE-2019-20383 for FineReader and CVE-2022-0517 for Mozilla VPN. Firefox 26 has no CVE-derived match in the current database snapshot. In line with the conceptual design, the environment correlation layer injected the corresponding `vulExists` and `vulProperty` facts, modeling each matched weakness as a client-side `remoteClient` exploit with a privilege escalation consequence. The STRIDE model contributed one additional Elevation of Privilege threat per host, each mapped to the MITRE ATT&CK technique T1203 (Exploitation for Client Execution).

MulVAL generated an attack graph of 44 nodes and 59 edges, of which 15 are leaf nodes and 5 are vulnerability leaves, drawn from 2 unique CVEs. The post-processing layer enumerated 81 distinct root-to-leaf attack paths and achieved full enrichment coverage, meaning that every CVE node was successfully matched against threat intelligence data. The raw attack graph generated by MulVAL is available as the original `AttackGraph.pdf` in the project repository. The detailed annotated graph produced by the post-processing layer is shown in Figure 20.

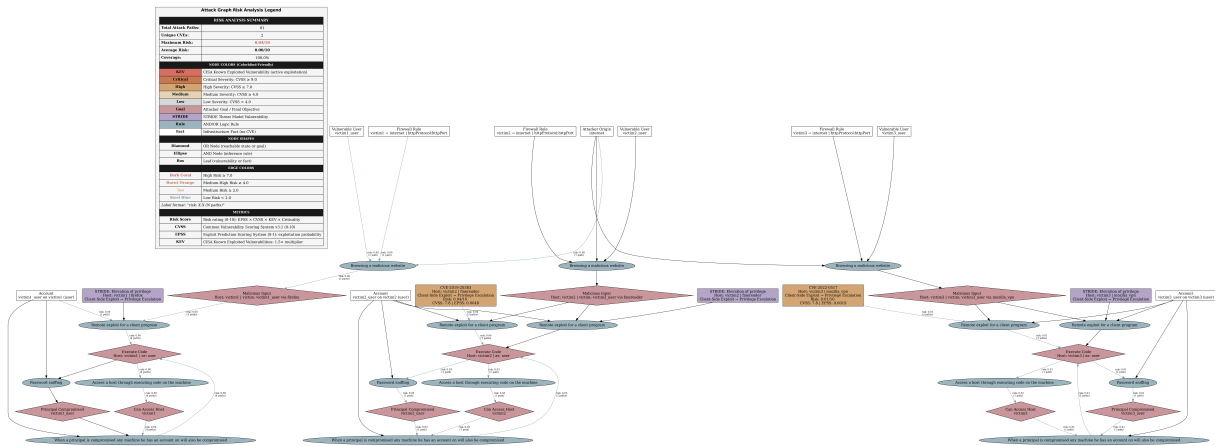


Figure 20: Scenario 1 detailed annotated attack graph produced by the post-processing layer, with nodes coloured by computed risk level.

The condensed host-level summary produced for this scenario is shown in Figure 21. It provides a compact view of each host's exposure and worst-case path risk.

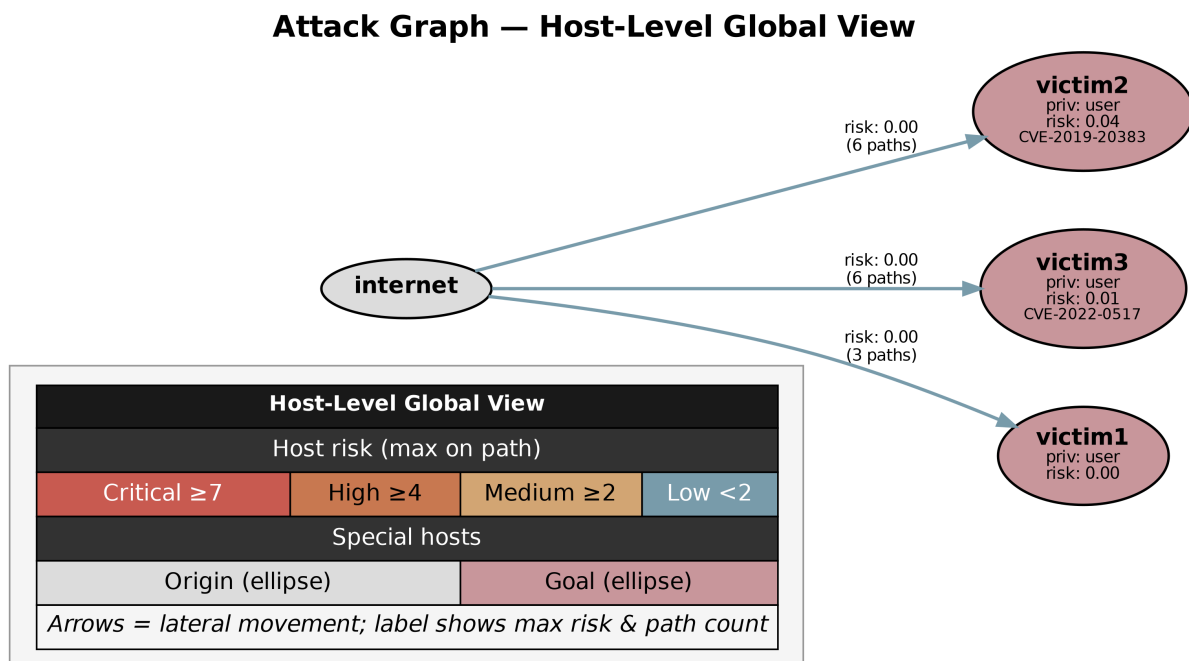


Figure 21: Scenario 1 host-level summary graph: each victim is reached directly from the internet, annotated with its worst-case path risk and exploited CVEs.

The summary view communicates the relative exposure of the three hosts. All three are reachable directly from the internet. victim2 carries the highest worst case path risk at 0.04 when rounded for visualization, victim3 follows at 0.01 and victim1 remains at 0.00.

The 0.00 labels on the internet-to-host arrows do not imply that the hosts have no exposure. Each arrow is labelled with the risk of the attack paths that produce the corresponding

transition. In this flat topology, the only transition is the attacker's initial network access. The paths that create it terminate at the network reachability precondition and therefore cross no vulnerability. The vulnerability that endangers each victim lies on the host itself and is recorded on the host node rather than on the incoming arrow. For each victim, the node value rather than the arrow label quantifies the exposure.

This ranking follows not from reachability, which is identical for all three hosts, but from the threat intelligence attributes of the underlying vulnerabilities. The highest-ranked path is driven by CVE-2019-20383 on `victim2`, with a CVSS base score of 7.8 and an EPSS score of 0.0048. CVE-2022-0517 on `victim3` has the same CVSS base score but a lower EPSS score of 0.00185, resulting in a lower path risk. The STRIDE derived fact on `victim1` remains structurally represented but has no associated CVE enrichment, so its paths receive a score of zero. This demonstrates that prioritization is governed by the estimated likelihood and impact of exploitation rather than by structural reachability alone.

5.2.1 Delta Analysis: Stable Baseline

Each run of the pipeline persists a single JSON artifact that contains two complementary objects. The first, the *risk report*, is the full result of the post-processing layer for the current graph. It records the graph version, a structural summary and the ranked list of highest risk attack paths, each annotated with its risk score and threat intelligence drivers. The second, the *delta report*, is written alongside it and compares the current risk report against the previously archived one. Listing 3 shows the initial portion of the current risk report for this scenario.

```

{
  "graph_version": "8c9799948c0633aa",
  "summary": {
    "total_nodes": 44,
    "total_leaf_nodes": 15,
    "total_vulnerability_leaf_nodes": 5,
    "unique_cves": 2,
    "total_attack_paths": 81,
    "enrichment_coverage_pct": 100.0
  },
  "top_attack_paths": [
    {
      "path_id": "05edc12ff37343b4",
      "goal": "execCode(victim2,user)",
      "risk_score": 0.0375,
      "path_length": 3,
      "leaf_cves": ["CVE-2019-20383"],
      "highest_risk_vuln": "CVE-2019-20383",
      "kev_on_path": false,
      "max_epss": 0.0048,
      "max_cvss": 7.8
    }
  ]
}

```

Listing 3: Current Scenario 1 risk report: the structural summary and the single highest risk path.

The most recent repeated execution is a steady-state run. Neither the environment nor the underlying intelligence changed, so the delta report contains no new or removed CVEs and no risk change. The resulting delta report is reproduced in Listing 4.

```

{
  "previous_version": "8c9799948c0633aa",
  "current_version": "8c9799948c0633aa",
  "vulnerability_delta": {
    "new_cves": [],
    "removed_cves": []
  },
  "risk_change": {
    "average_top_path_score_delta": 0.0,
    "new_top_paths": [],
    "removed_top_paths": []
  },
  "kev_additions": []
}

```

Listing 4: Scenario 1 delta report for an unchanged repeat execution.

The empty delta establishes a reproducible baseline for the current configuration. The unchanged graph version confirms that the structural model was not regenerated, while the zero average risk difference confirms that the repeated run used the same intelligence values. When a CPE mapping adds or removes a `vulExists` fact, the orchestrator detects the structural change and regenerates the graph before post-processing compares the resulting report with the archived baseline.

5.3 Scenario 2: Multi-Tier E-commerce Deployment

The second scenario increases both the size and the architectural depth of the environment, modeling a moderately sized multi-tier e-commerce deployment. An internet-facing nginx reverse proxy in a Demilitarized Zone (DMZ) fronts a Magento (Adobe Commerce) storefront, which is in turn backed by a MariaDB database and a Redis cache in a protected data tier. A back-office administrator workstation browses outbound to the internet, introducing a parallel client-side attack surface. Unlike the first scenario, this environment contains lateral movement, as network reachability rules permit an attacker to pivot from the proxy to the application server and from there into the data tier. The topology is shown in Figure 22.

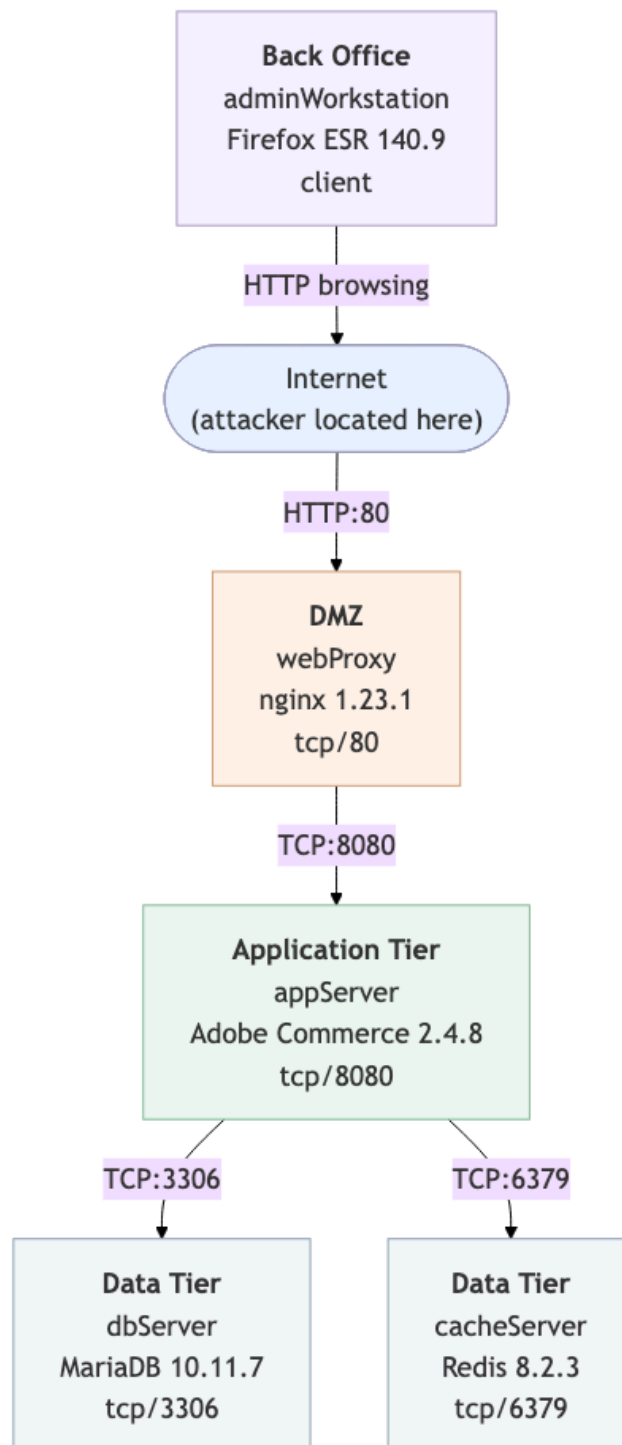


Figure 22: Scenario 2 topology: a multi-tier e-commerce deployment with a DMZ proxy, an application server, a data tier and a back-office workstation.

The attacker originates on the internet, and a code execution goal is declared for every host, allowing the framework to reason about the full set of reachable objectives rather than a single target. The per asset criticality and the confidentiality, integrity and availability priorities that drive the risk scoring are summarized in Table 4. Criticality was raised to HIGH for the

internet-facing and data bearing hosts, with the cache server retained at MEDIUM, while the CIA priorities were tuned per asset to reflect differing security concerns, for instance favouring confidentiality on the database and availability on the internet-facing proxy.

Table 4: Scenario 2: per asset criticality and confidentiality (C), integrity (I) and availability (A) priorities.

Asset	Criticality	C	I	A
webProxy	HIGH	MEDIUM	HIGH	HIGH
appServer	HIGH	HIGH	HIGH	HIGH
dbServer	HIGH	HIGH	HIGH	MEDIUM
cacheServer	MEDIUM	MEDIUM	MEDIUM	MEDIUM
adminWorkstation	HIGH	HIGH	MEDIUM	LOW

MulVAL produced a graph of 82 nodes and 125 edges, with 37 leaf nodes, 25 vulnerability leaves and 19 unique CVEs. The post-processing layer enumerated 1373 attack paths. All CVE identities were enriched, and every current CVE match carried the CVSS data needed to calculate a numeric path score. The raw attack graph generated by MulVAL is available as the original `AttackGraph.pdf` in the project repository. The detailed annotated graph produced by the post-processing layer is shown in Figure 23.

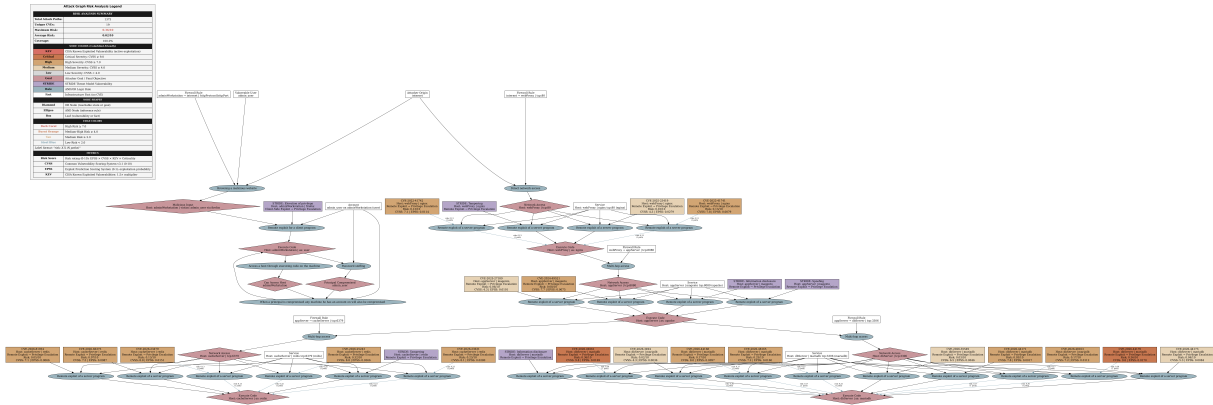


Figure 23: Scenario 2 detailed annotated attack graph produced by the post-processing layer, with nodes coloured by computed risk level.

The condensed host-level summary, shown in Figure 24, captures the chained nature of the environment: the attacker reaches webProxy from the internet, pivots to appServer and then fans out to the dbServer and cacheServer backends. The adminWorkstation contributes a STRIDE derived client-side threat, but its Firefox ESR CPE has no CVE match in the current intelligence snapshot. It therefore does not contribute a CVE-based risk path in this run. The

summary abstracts away individual reasoning steps while retaining attacker movement and risk concentration across the environment.

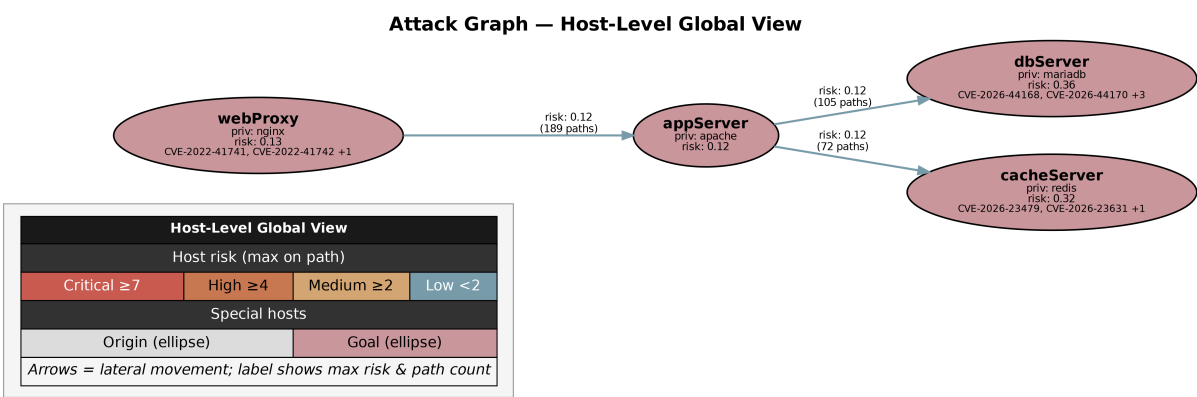


Figure 24: Scenario 2 host-level summary graph: lateral movement from the DMZ proxy through the application server into the data tier, with a separate workstation threat path.

A salient feature of the host-level summary graph developed in this work is the path count annotation added to each lateral movement edge. It reports how many of the selected attack paths traverse that host-to-host transition, exposing the bottlenecks of the modeled environment. The transition from webProxy to appServer is traversed by 189 paths. The onward transitions to the database and cache carry 105 and 72 paths respectively, identifying the application server as the central pivot for movement into the data tier. This information is operationally valuable: hardening or segmenting the application tier would invalidate a disproportionate share of the attack paths, an insight that the flat reachability view of a conventional attack graph does not surface.

The highest numeric path risk in this scenario is 0.3584, driven by CVE-2026-44170 on the MariaDB server. This CVE has a CVSS base score of 9.8 and an EPSS score of 0.01704. It is followed closely by CVE-2026-49261, also on MariaDB, with CVSS 9.8, EPSS 0.01691 and path risk 0.3557. The highest cache path is driven by CVE-2026-25243, with CVSS 8.8, EPSS 0.03663 and path risk 0.3228. None of the correlated CVEs is present in the KEV catalogue. The framework therefore distinguishes the larger structural exposure from the relatively low exploitation probabilities in the current intelligence snapshot. Were any EPSS score to rise, the same structural graph would be re-scored upward without regeneration.

5.4 Scenario 3: Enterprise Healthcare Information System

The third scenario represents the most structurally complex environment evaluated, modeling an enterprise hospital information system. An internet-facing HAProxy load balancer

terminates TLS and fronts an Apache Tomcat portal, which calls an Eclipse Jetty REST API. The API authenticates against a Keycloak identity provider and is backed by a PostgreSQL database, a Memcached cache and a RabbitMQ message broker carrying HL7 clinical events. A clinician workstation reads external email, providing a client-side entry vector. With eight hosts arranged across five tiers and the API server branching into four backend services, this is the most complex case among the evaluated scenarios for the correlation logic and path enumeration stage. The topology is shown in Figure 25.

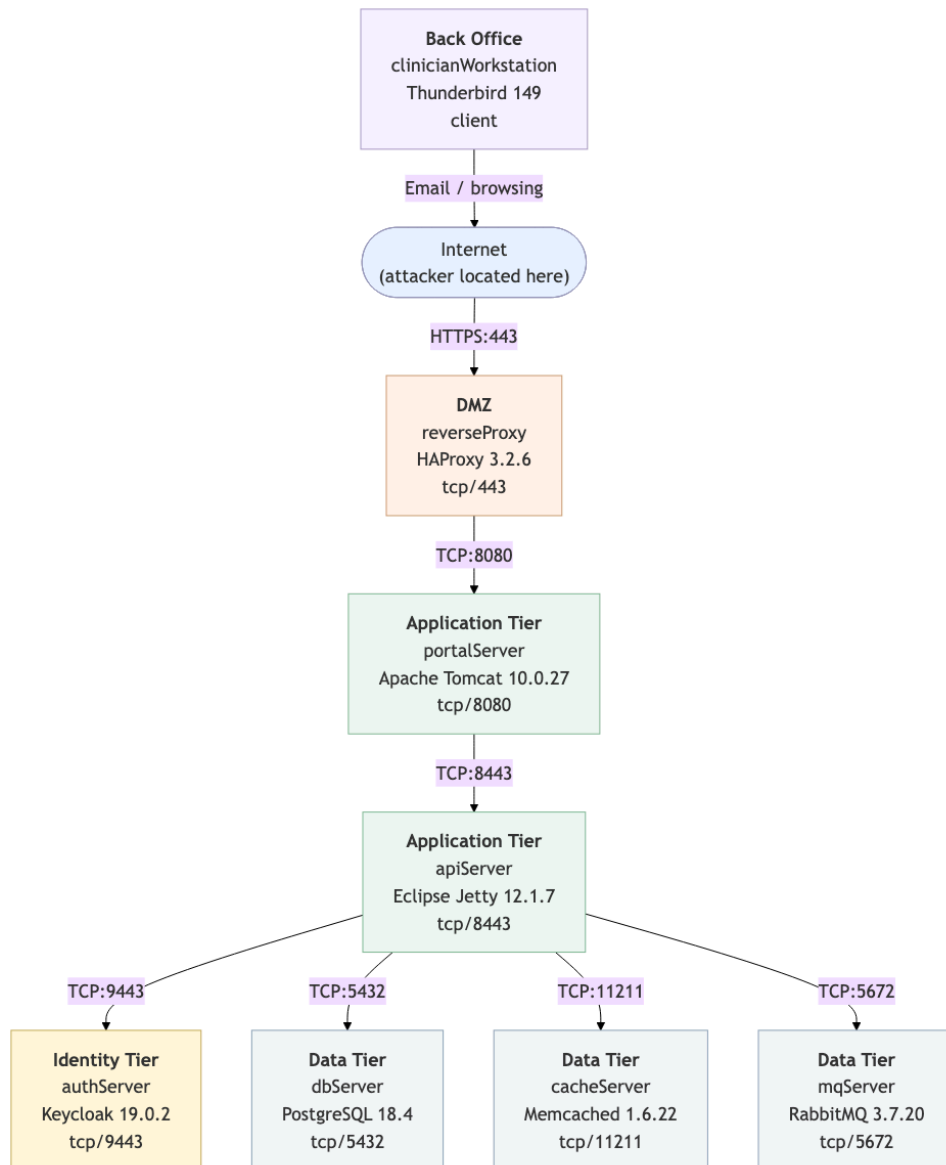


Figure 25: Scenario 3 topology: an eight-host healthcare information system with a DMZ load balancer, an application tier, an identity provider, a data tier and a clinician workstation.

A code execution goal was declared for all eight hosts. As in the previous scenario, criticality was assigned per asset, with the internet-facing, application, identity and data bearing hosts marked HIGH and the cache and message broker hosts retained at MEDIUM. The STRIDE model

for this scenario is the richest of the three, contributing eight threats spanning Tampering, Spoofing, Information Disclosure and Elevation of Privilege, each mapped to a corresponding MITRE ATT&CK technique and translated into supplementary logical facts.

The generated attack graph comprises 153 nodes and 246 edges, with 71 leaf nodes, 53 vulnerability leaves and 45 unique CVEs. Path enumeration yielded 14204 distinct attack paths, the largest of the three scenarios. All CVE identities were enriched, but CVE-2026-12611, CVE-2026-19203 and CVE-2026-19204 on Jetty have no CVSS score or vector in the current data source. They produce 114 paths with undefined risk, which are deliberately placed first in the risk report for manual review. The raw attack graph generated by MulVAL is available as the original AttackGraph.pdf in the project repository. The detailed annotated graph is reproduced in Appendix F. Only the host-level summary, shown in Figure 26, is presented here.

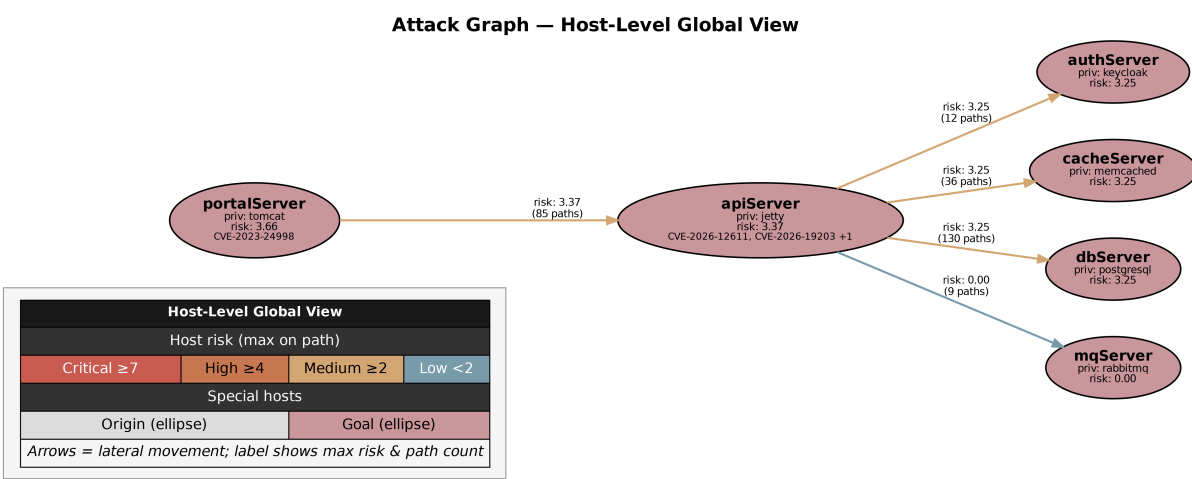


Figure 26: Scenario 3 host-level summary graph: deep lateral movement from the DMZ to the API server, which fans out into the identity and data tiers.

The summary graph presents this evaluated environment in a compact form. The attack progresses from `portalServer` to `apiServer`, which then branches to the four backend services. The edge labels record 85 paths from the portal to the API. The onward transitions from the API to the database, cache, identity provider and message broker carry 130, 36, 12 and 9 paths respectively. The API to database transition is the most frequent lateral movement in the summary, reflecting the broad branching of the application tier. The undefined Jetty paths are retained for review but are not assigned fabricated numeric scores. The highest numerically scored paths, summarized in Table 5, are driven by CVE-2023-24998 on the Tomcat portal. It has a CVSS base score of 7.5, an EPSS score of 0.48788 and no KEV status.

Table 5: Scenario 3: representative highest numerically scored attack paths and their threat intelligence drivers. Paths without CVSS are excluded from this numerical ranking.

Rank	Goal	Risk	Length	Primary CVE	CVSS	EPSS	KEV
1	execCode(portalServer, tomcat)	3.66	3	CVE-2023-24998	7.5	0.488	no
2	execCode(apiServer, jetty)	3.37	7	CVE-2023-24998	7.5	0.488	no
3	execCode(dbServer, postgresql)	3.25	11	CVE-2023-24998	7.5	0.488	no

The decreasing risk along the numeric ranking is a direct consequence of the directness factor $d(P)$ defined earlier in the scoring model. The portal goal, reached in 3 steps, is scored higher than the 11 step path to the database driven by the same vulnerability. Because $d(P) = \frac{1}{\log_2}(|P| + 2)$ decreases as a path lengthens, the same driver vulnerability necessarily yields a higher score on the shorter path. This is an arithmetic property of the scoring formula rather than a separate heuristic, yet it still lets the framework differentiate between paths that share an exploitation driver but differ in depth, supporting prioritization in deep architectures.

5.5 Evaluation of Dynamic Behavior

Beyond producing a single snapshot, the framework evolves the model as intelligence and asset definitions change. The mechanisms behind this were described in Chapter 4. This section evaluates them along the three dimensions identified in the design, drawing on the orchestrator’s change detection logic and the versioned risk reports archived across repeated runs of Scenario 1. The source code, scenario input data and generated outputs that support these tests are publicly available in the project repository.

Structural change detection is evaluated through steady-state runs and controlled changes to scenario facts. The repeated current Scenario 1 execution reported an identical graph version and an empty structural delta, so no regeneration occurred. Adding or removing a `vulExists` fact changes the normalized fact set and triggers regeneration. The detector automatically rebuilds the graph when the tracked `vulExists` facts or attack goals change. Changes to other scenario facts, including topology and service configuration, require forced regeneration. Adding or removing an asset is automatically detected when it changes one of the tracked fact sets.

Contextual re-scoring without regeneration is visible in the separation between the graph version and the risk report. The stable repeat execution recorded a zero risk delta because its intelligence inputs were unchanged. When a refreshed EPSS value changes the enrichment of a CVE already present in the graph, the post-processing layer recalculates the path scores while the graph version remains unchanged. This preserves the intended separation between structural regeneration and contextual risk analysis.

Asset reclassification provides a second contextual input, derived from the organization’s own assessment of an asset. Because the path risk score scales deterministically with the criticality factor α , reclassifying an asset re-ranks its paths with no change to the graph structure or to the underlying intelligence. Taking the highest-ranked path of Scenario 1, whose driver vulnerability and directness yield a criticality independent component of approximately 0.0536, Table 6 presents the resulting path risk and qualitative band for each level.

Table 6: Effect of asset reclassification on the highest-ranked path of Scenario 1, computed from its measured base risk. Reclassification re-ranks the path without graph regeneration.

Criticality	Factor α	Path risk	Risk level
LOW	0.4	0.0214	Low
MEDIUM (baseline)	0.7	0.0375	Low
HIGH	1.0	0.0536	Low
CRITICAL	1.3	0.0696	Low

Reclassifying the host from the baseline MEDIUM level to HIGH increases its highest path score, while a downgrade to LOW reduces it, with the structural model identical throughout. The selected CVEs retain a Low qualitative band because their EPSS probabilities are deliberately small. The change nevertheless affects prioritization without requiring the attack graph to be regenerated.

5.6 Comparative Discussion

The case studies and the dynamic behavior evaluation together allow the proposed framework to be contrasted with a conventional, static use of MulVAL and related to the objectives stated in Chapter 1. This comparison clarifies how the proposed pipeline extends structural reachability analysis with dynamic threat intelligence.

A conventional invocation of an attack graph generator answers a structural question: given a network configuration and a set of vulnerabilities, which hosts can the attacker reach and

through which exploit chains? This is valuable but incomplete. MulVAL adapters can supply later invocations with refreshed vulnerability repository data or scanner output, but each invocation still reasons over the particular fact base that it receives. On their own, these updates do not determine whether refreshed information is relevant to a specific environment, translate threat modeling output into facts, or separate a structural change from a contextual change. The proposed framework provides these surrounding capabilities. In each of the three scenarios, the structural reachability picture does not by itself provide a basis on which to decide where to act first. The same models, once enriched and scored by the proposed framework, provide a justified prioritization. In Scenario 1 the three equally reachable victims separate into distinct low-risk scores according to the estimated exploitation likelihood of their vulnerabilities, while the host without a matched CVE receives zero CVE risk. In Scenario 2 the application server is identified as the bottleneck through which most attack paths funnel. In Scenario 3 the API server emerges as the dominant pivot into the protected tiers. These conclusions require information beyond structural reachability alone.

The framework also addresses the evolution of risk over time. An individual graph is necessarily a snapshot, but it can be maintained through repeated correlation, selective regeneration and contextual rescoring. The dynamic evaluation showed that the model reprioritizes attack paths in response to revised EPSS scores while leaving the structural graph untouched, regenerates the graph when tracked vulnerability facts or attack goals change, and responds immediately to analyst-driven asset reclassification. Changes to topology or asset inventory can be incorporated through forced regeneration. The proposed framework therefore updates risk priorities when contextual inputs change while avoiding graph regeneration in those cases.

These results map directly to the objectives defined in Chapter 1. The end-to-end operation of the layered pipeline across three environments realizes the conceptual and architectural model linking dynamic threat intelligence to attack graphs (Objective 1) and is underpinned by the automated ingestion and correlation of CVE, CVSS, EPSS and KEV intelligence (Objective 2). The change detection, contextual re-scoring and reclassification behaviors evaluated in this chapter demonstrate the mechanisms for adaptive model evolution and re-prioritization (Objective 3). The host-level summary graphs, ranked path tables and delta reports transform the enriched models into actionable analytical insight (Objective 4). Finally, the three case studies validate the framework's functional behavior, dynamic update mechanisms and analytical usefulness in representative scenarios (Objective 5). Collectively, the evidence indicates that the framework achieves its central aim of advancing attack graphs from static representations into dynamic, intelligence-driven decision support tools.

5.7 Chapter Summary

This chapter evaluated the framework across three network scenarios with increasing complexity. The results showed that the complete pipeline generates enriched attack graphs from each environment, ranks paths using CVSS, EPSS, KEV status and asset criticality, and produces detailed and host-level views to support analysis. The dynamic evaluation further confirmed that changes to tracked vulnerability facts trigger regeneration, while revised intelligence and asset criticality re-rank paths without changing the logical graph.

Together, these findings demonstrate that the proposed approach extends static reachability analysis with current threat context and selective recomputation. The final chapter consolidates the dissertation's contributions, discusses the extent to which the objectives were achieved and outlines directions for future work.

Chapter 6

Conclusion and Future Work

This dissertation addressed an operational limitation of contemporary security modeling: the handling of changing threat intelligence around repeated attack graph generation. Although threat intelligence is published continuously and infrastructures evolve rapidly, each graph represents the facts available at the time of execution. This does not imply that MulVAL lacks mechanisms for receiving updated inputs. Its adapters can update vulnerability repository data and consume scanner output before later executions [5]. The work proposed, designed, implemented and evaluated a framework that correlates evolving threat intelligence with the modeled environment, determines when regeneration is required and updates risk priorities when it is not.

The proposed solution materializes this idea as a modular, event-driven pipeline of containerized services built around the MulVAL logical reasoning engine. It complements MulVAL's existing adapters by correlating intelligence with a CPE-based asset inventory, enabling analysis where vulnerability scans cannot be performed or are inappropriate. A persistent threat intelligence knowledge base is populated from authoritative sources, namely CVE and CPE records, CVSS severity metrics, EPSS exploitation probabilities and the CISA KEV catalogue. An environment correlation layer binds this intelligence to the modeled infrastructure and, in the same stage, translates the STRIDE threat model into logical facts, together producing the enriched fact base that MulVAL consumes. A central design decision underpins the entire framework: the interaction rules remain fixed while only the fact base evolves, which preserves compatibility with existing MulVAL rule sets and the favorable inference properties of the engine, while confining all adaptive behavior to the surrounding layers. An orchestration layer detects changes to vulnerability facts and attack goals, regenerating the graph for those changes and otherwise rescoring the existing model. Other scenario facts can be incorporated through forced regeneration. A post-processing layer enriches, scores, compares and visualizes the result. The case studies and the dynamic behavior evaluation in Chapter 5 demonstrated that this architecture operates end to end across environments of

increasing complexity and that it reprioritizes risk in response to evolving intelligence and analyst-driven asset reclassification without regenerating the graph for contextual updates.

6.1 Summary of Contributions

The contributions of this dissertation correspond directly to those outlined in Section 1.2 and are summarized below in light of the design, implementation and evaluation presented in the preceding chapters.

First, the work delivers a dynamic integration of threat intelligence into attack models. Rather than treating attack graphs as isolated snapshots, the framework defines explicit linkages between evolving threat data and the model’s fact base, so that vulnerability disclosures, severity metrics, exploitation probabilities and confirmed exploitation signals continuously inform the generated model. The case studies confirmed that this enrichment transforms an undifferentiated reachability picture into a justified prioritization, separating equally reachable targets according to the likelihood and impact of their underlying vulnerabilities.

Second, the work realizes mechanisms for adaptive model evolution and re-prioritization. The orchestrator’s two-phase change detection strategy regenerates the attack graph precisely when security-relevant facts change, while the post-processing scorer re-ranks attack paths whenever an intelligence signal, such as a revised EPSS score or a newly catalogued KEV entry, is updated. The evaluation showed that contextual re-scoring occurs without structural regeneration and that organization-driven asset reclassification takes effect immediately through the same contextual path. This validates the implemented separation between structural and contextual updates.

Third, the work provides a practical mechanism for integrating threat modeling with automated attack graph generation. The STRIDE model is validated against the modeled assets, and its threat categories and declared impacts are translated into logical facts consumable by MulVAL. This permits adversarial assumptions from qualitative threat modeling to participate in attack path generation alongside concrete vulnerability facts.

Taken together, the contributions establish an end-to-end method for incorporating evolving threat intelligence into logical attack graph analysis. The evaluation confirms that the framework can correlate vulnerability and threat modeling data with a modeled environment, distinguish structural changes from contextual risk updates, and present the resulting priorities through reports and host-level summaries. These results support its use as a research prototype for dynamic attack graph maintenance.

6.2 Future Work

While the framework achieves its stated objectives, several avenues remain open for extending its expressiveness, usability and analytical depth. The following directions focus on capabilities that would broaden its intelligence inputs and operational usefulness.

6.2.1 Richer cyber threat intelligence ingestion

The current ingestion layer consumes vulnerability-centric intelligence (CVE, CVSS, EPSS and KEV). A natural extension is to incorporate structured CTI expressed in standards such as STIX, enabling the framework to reason about threat actors, campaigns, malware and tactics, techniques and procedures. Integrating this richer, relationship-oriented intelligence would allow adversary context to inform attack-path prioritization and bring the model closer to genuine intelligence-driven decision making. The ingestion layer was deliberately designed to be extensible to such formats without affecting the downstream reasoning stages.

6.2.2 Adoption of CVSS 4.0

The scoring model currently derives vulnerability severity from the CVSS v3.1 base score, falling back to v3.0 when the former is unavailable. Since the release of CVSS 4.0, which refines the base metrics and introduces threat and supplemental dimensions better aligned with real-world exploitation, incorporating the version 4.0 score into the base risk computation would sharpen the severity signal and keep the framework aligned with the current standard. In such an extension, the v3.x scores would still need to be retained for the many vulnerabilities not yet rescored under 4.0, preserving coverage during the ongoing transition between versions.

6.2.3 Richer exploit consequences

The current translation represents all correlated vulnerabilities as privilege escalation events because it preserves MulVAL's standard interaction rules. Extending the framework to model data disclosure, data modification and service disruption as distinct logical consequences would require corresponding predicates and interaction rules, together with an extended translation layer. The CVSS confidentiality, integrity and availability ratings alone are insufficient to determine the concrete outcome of an exploit, so such a translation would require additional vulnerability semantics or analyst validation. This extension would allow the attack graph to capture goals other than code execution without treating every successful exploit as an escalation of privilege.

6.2.4 Automated translation of threat models

At present, the STRIDE threat model must be supplied to the framework as a structured JSON template, which the analyst authors by hand. A natural extension is to let the analyst keep performing the STRIDE analysis in their usual narrative form and to use a language model to translate that report into the JSON template the pipeline consumes, mapping the assets,

threats, categories and impacts to its machine-readable schema. In its minimal form this is a well-scoped translation task that already lowers the barrier to adoption. The same mechanism could later be extended so that the model also assists with the analysis itself, for instance by suggesting candidate threats or impacts for the analyst to review.

6.2.5 Learned asset criticality and CIA prioritization

Asset criticality and the relative importance of the confidentiality, integrity and availability dimensions are currently configured manually per asset. Future work could derive these values automatically from the threat model itself, for instance by inferring that an asset whose threat model emphasizes confidentiality concerns should weight confidentiality more heavily in its risk computation. A first realization could rely on deterministic rules mapping threat categories to CIA priorities, evolving subsequently toward a machine-learning approach trained to predict per-asset criticality from threat-modeling artifacts and historical data.

6.2.6 Risk-threshold filtering and risk acceptance

Two analyst-facing controls would make the output more actionable in day-to-day operations. The first is a configurable risk threshold: by specifying a minimum score, an analyst could restrict the generated graph and the reports to attack paths whose risk exceeds that value, for example suppressing everything below a score of 3.0 so that only the more pressing exposures are shown. The second is explicit risk acceptance, which would let an analyst mark specific vulnerabilities, paths or assets as accepted, so that known false positives or consciously tolerated risks are excluded from the prioritization rather than resurfacing on every run. Together these controls would focus the framework's attention on the risks an organization actually intends to act upon.

6.2.7 Graceful handling of the no-attack-path outcome

When MulVAL derives no attack path to any goal, the pipeline currently treats the result as a failure: the orchestrator exits with an error and the post-processing stage is skipped, so no report is produced. The absence of a reachable attack path is, however, a legitimate and often desirable outcome, and it does not imply that the environment is free of risk. Individual assets may still carry vulnerabilities that warrant remediation even when no end-to-end path currently chains them into a compromise of the declared goals. A more robust design would treat the empty graph as a first-class success: the pipeline would still run the post-processing stage and emit a report that records the vulnerabilities present on each asset, notes the absence of a viable attack path, and feeds the delta analysis, so that a transition from an exposed to a path-free environment is captured as a positive result rather than a broken run.

6.2.8 Scalability evaluation and optimization

The current evaluation verifies functional behavior and selective regeneration, but it does not establish the processing limits of the framework. Future work should measure execution

time, memory consumption and output size while systematically varying the number of hosts, services, vulnerabilities and feasible attack paths. The measurements should separate the costs of environment correlation, MulVAL inference, path enumeration and graph rendering.

The current host-level summary improves the readability of dense graphs by presenting only hosts, selected high-risk paths and risk annotations. It does not reduce the cost of generating the detailed graph or enumerating paths. Addressing computational scalability would require investigation of techniques such as incremental logical inference, selective graph generation, bounded or top-ranked path enumeration, and aggregation strategies that are applied before full graph visualization. Some of these approaches may require changes to MulVAL or its interaction rules, while others can be implemented in the surrounding pipeline. A dedicated evaluation is needed to determine which approach is appropriate and to quantify its effect.

6.2.9 Operational integration and visualization

Finally, the framework's practical impact would be strengthened by tighter integration with operational security tooling. Connecting it to SIEM platforms would enable observed events and alerts to drive model updates and would allow the resulting prioritization to feed back into operational workflows. In parallel, a dedicated graphical user interface for configuring scenarios, browsing the generated graphs and exploring delta reports would improve usability and make the dynamic, evolving nature of the model more immediately accessible to analysts.

These directions, ranging from richer intelligence sources and broader scoring inputs to automated translation of threat models, analyst-driven risk filtering and operational integration, would further consolidate the transition of attack graphs from static representations into living, intelligence-driven instruments for proactive cyber defense. They also outline a path from the current research prototype toward a more comprehensive operational decision-support capability.

Bibliography

- [1] J. Klos, “Attack Graph Generation Algorithm Implementation,” Bachelor's Thesis, 2018.
- [2] A. Nadeem, S. E. Verwer, S. Moskal, and S. J. Yang, “Enabling Visual Analytics via Alert-driven Attack Graphs,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 2021, pp. 2420–2422. doi: 10.1145/3460120.3485361.
- [3] OASIS Cyber Threat Intelligence (CTI) Technical Committee, “Introduction to STIX,” OASIS STIX Documentation. Accessed: 2026. [Online]. Available: <https://oasis-open.github.io/cti-documentation/stix/intro.html>
- [4] O. Gadyatskaya and S. Mauw, “Attack-Tree Series: A Case for Dynamic Attack Tree Analysis,” in *Proceedings of the 6th International Workshop on Graphical Models for Security (GraMSec 2019)*, in Lecture Notes in Computer Science, no. 11720. Springer, 2019, pp. 7–19.
- [5] MulVAL Project, “The MulVAL Project,” MulVAL. Accessed: 2018. [Online]. Available: <http://people.cs.ksu.edu/~xou/mulval/>
- [6] Ö. Aslan, S. S. Aktuğ, M. Ozkan-Okay, A. A. Yilmaz, and E. Akin, “A Comprehensive Review of Cyber Security Vulnerabilities, Threats, Attacks, and Solutions,” *Electronics*, vol. 12, no. 6, 2023, doi: 10.3390/electronics12061333.
- [7] OWASP Foundation, “Threat Modeling Process,” OWASP Foundation. Accessed: 2025. [Online]. Available: https://owasp.org/www-community/Threat_Modeling_Process
- [8] H. M. K. K. B. Herath and G. D. S. P. Wimalaratne, “A Collaborative Platform Featuring Visibility, Tracking, Monitoring and Awareness for Building Security In,” *International Journal of Computer and Communication Engineering (IJCCE)*, vol. 7, no. 4, pp. 145–166, 2018, doi: 10.17706/ijcce.2018.7.4.145-166.

- [9] Microsoft, “Threats - Microsoft Threat Modeling Tool,” Microsoft Learn - Azure Security. Accessed: 2025. [Online]. Available: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>
- [10] S. Koussa, “STRIDE Threat Modelling: What You Need to Know,” Software Secured. Accessed: 2026. [Online]. Available: <https://www.softwaresecured.com/post/stride-threat-modelling>
- [11] A. Shostack, “Uncover Security Design Flaws Using the STRIDE Approach,” *MSDN Magazine*, 2006, Accessed: 2025. [Online]. Available: <https://learn.microsoft.com/en-us/archive/msdn-magazine/2006/november/uncover-security-design-flaws-using-the-stride-approach>
- [12] Barnes Cybersecurity, “Cyber STRIDE - Threat Modeling Framework Explained,” Barnes Cybersecurity. Accessed: 2025. [Online]. Available: <https://barnes.ch/cyber-STRIDE.html>
- [13] A. Wolf, D. Simopoulos, L. D'Avino, and P. Schwaiger, “The PASTA Threat Model Implementation in the IoT Development Life Cycle,” in *INFORMATIK 2020*, R. Reussner, A. Koziolk, and R. Heinrich, Eds., in Lecture Notes in Informatics (LNI). Gesellschaft für Informatik, 2021.
- [14] A. Davis, “Enhancing System Security Through OCTAVE,” *Journal of Cybersecurity Systems Management*, vol. 17, no. 3, pp. 15–29, 2023.
- [15] A. Hammami, “The Art of Threat Modeling,” *Journal of Computer Sciences and Informatics*, 2024, doi: 10.5455/JCSI.20240710052550.
- [16] R. Miller and H. Thompson, “Identifying Assets in Threat Modeling,” *Journal of Strategic Security Solutions*, vol. 18, no. 2, pp. 120–134, 2022.
- [17] B. White, “Vulnerability Analysis in Software Development,” *Software Security Journal*, vol. 14, no. 3, pp. 165–180, 2022.
- [18] J. Brown and F. Zhao, “Technical Scope in Threat Modeling: PASTA's Approach,” *Journal of Information Security Research*, vol. 14, no. 1, pp. 112–130, 2022.
- [19] M. Garcia and T. Johnson, “Enhancing Web Application Security Using Microsoft Threat Modeling,” *Journal of Information Security*, vol. 14, no. 3, pp. 145–162, 2023.

- [20] K. Lee, “Evaluating Risk with DREAD and CVSS,” *Security Metrics Journal*, vol. 11, no. 3, pp. 234–249, 2023.
- [21] H. Miller, “OCTAVE: Comprehensive Risk Management for the Modern Enterprise,” *International Journal of Information Security Management*, vol. 30, no. 1, pp. 202–217, 2024.
- [22] B. Schneier, “Attack Trees,” *Dr. Dobbs Journal*, vol. 24, no. 12, pp. 21–29, 1999.
- [23] S. Mauw and M. Oostdijk, “Foundations of Attack Trees,” in *Information Security and Cryptology: ICISC 2005*, Springer, 2006, pp. 186–198.
- [24] Yuga, “Guide to Threat Modeling using Attack Trees,” Practical DevSecOps. Accessed: 2025. [Online]. Available: <https://www.practical-devsecops.com/threat-modeling-using-attack-trees/>
- [25] W. Wideł, M. Audinot, B. Fila, and S. Pinchinat, “Beyond 2014: Formal Methods for Attack Tree-Based Security Modeling,” *ACM Computing Surveys*, vol. 52, no. 4, 2019, doi: 10.1145/3331524.
- [26] H. S. Lallie, K. Debattista, and J. Bal, “A Review of Attack Graph and Attack Tree Visual Syntax in Cyber Security,” *Computer Science Review*, vol. 35, p. 100219, 2020, doi: 10.1016/j.cosrev.2019.100219.
- [27] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, “Automated Generation and Analysis of Attack Graphs,” in *Proceedings of the 2002 IEEE Symposium on Security and Privacy*, IEEE, 2002, pp. 273–284.
- [28] SentinelOne, “What are Attack Graphs? Key Components Explained,” SentinelOne Cybersecurity 101. Accessed: 2025. [Online]. Available: <https://www.sentinelone.com/cybersecurity-101/cybersecurity/attack-graphs/>
- [29] Red Hat, “What is a CVE?,” Red Hat Security Topics. Accessed: 2025. [Online]. Available: <https://www.redhat.com/en/topics/security/what-is-cve>
- [30] A. Gomstyn and A. Jonker, “What Is the Common Vulnerability Scoring System (CVSS)?,” IBM Think - Cybersecurity Topics. Accessed: 2025. [Online]. Available: <https://www.ibm.com/think/topics/cvss>
- [31] FIRST, “CVSS 3.0,” FIRST - Forum of Incident Response and Security Teams. Accessed: 2026. [Online]. Available: <https://www.first.org/cvss/calculator/3.0>

- [32] MITRE, “About CWE (Common Weakness Enumeration),” Common Weakness Enumeration. Accessed: 2025. [Online]. Available: <https://cwe.mitre.org/about/index.html>
- [33] C. Yianni, “Common Weakness Enumeration (CWE),” Intruder.io Glossary. Accessed: 2025. [Online]. Available: <https://www.intruder.io/glossary/what-is-cwe>
- [34] B. A. Cheikes, D. A. Waltermire, and K. Scarfone, “Common Platform Enumeration: Naming Specification Version 2.3,” NISTIR 7695, 2011. doi: 10.6028/NIST.IR.7695.
- [35] J. Jacobs, S. Romanosky, B. Edwards, I. Adjerd, and M. Roytman, “Exploit Prediction Scoring System (EPSS),” *Digital Threats: Research and Practice*, pp. 1–17, 2021, doi: 10.1145/3436242.
- [36] Cybersecurity and Infrastructure Security Agency (CISA), “Reducing the Significant Risk of Known Exploited Vulnerabilities,” CISA. Accessed: 2026. [Online]. Available: <https://www.cisa.gov/known-exploited-vulnerabilities>
- [37] SANS Institute, “What Is Open-Source Intelligence (OSINT)?,” SANS Institute Blog. Accessed: 2025. [Online]. Available: <https://www.sans.org/blog/what-is-open-source-intelligence>
- [38] M. Lee, *Cyber Threat Intelligence*. John Wiley & Sons, 2023.
- [39] V. Mavroeidis and S. Bromander, “Cyber Threat Intelligence Model: An Evaluation of Taxonomies, Sharing Standards, and Ontologies within Cyber Threat Intelligence,” in *2017 European Intelligence and Security Informatics Conference (EISIC)*, IEEE, 2017, pp. 91–98. doi: 10.1109/EISIC.2017.20.
- [40] Joint Task Force Transformation Initiative, “Guide for Conducting Risk Assessments (SP 800-30 Rev. 1),” NIST Special Publication 800-30 Revision 1, 2012.
- [41] A.-M. Konsta, A. Lluch Lafuente, B. Spiga, and N. Dragoni, “Survey: Automatic Generation of Attack Trees and Attack Graphs,” *Computers & Security*, vol. 137, p. 103602, 2024, doi: 10.1016/j.cose.2023.103602.
- [42] Z. A. Sheikh and Y. Singh, “A Hybrid Threat Assessment Model for Security of Cyber Physical Systems,” *IEEE Transactions on Dependable and Secure Computing*, pp. 1–15, 2023, doi: 10.1109/TDSC.2023.10053332.

- [43] K. H. Kim, K. Kim, and H. K. Kim, "STRIDE-based Threat Modeling and DREAD Evaluation for the Distributed Control System in the Oil Refinery," *ETRI Journal*, vol. 44, no. 6, pp. 741–753, 2022, doi: 10.4218/etrij.2021-0181.
- [44] S. Koussa, "Comparison of STRIDE, DREAD & PASTA," Software Secured. Accessed: 2026. [Online]. Available: <https://www.softwaresecured.com/post/comparison-of-stride-dread-pasta>
- [45] D. Tayouri, N. Baum, A. Shabtai, and R. Puzis, "A Survey of MulVAL Extensions and Their Attack Scenarios Coverage," *IEEE Access*, 2023, doi: 10.1109/ACCESS.2023.3257721.
- [46] P. H. Meland, D. G. Spampinato, E. Hagen, E. T. Baadshaug, K.-M. Krister, and K. S. Velle, "SeaMonster: Providing Tool Support for Security Modeling," in *Norsk Informasjonssikkerhetsskonferanse (NISK)*, NISK, 2008.
- [47] P. Ammann, D. Wijesekera, and S. Kaushik, "Scalable, Graph-Based Network Vulnerability Analysis," in *Proceedings of the 9th ACM Conference on Computer and Communications Security*, ACM, 2002, pp. 217–224.
- [48] S. Noel, S. Jajodia, B. O'Berry, and M. Jacobs, "Efficient Minimum-Cost Network Hardening via Exploit Dependency Graphs," in *Proceedings of the 19th Annual Computer Security Applications Conference*, IEEE, 2003, pp. 86–95.
- [49] S. Jajodia, S. Noel, and B. O'Berry, "Topological Analysis of Network Attack Vulnerability," in *Proceedings of the 2nd ACM Symposium on Information, Computer and Communications Security*, ACM, 2007, pp. 247–266.
- [50] S. Jajodia, S. Noel, P. Kalapa, M. Albanese, and J. Williams, "Cauldron: Mission-Centric Cyber Situational Awareness with Defense in Depth," 2011.
- [51] X. Ou, W. F. Boyer, and M. A. McQueen, "A Scalable Approach to Attack Graph Generation," in *Proceedings of the 13th ACM Conference on Computer and Communications Security*, ACM, 2006, pp. 336–345.
- [52] Y. Liu and H. Man, "Network Vulnerability Assessment Using Bayesian Networks," in *Proceedings of SPIE*, 2005, pp. 61–71.
- [53] K. Ingols, R. Lippmann, and K. Piwowarski, "Practical Attack Graph Generation for Network Defense," in *Proceedings of the 22nd Annual Computer Security Applications Conference*, IEEE, 2006, pp. 121–130.

- [54] M. L. Artz, "NetSPA: A Network Security Planning Architecture," M.Eng. Thesis, 2002. Accessed: 2026. [Online]. Available: <https://dspace.mit.edu/entities/publication/2a76a510-7aa2-49c7-90b4-6522cfafaff6>
- [55] H. Holm, T. Sommestad, M. Ekstedt, and M. Nordstrom, "CySeMoL: A Tool for Cyber Security Analysis of Enterprises," in *Proceedings of the 22nd International Conference and Exhibition on Electricity Distribution*, 2013, pp. 1–4.
- [56] S. Noel, E. Harley, K. H. Tam, M. Limiero, and M. Share, "CyGraph: Graph-Based Analytics and Visualization for Cybersecurity," 2016, *Elsevier*.
- [57] A. Nadeem, S. Verwer, S. Moskal, and S. J. Yang, "Alert-Driven Attack Graph Generation Using S-PDFA," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 2, pp. 731–746, 2022.
- [58] Z. Li, J. Zeng, Y. Chen, and Z. Liang, "AttackKG: Constructing Technique Knowledge Graph from Cyber Threat Intelligence Reports," 2021.
- [59] T. Li, Y. Jiang, C. Lin, M. S. Obaidat, Y. Shen, and J. Ma, "DeepAG: Attack Graph Construction and Threats Prediction with Bi-Directional Deep Learning," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 1, pp. 740–757, 2023.
- [60] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A Logic-Based Network Security Analyzer," in *Proceedings of the 14th USENIX Security Symposium*, USENIX Association, 2005.
- [61] OVAL Community, "Getting Started — The OVAL Community Guidelines," OVAL Community Guidelines. Accessed: 2026. [Online]. Available: <https://oval-community-guidelines.readthedocs.io/en/latest/getting-started.html>
- [62] B. Andersen and T. Fagerhaug, *Root Cause Analysis: Simplified Tools and Techniques*. ASQ Quality Press, 2006.
- [63] P. Kemper and W. H. Sanders, *Computer Performance Evaluation: Modelling Techniques and Tools*. Springer, 2003.
- [64] D. Byers, S. Ardi, N. Shahmehri, and C. Duma, "Modeling Software Vulnerabilities with Vulnerability Cause Graphs," in *Proceedings of the 22nd IEEE International Conference on Software Maintenance*, IEEE, 2006, pp. 411–422.

- [65] G. Sindre and A. L. Opdahl, "Eliciting Security Requirements with Misuse Cases," *Requirements Engineering*, vol. 10, no. 1, pp. 34–44, 2005.
- [66] L. Rostad, "An Extended Misuse Case Notation: Including Vulnerabilities and the Insider Threat," in *Proceedings of the 12th Working Conference on Requirements Engineering: Foundation for Software Quality*, 2006.
- [67] S. Ardi, D. Byers, and N. Shahmehri, "Towards a Structured Unified Process for Software Security," in *Proceedings of the 2006 International Workshop on Software Engineering for Secure Systems*, ACM, 2006, pp. 3–10.
- [68] A. Nadeem, S. E. Verwer, S. Moskal, and S. J. Yang, "Alert-Driven Attack Graph Generation Using S-PDFA," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 2, pp. 731–746, 2022, doi: 10.1109/TDSC.2021.3117348.
- [69] E. Rencelj Ling and M. Ekstedt, "A threat modeling language for generating attack graphs of substation automation systems," *International Journal of Critical Infrastructure Protection*, vol. 41, p. 100601, 2023, doi: 10.1016/j.ijcip.2023.100601.

Appendix A

Risk Scoring Configuration Reference

The asset criticality factor α multiplies the base risk in the path risk computation, so a value below 1.0 dampens and a value above 1.0 amplifies an asset’s contribution to the score. It is selected from one of four named levels. Table 7 lists the values applied throughout the evaluation.

Table 7: Asset criticality factor α per named level.

Criticality level	Factor α	Effect on the path risk score
LOW	0.4	Strongly reduces the score
MEDIUM (default)	0.7	Moderately reduces the score when no criticality is declared
HIGH	1.0	Leaves the score unscaled
CRITICAL	1.3	Amplifies the score

The remaining global parameters, applied uniformly across all experiments unless explicitly overridden per asset, are summarized in Table 8.

Table 8: Global scoring and enumeration parameters.

Parameter	Value	Role
KEV multiplier (k_{KEV})	1.5 / 1.0	Amplifies vulnerabilities listed in CISA KEV and is neutral otherwise
CIA multiplier (k_{CIA})	0.5-1.5	Weights confidentiality, integrity and availability impact per asset priority and is neutral (1.0) when priorities are equal
Top paths retained	10	Highest risk root-to-leaf paths kept in the summary report
Maximum path depth	30	Bound on path enumeration and cycle guard
Summary graph path cap	200	Highest risk paths aggregated into the host-level summary graph
Default asset criticality	MEDIUM	Applied when an asset declares no explicit criticality

The CIA multiplier is interpreted relatively: when the confidentiality, integrity and availability priorities of an asset are equal, the weighted average collapses to a neutral value of 1.0, so that impact prioritization only takes effect when an analyst deliberately favours one dimension over the others.

Appendix B

STRIDE-to-MulVAL Mapping

As part of the environment correlation layer, each validated STRIDE threat is translated into MulVAL facts. This step is necessary because MulVAL reasons only over its own fixed vocabulary of `vulExists/vulProperty` facts, a small set of exploitation primitives and a small set of consequences. A STRIDE threat, being a qualitative category with a declared impact, carries no meaning to the engine on its own. The mapping therefore turns each threat into a synthetic vulnerability fact, selecting the exploitation primitive from the STRIDE category and the consequence from the declared impact according to Table 9 and Table 10. This allows adversarial assumptions identified during threat modeling to participate in attack graph generation alongside vulnerability facts derived from CVEs, without changing MulVAL’s interaction rules.

The mapping operationalizes the implementation and does not claim a universal semantic equivalence between STRIDE categories and MulVAL primitives. STRIDE identifies the security property under threat, whereas a MulVAL primitive selects a rule family with specific attack preconditions. The defaults in Table 9 therefore represent the entry condition assumed for the case studies: categories modelled as attacks through a reachable service use `remoteExploit`, repudiation assumes an attacker has already obtained a local foothold and uses `localExploit`, and elevation of privilege is represented by the client-side rule family `remoteClient`, for which the handler supplies the required user-context facts. The purpose is to obtain a deterministic and inspectable translation into the limited MulVAL vocabulary. In an operational deployment, the primitive should instead be selected for each threat from its concrete attack vector and available preconditions, since the STRIDE category alone does not determine either of them.

Table 9: Default implementation mapping of STRIDE categories to MulVAL exploitation primitives. Unrecognized categories default to remoteExploit.

STRIDE category	MulVAL exploit primitive
Spoofing	remoteExploit
Tampering	remoteExploit
Repudiation	localExploit
Information Disclosure	remoteExploit
Denial of Service	remoteExploit
Elevation of Privilege	remoteClient

Table 10: Default implementation mapping of declared threat impacts to MulVAL consequences. Unrecognized impacts default to privEscalation.

Declared impact	MulVAL consequence
account_compromise	privEscalation
data_modification	privEscalation
log_tampering	privEscalation
data_exfiltration	privEscalation
privilege_escalation	privEscalation
unauthorized_access	privEscalation
remote_code_execution	privEscalation
arbitrary_code_execution	privEscalation
data_exposure	privEscalation
service_disruption	dos
availability_loss	dos
service_unavailability	dos

For a threat mapped to the remoteClient primitive, the base scenario may optionally include the minimal user-context facts (inCompetent() and hasAccount()) required by the corresponding client-side interaction rules. These facts are included only when the threat model assumes that the asset’s user can be induced to interact with malicious content. The STRIDE handler itself emits the vulExists() and vulProperty() facts.

Appendix C

Threat Intelligence Database Schema

Listing 5 reproduces the schema of the shared Threat Intelligence Database, organized around the CVE identifier as the principal key. It is provisioned automatically when the database container is initialized.

```

CREATE SCHEMA IF NOT EXISTS threat_intel;

CREATE TABLE IF NOT EXISTS threat_intel.cves (
    cve_id          VARCHAR(50) PRIMARY KEY,
    published_date   TIMESTAMP,
    last_modified_date TIMESTAMP,
    cvss_score_3_1   FLOAT,
    cvss_vector_3_1  VARCHAR(200),
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS threat_intel.cve_cpe_mapping (
    cve_id VARCHAR(50),
    cpe     VARCHAR(300),
    PRIMARY KEY (cve_id, cpe),
    FOREIGN KEY (cve_id) REFERENCES threat_intel.cves(cve_id)
);

CREATE TABLE IF NOT EXISTS threat_intel.epss (
    cve_id          VARCHAR(50) PRIMARY KEY,
    epss_score       FLOAT,
    epss_percentile  FLOAT,
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS threat_intel.kev (
    cve_id          VARCHAR(50) PRIMARY KEY,
    product          VARCHAR(200),
    date_added       DATE,
    known_ransomware_use BOOLEAN DEFAULT FALSE,
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Listing 5: Threat Intelligence Database schema (PostgreSQL).

Appendix D

Representative Scenario Input Artifacts

To illustrate the inputs consumed by the pipeline, this section reproduces the artifacts of Scenario 1 (Client-Side Exposure). The asset-to-CPE mapping associates each asset and service with its CPE identifier and, optionally, its criticality and CIA priorities.

```

{
  "assets": [
    {
      "asset": "victim1",
      "criticality": "HIGH",
      "cia_weights": {
        "confidentiality": "HIGH",
        "integrity": "MEDIUM",
        "availability": "LOW"
      },
      "client_software": [
        { "software_name": "firefox",
          "cpe": "cpe:2.3:a:mozilla:firefox:26:*:*:*:*:*:*" }
      ]
    },
    {
      "asset": "victim2",
      "criticality": "MEDIUM",
      "client_software": [
        { "software_name": "finereader",
          "cpe": "cpe:2.3:a:abbyy:finereader:10.0:-:pro:*:*:*:*" }
      ]
    },
    {
      "asset": "victim3",
      "client_software": [
        { "software_name": "mozilla_vpn",
          "cpe": "cpe:2.3:a:mozilla:vpn:2.3:*:*:*:*:*:*" }
      ]
    }
  ]
}

```

Listing 6: Scenario 1 asset_cpe_mapping.json.

The STRIDE threat model declares, for each asset, the threat category, impact and the corresponding MITRE ATT&CK technique.

```

{
  "system": "scenario1",
  "assets": [
    { "name": "victim1", "service": "firefox" },
    { "name": "victim2", "service": "finereader" },
    { "name": "victim3", "service": "mozilla_vpn" }
  ],
  "threats": [
    {
      "id": "T1",
      "stride": "Elevation of Privilege",
      "asset": "victim1",
      "impact": "remote_code_execution",
      "description": "Remote exploitation of the vulnerable client
software leads to code execution on victim1",
      "mitre_attack": "T1203 - Exploitation for Client Execution"
    }
  ]
}

```

Listing 7: Scenario 1 stride_definition.json (one of three analogous threats shown).

Finally, the enriched scenario.P combines the author-written base network description with the vulnerability facts that the environment correlation layer appends below it. In the base description, attackerLocated and hacl declare the attacker's location and the allowed network reachability, installed records the software on each host, the inCompetent/hasAccount pairs supply the user context required by the client-side rules, and attackGoal states the objectives. The correlation layer then appends one vulExists/vulProperty pair per matched vulnerability, drawn both from the CVEs correlated to each asset and from the STRIDE model (the threat_* facts).

```

/* --- Base network description (author-written scenario1MV.P) --- */
/* network connection */
attackerLocated(internet).
hacl(victim1, internet, httpProtocol, httpPort).
hacl(victim2, internet, httpProtocol, httpPort).
hacl(victim3, internet, httpProtocol, httpPort).
installed(victim1, firefox).
installed(victim2, finereader).
installed(victim3, mozilla_vpn).
/* users */
inCompetent(victim1_user).
hasAccount(victim1_user, victim1, user).
inCompetent(victim2_user).
hasAccount(victim2_user, victim2, user).
inCompetent(victim3_user).
hasAccount(victim3_user, victim3, user).
/* attack goals */
attackGoal(execCode(victim1, _)).
attackGoal(execCode(victim2, _)).
attackGoal(execCode(victim3, _)).
/* --- Vulnerability facts appended by the environment correlation layer
--- */
/* (CVE-derived and STRIDE-derived) */
vulExists(victim2, 'CVE-2019-20383', finereader).
vulProperty('CVE-2019-20383', remoteClient, privEscalation).
vulProperty(threat_t2_elevation_of_privilege, remoteClient, privEscalation).
vulExists(victim2, threat_t2_elevation_of_privilege, finereader).
vulExists(victim3, 'CVE-2022-0517', mozilla_vpn).
vulProperty('CVE-2022-0517', remoteClient, privEscalation).
vulProperty(threat_t3_elevation_of_privilege, remoteClient, privEscalation).
vulExists(victim3, threat_t3_elevation_of_privilege, mozilla_vpn).
vulProperty(threat_t1_elevation_of_privilege, remoteClient, privEscalation).
vulExists(victim1, threat_t1_elevation_of_privilege, firefox).

```

Listing 8: Enriched MulVAL scenario (scenario.P) for Scenario 1.

Appendix E

STIX Campaign Object Example

As discussed in the background chapter, STIX expresses each threat intelligence object in JSON, so that objects can be linked to one another to form a coherent intelligence picture. The following snippet illustrates a STIX 2.1 Campaign object.

```
{
  "type": "campaign",
  "id": "campaign--8e2e2d2b-17d4-4cbf-938f-98ee46b3cd3f",
  "spec_version": "2.1",
  "created": "2016-04-06T20:03:00.000Z",
  "modified": "2016-04-06T20:03:23.000Z",
  "name": "Green Group Attacks Against Finance",
  "description": "Campaign by Green Group against targets in the
financial services sector."
}
```

Listing 9: Example of a STIX 2.1 Campaign object [3].

Appendix F

Detailed Annotated Attack Graph Output

The following figure presents the detailed annotated attack graph for Scenario 3, produced by the post-processing layer. It shows the MulVAL attack graph enriched with threat intelligence and node-level risk scores. The corresponding raw MulVAL graph is available as the original `AttackGraph.pdf` in the project repository.

F.1 Scenario 3: Enterprise Healthcare Information System

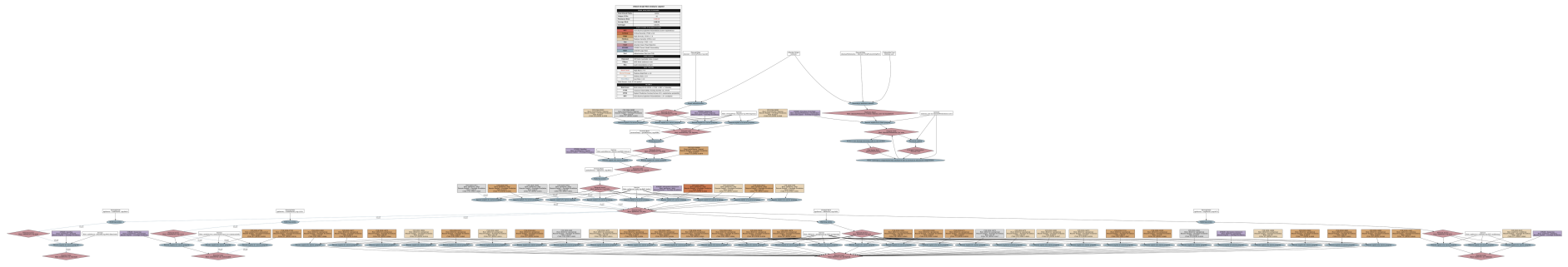


Figure 27: Scenario 3 annotated attack graph produced by the post-processing layer, with threat intelligence enrichment and per-node risk scores.

